Allen E. Paulson College of Engineering and Computing

Department of Electrical and Computer Engineering

EENG 4640 – EE/CpE Capstone Senior Project

EE and CpE Programs Required Course – 4 Credit Hours

Final Project Report – Fall/Spring 2026

Project Title:

Project Glass House

Team Members:

Tamera Gilmore – Team Lead, Hardware and Power Lead

tg14578@georgiasouthern.edu

Bryan Bosely – Antenna Selection, Node Design, and Wiring

bb28539@georgiasouthern.edu

Jack Jones – Software Lead, Signal Processing Lead

jj25347@georgiasouthern.edu

Perrin Cole Freeman – Embedded Systems Lead

pf03673@georgiasouthern.edu

Faculty Advisor(s): Dr. Seungmo Kim

Allen E. Paulson College of Engineering and Computing

Department of Electrical and Computer Engineering

EENG 4640 – EE/CpE Capstone Senior Project

EE and CpE Programs Required Course – 4 Credit Hours

TABLE OF CONTENTS

TABLE OF CONTENTS 2
LIST OF FIGURES 5
LIST OF TABLES 6
LIST OF ABBREVIATIONS 7
LIST OF EQUATIONS 9
ABSTRACT 10
1.0 INTRODUCTION 1
1.1 Objective of the Project 1
1.2 Limitations of the Project 2
2.0 PROJECT SUMMARY 3
3.0 PROJECT BACKGROUND 4
4.0 LITERATURE REVIEW 5
5.0 PROJECT GOALS AND ENGINEERING REQUIREMENTS 7
6.0 METHODOLOGY 8
6.1 Design Process 8
6.2 Tools and Resources 9
6.3 Design Rationale and Trade-Offs 10
6.4 Testing and Validation Plan 11
7.0 PROJECT SCHEDULING, TASKS, AND COSTS 16
7.1 Work Breakdown Structure 16
7.2 Task Assignments and Team Responsibilities 16
7.3 Project Schedule and Timeline 16
7.4 Risk Analysis 17
7.5 Bill of Materials 18
7.6 Milestones and Deliverables 19
8.0 SYSTEM DESIGN AND IMPLEMENTATION 22
8.1 System Overview 22
8.2 Hardware Design 22
8.3 Software Design 26
8.4 System Integration 30
8.5 Implementation and Testing Results 31
8.6 Summary 34
9.0 PROFESSIONAL COMPONENTS 35
9.1 Economic Concerns 35
9.2 Environmental Concerns 35
9.3 Sustainability Concerns 35
9.4 Manufacturability Concerns 36

9.5 Ethical Concerns 36
9.6 Health and Safety Concerns 36
9.7 Social Concerns 36
9.8 Legal or Political Concerns 37
10.0 STANDARDS AND COMPLIANCE 38
11.0 RESULTS 39
11.1 Error Analysis 41
12.0 DELIVERABLES 44
12.1 Hardware Deliverable 44
12.2 Embedded Firmware Deliverable 44
12.3 Host Software Deliverable 44
12.4 Public Project Repository and Website Deliverables 44
12.5 Documentation Deliverable 45
13.0 FAILED DESIGN APPROACHES 47
13.1 Four Node Architecture 47
13.2 Random Forest Model 47
13.3 ESP32-DevKitC-32UE 47
13.4 ASCII CSV Serial Format 48
13.5 ESP-NOW Mesh 48
13.6 On-Board Random Forest Inference 48
13.7 Single Centralized Antenna Array 49
13.8 Fixed-Threshold Variance Detection 49
13.9 Direct LCD Render From the Coordinator 49
13.10 5-Second Latency Target 50
14.0 DISCUSSION 51
15.0 CONCLUSION 53
15.1 Future Work 55
16.0 REFERENCES 57
APPENDICES 60
Appendix A. Full Programming Code 60
Appendix B. Circuit Schematics, Wiring Diagrams, and PCB Layouts 234
Appendix C. User Manual 236
Appendix D. IEEE Code of Ethics 242

LIST OF FIGURES

Figure 1. Gantt chart for project schedule

Figure 2a. Node enclosure, isometric interior view (SolidWorks)

Figure 2b. Node enclosure, side profile with power switch cutout (SolidWorks)

Figure 2c. Node enclosure, top-down interior view (SolidWorks)

Figure 2d. Node enclosure lid with node identifier marking (SolidWorks)

Figure 2e. Assembled perimeter node, photograph of completed hardware

Figure 3a. Central device, front view with LCD cutout and controls

Figure 3b. Central device, isometric view with lid removed showing internal components

Figure 3c. Central device, top-down view showing Raspberry Pi 4 and power bank layout

Figure 3d. Central device, underside base view showing mounting features

Figure 4. Perimeter node power and RF block diagram

Figure 5. Central device block diagram

Figure 6. ESP32 shouter/listener firmware flow diagram

Figure 7. Host data capture and processing flow diagram

Figure 8. Raspberry Pi inference and display flow diagram

Figure 9. Test environment floor plan with node placement and quadrant layout (Section 6.4)

Figure B-1. Perimeter node wiring diagram (Appendix B)

LIST OF TABLES

Table 1. Bill of Materials

Table 2. Task Assignments and Team Responsibilities

Table 3. Risk Analysis

Table 4. Milestones and Deliverables

Table 5. Per-Quadrant Identification Accuracy

Table 6. False-Positive Rate (No-Occupant Trials)

Table 7. Tracking Response for Moving-Subject Cases

Table 8. First-Lock Latency and Steady-State Refresh

Table 9. Engineering Requirement Verification

Table 10. Estimated Error Budget

Table 11. Project Repository and Website URLs

LIST OF ABBREVIATIONS

- ABS: Acrylonitrile Butadiene Styrene
- BOM: Bill of Materials
- BPM: Beats Per Minute
- COBS: Consistent Overhead Byte Stuffing
- CSI: Channel State Information
- CV: Coefficient of Variation
- DC: Direct Current
- DTR: Data Terminal Ready
- EMA: Exponential Moving Average
- ESP: Espressif Systems Platform
- FCC: Federal Communications Commission
- FFT: Fast Fourier Transform
- FreeRTOS: Free Real-Time Operating System
- GUI: Graphical User Interface
- HDMI: High-Definition Multimedia Interface
- HMM: Hidden Markov Model
- IEEE: Institute of Electrical and Electronics Engineers
- IPS: In-Plane Switching
- ISM: Industrial, Scientific, and Medical (band)
- JST: Japan Solderless Terminal
- LCD: Liquid Crystal Display
- LiPo: Lithium Polymer
- LVGL: Light and Versatile Graphics Library
- MAD: Median Absolute Deviation
- NDP: Null Data Packet
- OFDM: Orthogonal Frequency Division Multiplexing
- PCB: Printed Circuit Board

- PLA: Polylactic Acid
- PSD: Power Spectral Density
- RF: Radio Frequency
- RSSI: Received Signal Strength Indicator
- RTS: Request to Send
- SAR: Search and Rescue
- SMA: SubMiniature version A
- SNR: Signal-to-Noise Ratio
- SoC: System on Chip
- SoftAP: Software-Enabled Access Point
- STA: Station (Wi-Fi Client Mode)
- SVD: Singular Value Decomposition
- UART: Universal Asynchronous Receiver-Transmitter
- UDP: User Datagram Protocol
- USB: Universal Serial Bus
- UWB: Ultra-Wideband

LIST OF EQUATIONS

- Equation (1): Ohm's Law – $V = IR$
- Equation (2): Electrical Power – $P = VI$
- Equation (3): Kirchhoff's Current Law (KCL) – $\sum \quad I_{in} = \sum \quad I_{out}$
- Equation (4): Kirchhoff's Voltage Law (KVL) – $\sum \quad V_{in} = \sum \quad V_{out}$
- Equation (5): RF Power Density – $S = \frac{P_t \cdot G}{4\pi \cdot d^2}$
- Equation (6): CSI Representation – $H(f,t) = |H(f,t)| \cdot e^{j\theta(f,t)}$
- Equation (7): SVD Drift Cancellation – $log\ log\ |H| = U\,\Sigma\,V^T,\ \widetilde{H} = U_{2:}\,\Sigma_{2:}\,V_{2:}^T$
- Equation (8): Welch Breathing-Band Power Ratio – $r(f,L) = \frac{\sum_{0.15 \le f \le 0.40\ Hz} \quad P_{L,f}(f)}{\sum_{all\ f} \quad P_{L,f}(f)}$
- Equation (9): Soft Endpoint Attribution – $E_n = \sum \quad 0.5 \cdot w_L \cdot s_L,\ w_L = \sqrt{\frac{N_L}{N_{max}}}$
- Equation (10): HMM Forward Filter – $\alpha_t(q) = b_q(E_t) \cdot \sum_{q'} \quad a_{q',q} \cdot \alpha_{t-1}(q')$, $\hat{q}_t = arg\ m_q\ \alpha_t(q)$

ABSTRACT

Locating survivors trapped beneath collapsed structures is a time-critical task, and conventional search-and-rescue (SAR) methods remain constrained by line-of-sight requirements, hazardous conditions, and risk to rescue personnel. Project Glasshouse addressed this need by designing and prototyping a distributed radio-frequency (RF) sensing system that detects and localizes human presence through obstructions using Wi-Fi Channel State Information (CSI), without wearable devices or direct line-of-sight access.

The system integrates four battery-powered ESP32-S3 perimeter sensing nodes with a fifth ESP32-S3 coordinator node connected to a Raspberry Pi 4 central processing hub via a Consistent Overhead Byte Stuffing (COBS)-encoded USB serial link. A Python signal processing pipeline applies Hampel outlier rejection and Singular Value Decomposition (SVD) drift cancellation to the per-link CSI, computes a Welch periodogram on the breathing band (0.15–0.40 Hz) using the Fast Fourier Transform (FFT), attributes the resulting per-link signal scores to the four perimeter nodes through soft endpoint weighting, and smooths the per-node scores through a four-state Hidden Markov Model (HMM) forward filter to produce a four-quadrant classification. A continuous (x, y) position estimate is derived in parallel from wall-pair perturbation ratios and smoothed by an exponential moving average (EMA), with the combined output rendered in real time on a Liquid Crystal Display (LCD) screen.

Functional validation in a controlled indoor environment yielded a quadrant identification accuracy of 80 percent (32 of 40 trials) and a false-positive rate of 7.5 percent across the unoccupied-window cases. Each perimeter node sampled CSI at approximately 50 Hz and the display refreshed about once per second, but the breathing-band analytical pipeline required roughly 30 seconds of CSI to fill its long window before the first quadrant could lock in. That 30-second initial latency exceeded the 5-second engineering target defined in Section 5.0, and the gap is discussed in Section 11. The system was validated across a 15 m by 15 m four-node sensing footprint, with adjacent perimeter nodes spaced about 10 to 11 meters apart. Operation in the 2.4 GHz ISM band gave a nominal through-air range of about 100 meters in open environments, and detectable perturbations extended roughly 1 to 2 meters past the node perimeter in the test space. The complete five-node system was assembled for under $450, less than one percent of the $48,000-plus per-unit cost of commercial through-wall rescue radar.

Future work focuses on shortening the window-fill time, adding nodes for finer spatial resolution, and surfacing physiological output for triage in multi-victim scenarios.

1.0 INTRODUCTION

Structural collapses caused by natural disasters and building failures frequently trap victims beneath debris where they cannot be seen, heard, or reached without significant excavation. Survival probability decreases sharply with time following a structural collapse, and SAR teams consequently face acute pressure to locate victims quickly while minimizing risk to personnel who must operate in unstable environments [1].

Existing technologies each carry significant limitations. Trained search dogs are constrained by environmental contamination and handler fatigue [2]. Thermal cameras require near-direct line of sight and fail when victims are buried under significant debris [3]. Acoustic sensors depend on the victim being conscious and able to signal [4]. Commercial RF-based rescue radar systems provide through-wall detection capability, but the per-unit cost of over $48,000 makes widespread deployment impractical for most agencies [5].

Radio frequency signals offer an alternative because they penetrate non-metallic materials and are measurably disturbed by human presence through reflection, absorption, diffraction, and multipath scattering. Project Glasshouse developed a distributed RF-based human localization system that uses Wi-Fi CSI and on-board edge signal processing to generate real-time estimates of victim location for SAR operations, and removes the need for rescuers to enter unstable structures. The remainder of this report covers the project background and literature review in Sections 3.0 and 4.0, methodology in Section 6.0, system design and implementation in Section 8.0, professional components and standards compliance in Sections 9.0 and 10.0, and experimental results and conclusions in Sections 11.0 through 15.0.

1.1 Objective of the Project

The primary objective was to design, build, and demonstrate a distributed RF sensing system that detects and localizes human presence within a monitored area. The system was required to collect CSI data across four spatially distributed ESP32-S3 perimeter nodes, perform on-board signal processing to extract motion features, and relay the results to a central Raspberry Pi 4 for visualization on an LCD display accessible to first responders. Supporting objectives included battery-powered autonomous node operation, a total system cost under $450, and compliance with

IEEE 802.11 Wi-Fi standards and Federal Communications Commission (FCC) RF exposure limits.

1.2 Limitations of the Project

Several constraints bound system performance. The signal processing approach is environment-specific: CSI patterns vary with room geometry, building materials, and node placement, so performance observed in development may differ in real disaster environments that contain dense concrete or metallic debris. Spatial resolution is limited by the four-node prototype, which restricts localization to broad quadrants rather than precise coordinates. FCC regulations restrict unlicensed operation to the 2.4 GHz ISM band, which attenuates more through structural materials than lower-frequency alternatives and may reduce detection reliability in heavily constructed collapse environments. Each node relies on a single lithium polymer (LiPo) battery with no redundancy, and the Raspberry Pi is a single point of failure with no backup processing unit.

2.0 PROJECT SUMMARY

Project Glasshouse is a distributed RF sensing network consisting of four battery-powered ESP32-S3 perimeter nodes and one coordinator ESP32-S3 connected to a Raspberry Pi 4 via USB. Each perimeter node operates as a Wi-Fi station (STA) client on a local network hosted by the coordinator and transmits and receives CSI packets at approximately 50 Hz. Processed data is forwarded to the coordinator, which streams it over USB serial to the Raspberry Pi, where a Python pipeline decodes incoming frames, aggregates link-level motion reports, and applies a zone detection algorithm to estimate which of four spatial quadrants is most likely occupied. The result is rendered in real time on an LCD display, which gives first responders an immediate visual indication of probable survivor location without the need for them to enter the structure.

The system was designed with sustainability and deployability in mind. Node enclosures were 3D-printed in recyclable polylactic acid (PLA), and the distributed battery-powered architecture eliminates dependency on grid power or cloud infrastructure. RF emissions remain well below IEEE C95.1 and FCC Title 47 exposure limits. At under $450 for a complete kit, a single agency budget that would purchase one commercial radar unit could instead equip over 100 rescue teams, which makes RF-based victim sensing broadly accessible as a standard SAR utility.

3.0 PROJECT BACKGROUND

RF sensing using commodity Wi-Fi hardware has emerged as a practical approach to fill the gap left by existing SAR technologies. When Wi-Fi signals propagate through an environment, human presence introduces measurable disturbances in the wireless channel through reflection, absorption, diffraction, and multipath scattering. These disturbances are captured using CSI, a physical-layer measurement available on modern Wi-Fi chipsets that provides amplitude and phase data across multiple Orthogonal Frequency Division Multiplexing (OFDM) subcarriers [6]. Unlike Received Signal Strength Indicator (RSSI) measurements, which provide only a single coarse power value, CSI offers fine-grained per-subcarrier information sensitive to subtle environmental changes, and is therefore well suited for through-wall human detection [7].

The ESP32-S3 microcontroller is a System on Chip (SoC) that combines a dual-core processor, Wi-Fi radio, and flash memory in a single package, and it provides an accessible platform for CSI-based sensing. It supports CSI extraction through the ESP-IDF framework, which permits capture of measurements across available Wi-Fi subcarriers, and its integrated processing capability supports on-board signal analysis in battery-powered deployments [8]. Variance-based change detection identifies shifts in CSI amplitude statistical properties indicative of motion, while FFT analysis of phase components extracts periodic micro-motion signatures associated with breathing and cardiac activity [9]. Lithium polymer batteries supply direct current (DC) power to each sensing node through an Adafruit Powerboost 1000C. The power delivered to each node is given by Equation (2), and regulated voltages are distributed across the power subsystem in accordance with Kirchhoff's Voltage Law, stated in Equation (4). Node-level current budgets are managed using Kirchhoff's Current Law, stated in Equation (3) [10].

4.0 LITERATURE REVIEW

Advanced RF sensing systems have demonstrated the ability to reconstruct human posture through walls. Zhao et al. showed that RF reflections combined with deep learning can reconstruct 3D human body meshes [11], and Raja et al. demonstrated that multifrequency RF features improve 3D pose estimation robustness [12]. Both studies confirm that RF sensing can extract high-dimensional spatial information, although they rely on specialized hardware and complex models that are impractical for SAR field deployment.

For SAR specifically, the detection of stationary or low-motion victims is critical. Studies have demonstrated RF detection of subtle human movement behind walls, including micro-motion such as breathing [13], and Uysal and Filik proposed an improved RF sensing framework for through-wall human detection in cluttered indoor environments [7]. Lower-frequency systems operating at 433 MHz have also detected indoor occupants from outside a building, which demonstrated feasibility for emergency scenarios [14].

RF sensing has been applied to activity recognition and gait analysis. Hsu et al. extracted gait velocity and stride length from surrounding RF signals [15], and CSI-based systems have classified activities such as walking, sitting, and falling with high accuracy using machine learning [6]. Open-source frameworks that combine CSI sensing with deep learning have also estimated 3D human pose, which shows further potential for spatial awareness applications [16].

FFT analysis of CSI phase data is an effective technique for extracting periodic micro-motion signatures associated with physiological activity. Human respiration and cardiac activity produce periodic physical displacements that induce measurable phase variations in Wi-Fi CSI signals. By applying FFT to the time-domain CSI phase signal, these periodic components can be isolated as spectral peaks within characteristic frequency bands: 0.1–0.5 Hz for respiration and 0.8–2.5 Hz for cardiac activity. Ishabakaki et al. confirmed that FFT-based spectral analysis of RF phase data is a well-established and effective approach for detecting breathing and heart rate without physical contact [9]. Compared to machine learning approaches, FFT-based methods require no pre-collected training data, operate effectively from the first frames of acquisition, and adapt naturally to new environments. These properties are directly suited to SAR deployment conditions where the environment is unknown and time-critical.

RF localization has also been proposed for emergency occupancy estimation. Indoor emergency response systems have demonstrated the feasibility of RF-based occupancy estimation [17], and unmanned aerial vehicles and robotic RF platforms have been explored for SAR operations in which drones equipped with RF sensors detect victims in hazardous environments [18], [19].

Antenna performance significantly influences RF sensing systems. Industry guidelines describe how frequency, material properties, and antenna placement affect penetration and sensing accuracy [20]. A Universal 2.4 GHz 7 dBi high-gain SMA antenna was selected for each node based on its compatibility with the ESP32-S3 SMA connector, its high gain relative to a standard dipole, and its low cost as a commodity component. During development, both omnidirectional and directional antenna configurations were evaluated, and the 7 dBi SMA antenna was adopted for the final prototype based on observed CSI variance performance. Vertical antenna orientation reduces multipath effects from ground reflection, which improves measurement accuracy and direction-inference capability.

The IEEE C95.7 standard defines recommended practices for electromagnetic energy safety programs from 0 Hz to 300 GHz. It complements the IEEE C95.1 exposure limits with an organizational framework that minimizes human exposure risk and addresses personnel training [21]. Compliance with these standards ensures that transmitted power levels remain safe for both operators and potential victims in disaster environments.

Despite these advances, key gaps remain. Many existing systems rely on single transmitter-receiver pairs that provide only binary presence detection, while distributed low-cost localization using commodity hardware is still limited. Most prior work targets smart homes or healthcare monitoring rather than SAR scenarios, and few systems provide spatial visualizations useful for rescue personnel. These gaps motivate the design of Project Glasshouse: a portable, distributed, low-cost RF localization system that uses multiple ESP32-S3 nodes and deterministic FFT-based data fusion to generate a real-time quadrant occupancy estimate for emergency response.

5.0 PROJECT GOALS AND ENGINEERING REQUIREMENTS

The primary goal of Project Glasshouse was to design and deploy a distributed RF-based human localization system that detects and estimates the location of a person within a monitored area using Wi-Fi CSI, without line-of-sight access or entry into the monitored structure, and that displays the result in real time on a portable LCD screen for first responders.

Engineering requirements were defined as measurable, traceable targets. The system was required to: (1) achieve a CSI sampling rate of approximately 50 Hz across four perimeter nodes; (2) maintain end-to-end latency from signal acquisition to display update of five seconds or less; (3) correctly identify the occupied spatial quadrant with a minimum accuracy of 75% in a controlled indoor test environment; (4) keep total system cost under $450 for a complete five-node kit; (5) operate all perimeter nodes on battery power without grid dependency; (6) comply with IEEE 802.11, IEEE C95.1-2019, and FCC Title 47 regulations; and (7) demonstrate the complete end-to-end pipeline in a controlled environment as proof of concept. These requirements informed hardware selection, algorithm design, and validation methodology throughout the project lifecycle.

6.0 METHODOLOGY

6.1 Design Process

Design followed an iterative engineering process that began with requirements definition: non-line-of-sight operation, battery-powered autonomous nodes, real-time spatial output, and total cost under $450. A literature review evaluated candidate RF sensing technologies and led to the selection of Wi-Fi CSI for its availability on commodity microcontroller hardware, its fine-grained per-subcarrier measurement capability, and its demonstrated effectiveness in through-wall human detection.

Hardware prototyping began with the ESP32 platform, chosen for its integrated Wi-Fi radio, its CSI extraction support through the ESP-IDF framework, and its low power consumption. Early prototypes validated basic CSI packet capture using a two-board ESP-NOW configuration. The architecture evolved into a five-node design in which a dedicated coordinator ESP32-S3 runs an open Wi-Fi SoftAP that all four perimeter nodes associate with as STA clients. Each perimeter node periodically transmits a Null Data Packet (NDP) probe at approximately 50 Hz to stimulate its three peers, captures CSI from peer responses through a promiscuous-mode Wi-Fi callback, and forwards each binary CSI frame to the coordinator over UDP. The coordinator bridges the aggregate stream to the host through a COBS-encoded USB-Serial link at 921,600 baud.

The software pipeline was developed in parallel. The host detector decodes each UDP frame, buffers per-link I/Q records into a six-second short window for variance gating and a 90-second long window for the breathing-band analysis, and applies a sequence of analytical operations on each emission tick: Hampel outlier rejection on per-link mean amplitudes, Singular Value Decomposition (SVD) drift cancellation that removes the dominant temporal mode per link (Equation 7), and a Welch periodogram across the breathing band 0.15–0.40 Hz (Equation 8). The resulting per-link signal scores are attributed to the four perimeter nodes through soft endpoint weighting, with each link's contribution scaled by the square root of its packet count relative to the maximum across the same window (Equation 9), and the per-node scores are smoothed by a four-state Hidden Markov Model (HMM) forward filter (Equation 10) to produce the displayed quadrant. The pipeline is purely deterministic and physics-driven, which removes the need for labeled training data or model retraining in new environments.

6.2 Tools and Resources

Project Glasshouse used both software and hardware tools across design, fabrication, bring-up, debugging, testing, and documentation. The tables below list each tool and the activity it supported.

6.2.1 Software Tools

Tool	Purpose
GitHub	Version control, branch-based development, issue tracking, and final source archive.
Arduino IDE	ESP32-S3 firmware authoring, compilation, and flashing for the perimeter and coordinator nodes.
ESP-IDF (v5.x)	Underlying SDK invoked through Arduino IDE for the low-level Wi-Fi and CSI extraction APIs.
Visual Studio Code	Primary editor for the Python pipeline and project documentation, with the Pylance and Markdown extensions.
Python 3.11 (NumPy, SciPy, pyserial, Pillow)	Host pipeline: COBS decode, Hampel/SVD/Welch analysis, soft endpoint attribution, HMM filter, and LCD render.
SolidWorks 2023	Mechanical design of the perimeter-node and central-device enclosures; produced the .STL files used for FDM 3D printing.
Wireshark / tcpdump	UDP packet inspection during four-node bring-up. This is how the 512-byte truncation defect was found.
Microsoft Word and Excel	Final report authoring, milestone tracking, and tabular accuracy and latency analysis.
Next.js and Vercel	Public project website used as a documentation deliverable (see Section 12).

6.2.2 Hardware Tools

Tool	Role in design, testing, or documentation
Saleae Logic 8 logic analyzer	Captured the COBS-encoded UART stream between the coordinator and the Raspberry Pi at 921,600 baud during the 512-byte truncation debug.
Rigol DS1054Z 4-channel oscilloscope	Verified the PowerBoost 1000C output rail under transmit-burst load and checked the ESP32-S3 reset and POR timing.

Fluke 117 digital multimeter	Continuity, voltage, and current spot checks during board bring-up. Also used to log battery voltage drop under load.
Korad KA3005P bench power supply (0-30 V, 5 A)	Powered nodes from the bench so that thermal and current measurements were not coupled to the LiPo state of charge.
Hakko FX-888D soldering station	Assembly of perimeter and coordinator carrier boards. Used for SMA antenna connector and JST battery lead rework.
Creality Ender 3 V2 FDM 3D printer (PLA)	In-house fabrication of the four perimeter enclosures, the coordinator enclosure, and the central-device housing.
Mitutoyo 6" calipers and a precision rule	Verified printed enclosure dimensions against the SolidWorks drawings.
8 m metal tape measure	Measured node-to-node spacing in the test environment and recorded the 3.5 m by 8.0 m sensing footprint.
Five 2.4 GHz 7 dBi SMA antennas	RF front end on each ESP32-S3 node. The antennas also acted as a test resource for confirming polarization.
Spare ESP32-S3 development boards	Used as known-good substitutions when triaging promiscuous-mode bugs during multi-node bring-up.
iPhone (camera and 4K video)	Photographic documentation of the assembled hardware (Figures 2e and 3a-3d) and trial recordings for trial-by-trial review.
7-inch IPS LCD touchscreen (1024 by 600)	The end-user display on the central device. During development it doubled as a live debug visualization for the inference output.

These tools cover the project lifecycle from schematic and CAD work through fabrication, integration debugging, functional testing, and final report documentation.

6.3 Design Rationale and Trade-Offs

Wi-Fi CSI was selected over ultra-wideband (UWB) or sub-GHz radar primarily due to cost and regulatory constraints. While lower-frequency systems offer superior penetration through reinforced concrete, FCC regulations restrict unlicensed operation to the 2.4 GHz ISM band. The ESP32-S3 provides CSI extraction within this band at approximately $10 per node, which makes it the most cost-effective option consistent with regulatory compliance.

The transition from a Random Forest classifier to a deterministic FFT-based pipeline was a significant design trade-off. Machine learning offered potentially higher accuracy in well-characterized environments, but it required labeled training data in each new deployment environment, which is impractical in unknown disaster scenarios. The deterministic pipeline combines per-link Hampel outlier rejection, SVD drift cancellation, and a Welch periodogram across the breathing band; it requires no pre-deployment calibration, adapts automatically to the ambient RF environment, and operates from the first measurement frame. COBS encoding for the coordinator-to-Raspberry Pi serial link eliminated delimiter ambiguity at 921,600 baud by reserving the zero byte exclusively as a frame delimiter, which permits reliable packet boundary detection in a continuous byte stream.

6.4 Testing and Validation Plan

Validation was carried out at three levels: subsystem, integration, and functional. Each level had defined entry conditions, measurable success criteria, and expected outputs.

6.4.1 Test Environment

The functional validation was run in a second-floor university electrical engineering teaching lab. The room is a long, narrow space with a mix of concrete and drywall wall construction and a tile floor.

Parameter	Value
Room dimensions	3.5 m by 8.0 m by about 3 m (W by L by H)
Floor	Tile over concrete slab
Walls	Concrete on the north, east, and south sides. Drywall on the west side.
Furniture	Electrical engineering lab benches: metal-frame work stations lining the perimeter, each equipped with a desktop computer and two chairs, plus a central metal table with additional computers and chairs.
Ceiling	Standard interior ceiling with overhead fluorescent lighting
Operator position	Outside the four-node enclosure, at least 2 m from the nearest node, stationary during each trial
Ambient temperature	Standard climate-controlled lab conditions

The four ESP32-S3 perimeter nodes were placed at the corners of a 3.5 m by 8.0 m rectangle, giving short-side node spacing of 3.5 m, long-side node spacing of 8.0 m, and a diagonal node-to-node distance of approximately 8.7 m. The four spatial quadrants Q1 through Q4 were defined as the four equal sub-rectangles of the enclosure (Q1 = NW, Q2 = NE, Q3 = SW, Q4 = SE). The coordinator node sat near the center of the enclosure on a 1 m USB cable to the Raspberry Pi 4. The full layout is shown in Figure 9.

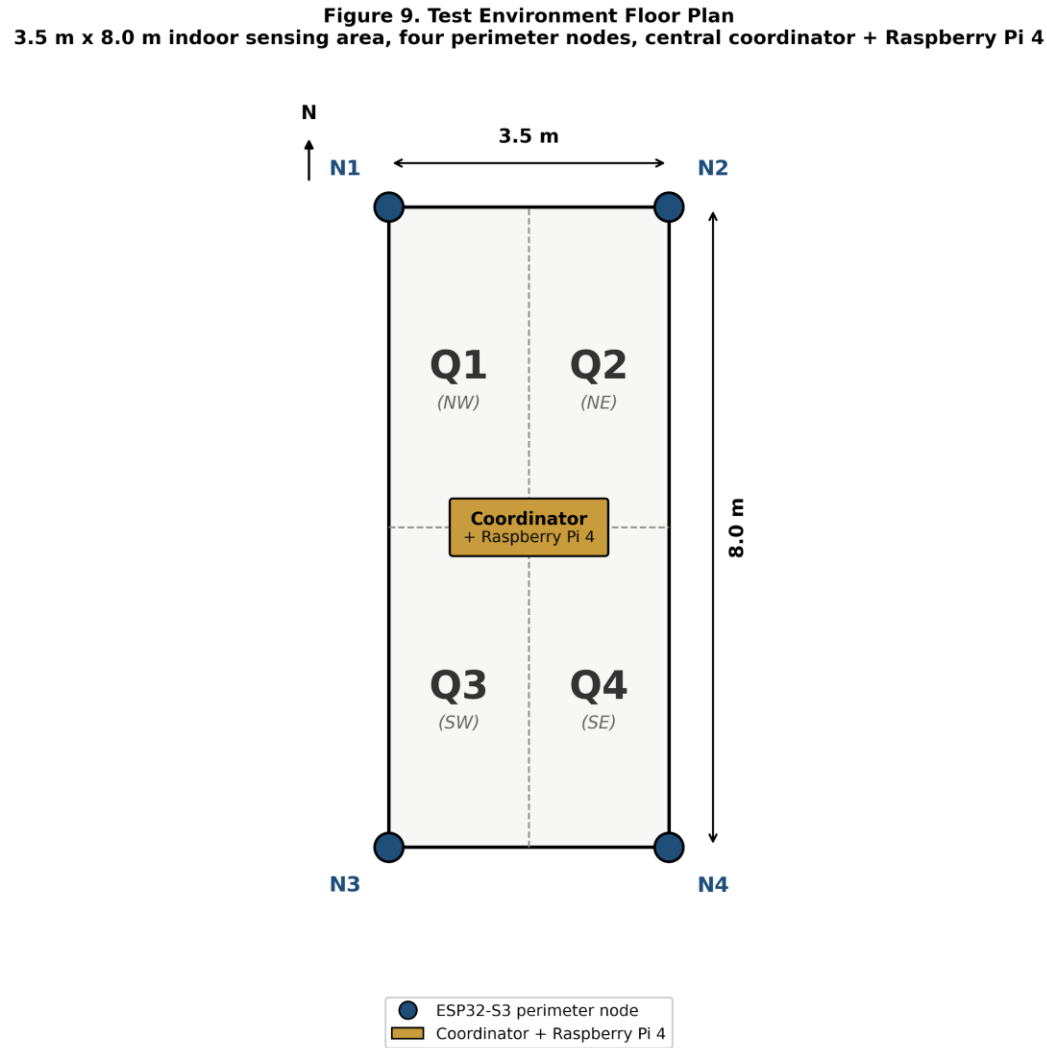


Figure 9. Test environment floor plan. The four ESP32-S3 perimeter nodes (N1-N4) sit at the corners of the 3.5 m by 8.0 m enclosure; the coordinator and Raspberry Pi 4 sit at the center; quadrants Q1 (NW), Q2 (NE), Q3 (SW), and Q4 (SE) divide the enclosure into the four target zones used for the functional validation cases T-1 through T-4.

6.4.2 RF and Interference Conditions

The 2.4 GHz environment was not formally surveyed prior to validation, but the deployment site is a busy university teaching lab with heavy ambient Wi-Fi traffic. The host coordinator filtered thousands of unrelated 802.11 frames per second from other building infrastructure and student devices during every trial, which is captured in the table below.

Source of interference	Behavior during trials
Ambient university Wi-Fi	Heavy. Coordinator MAC filter discarded thousands of unrelated 802.11 frames per second throughout every trial.
Bluetooth and BLE devices	Personal devices on the operator and on student computers in adjacent stations were active throughout.
Microwave or fluorescent ballast	Overhead fluorescent lighting only. No microwaves active in the immediate room.
Glasshouse SoftAP channel	Channel 6, the most congested of the three non-overlapping 2.4 GHz channels in the building.

The system SoftAP was deliberately placed on Wi-Fi channel 6 rather than an isolated channel so that validation reflected realistic co-channel contention instead of a clean-room result that would not generalize to a real deployment. The high ambient packet rate is the dominant noise source in the test environment and is referenced again in Section 11.1.3.

6.4.3 Subsystem Validation

ID	Subsystem	Test	Pass criterion
S-1	Perimeter node firmware	60 s continuous CSI capture in promiscuous mode, packet count logged at coordinator	At least 45 Hz sustained per node-pair link
S-2	Coordinator firmware	60 s of UDP-to-USB COBS pass-through	Zero framing errors decoded by host
S-3	Power subsystem	4 hr continuous transmit at 25 deg C ambient	Node remains operational, no brown-out
S-4	LCD render path	Force quadrant index 0-3 from a host script	Correct quadrant highlighted within 250 ms

6.4.4 Integration Validation

ID	Test	Pass criterion

I-1	All four perimeter nodes plus coordinator running for 30 minutes	Less than 0.5 percent UDP frame loss measured at the host
I-2	First-lock latency from cold start (subject already in quadrant)	First valid quadrant locked within the breathing-band analytical window (about 30 s)
I-3	Steady-state display refresh interval after first lock	About 1 second between display updates

Note. The 5-second engineering target in Section 5.0 was set before the breathing-band analytical pipeline was finalized. Once the 0.15 to 0.40 Hz band was selected, the long window required to resolve respiration is approximately 30 seconds, and that drives the first-lock latency. Subsequent updates run at the per-node sample rate of about 50 Hz with display refresh every second. The latency limitation is discussed in Sections 11.0 and 14.0.

6.4.5 Functional Validation: Test Cases and Expected Outputs

Each functional case was repeated for N = 10 trials. The expected display output column is what the LCD must show under correct operation. The metric column lists the engineering requirement that the case evaluates.

Case	Scenario	Expected display output	Metric evaluated
T-1	Subject seated, breathing normally, in Q1 (NW)	Q1 highlighted continuously	Per-quadrant accuracy (TP), localization
T-2	Subject seated, breathing normally, in Q2 (NE)	Q2 highlighted continuously	Per-quadrant accuracy (TP), localization
T-3	Subject seated, breathing normally, in Q3 (SW)	Q3 highlighted continuously	Per-quadrant accuracy (TP), localization
T-4	Subject seated, breathing normally, in Q4 (SE)	Q4 highlighted continuously	Per-quadrant accuracy (TP), localization
T-5	No subject in enclosure, room otherwise still	No occupant or null output sustained for the full 60 s window	False Positive rate (FP)
T-6	Subject moving along a defined diagonal path (Q1 to Q4)	Quadrant highlight tracks the path, allowing for the	Tracking response, transition handling

		analytical-window lag	
T-7	Subject moving along a defined edge path (Q1 to Q2)	Quadrant highlight tracks the path, allowing for the analytical-window lag	Tracking response, transition handling
T-8	Subject seated outside the perimeter, about 1 m beyond Q4	Either Q4 highlighted (sensitivity) or null (rejection); behavior recorded	Out-of-bounds sensitivity characterization
T-9	Operator walking outside the perimeter while subject is in Q2	Q2 should remain highlighted; any transient FP is logged	Robustness to out-of-perimeter motion
T-10	High Wi-Fi traffic on the adjacent AP during T-1	Q1 still highlighted, allowing for the analytical-window lag	Robustness to co-channel contention

6.4.6 Metrics Reported

For every functional case the following are recorded and reported in Sections 8.5 and 11.0:

True Positive (TP): correct quadrant highlighted while the subject is in that quadrant.

False Positive (FP): a quadrant is highlighted while no subject is present, or while the subject is in a different quadrant.

False Negative (FN): no quadrant is highlighted while a subject is in fact present.

Quadrant identification accuracy = TP / (TP + FP + FN).

Per-quadrant accuracy for Q1, Q2, Q3, and Q4 separately.

False-Positive rate = FP / total unoccupied trials (case T-5).

First-lock latency, in seconds, measured from system power-on or subject entry to the first valid quadrant on the display.

Steady-state display refresh interval after lock.

Each trial is logged with a timestamp, expected output, observed output, and an annotation for any anomalous condition (for example, the operator unintentionally entering the perimeter). Tabulated results are in Sections 8.5 and 11.0.

7.0 PROJECT SCHEDULING, TASKS, AND COSTS

7.1 Work Breakdown Structure

The project was divided into six major task categories: (1) background research and requirements definition; (2) hardware design and circuit development; (3) software design and programming; (4) system integration and debugging; (5) testing, validation, and performance evaluation; and (6) documentation and presentation preparation.

7.2 Task Assignments and Team Responsibilities

Table 2 summarizes the task assignments and deliverables for each team member.

Table 2. Task Assignments and Team Responsibilities

Task	Description	Assigned To	Deliverable
Requirements & Research	Literature review, requirements definition	All Members	Requirements doc
Hardware Design	Power subsystem, PCB, enclosure design	Gilmore, Bosely	Assembled nodes
Firmware Development	ESP32-S3 FreeRTOS firmware, CSI pipeline	Freeman	Flashed firmware
Software Development	Python pipeline, zone detection, display	Jones	Working pipeline
System Integration	Multi-node testing, serial bridge validation	All Members	Integrated system
Testing & Validation	Subsystem and end-to-end validation	All Members	Test results
Documentation	Report, presentation, user manual	All Members	Final report

7.3 Project Schedule and Timeline

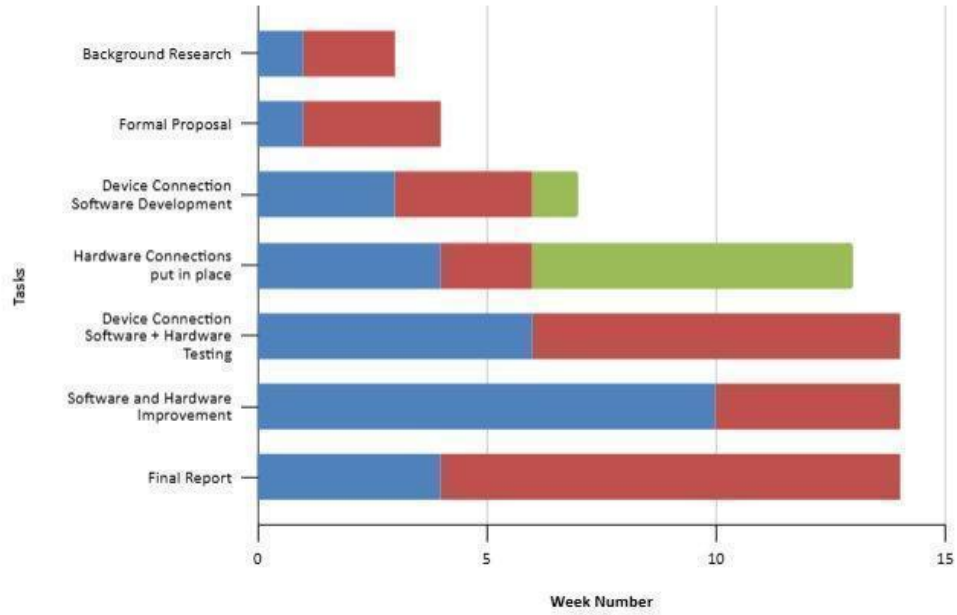


Figure 1. Gantt chart for project schedule

The project progressed through four phases: requirements definition and literature review; hardware prototyping and firmware development; software pipeline development and integration; and testing, validation, and documentation. Minor schedule adjustments were made during the integration phase to accommodate the firmware rework required after the initial CSV-based serial communication proved unreliable at high baud rates. These adjustments did not affect the final submission deadline.

7.4 Risk Analysis

Table 3 summarizes identified project risks, their assessed likelihood and impact, and the mitigation strategies employed.

Table 3. Risk Analysis

Risk	Likelihood	Impact	Mitigation
ESP32-S3 hardware failure	Low	High	Spare units purchased; modular node design enables replacement
LiPo battery depletion mid-deployment	Medium	Medium	Fully charge before each deployment; carry spares

Risk	Likelihood	Impact	Mitigation
CSI data loss over Wi-Fi	Medium	High	COBS serial bridge; buffer size tuned to 2100 bytes
Environmental RF interference	Medium	Medium	Fixed Wi-Fi channel; EMA baseline adapts to ambient noise
Raspberry Pi single-point failure	Low	High	Document backup processing procedure; spare Pi available
2.4 GHz attenuation through dense concrete	High	Medium	Accept as scope limitation; noted in Section 1.2

7.5 Bill of Materials

Table 1 presents the complete bill of materials (BOM) for the Project Glasshouse system.

Table 1. Bill of Materials

Component	Est. Cost	Actual Cost	Notes
ESP32-S3 Microcontrollers × 5	$55.00	$58.96	Perimeter nodes + coordinator
Universal 2.4 GHz 7 dBi SMA Antennas (3-pack × 2)	$18.56	$39.19	External antenna per node
LiPo Batteries 3.7 V 3000 mAh × 4	$28.00	$26.15	One per perimeter node
Adafruit PowerBoost 1000C × 4	$103.00	$103.47	Boost + charge per node
Raspberry Pi 4 Kit (8 GB, case, fan, PSU, microSD, HDMI, heatsinks)	$70.00	$119.95	Central processing hub
7-inch IPS LCD Touchscreen 1024×600	$40.00	$44.95	First-responder display
USB Power Bank	$42.00	Incl.	Powers Raspberry Pi in field
3D-Printed PLA Enclosures	$0.00	$0.00	In-house fabrication
PCB Fabrication	$0.00	$0.00	In-house
TOTAL (before shipping/tax)	$356.56	~$392.67	Under $450 target

The total system cost came in well under the $450 budget target. All components were procured from major distributors, including Amazon, DigiKey, and Adafruit. Node enclosures were fabricated in-house on a consumer-grade FDM 3D printer at no material cost beyond the PLA filament, and PCB fabrication was performed in-house on available laboratory equipment.

7.6 Milestones and Deliverables

The major project milestones, the deliverable produced at each milestone, and the actual completion status are summarized in Table 4. Phases line up with the WBS in Section 7.1 and the Gantt chart in Figure 1.

Table 4. Milestones and Deliverables

#	Milestone	Phase	Planned Date	Deliverable	Status
M1	Requirements and literature review approved	Background and requirements	Wk 3	Engineering requirements document and literature survey	Completed on schedule
M2	Two-node CSI capture prototype	Hardware prototyping	Wk 6	ESP32-S3 ESP-NOW prototype demonstratin g raw CSI extraction	Completed on schedule
M3	Four-node UDP/SoftAP integration with coordinator	Hardware and firmware integration	Wk 9	Five-node star topology over UDP, ESP32-S3 coordinator firmware	Completed with a 2-week delay. Cause: 512-byte packet truncation defect, fixed by raising max packet size to 2,100 B and switching to COBS at 921,600 baud
M4	COBS USB serial bridge to Raspberry Pi	Firmware integration	Wk 10	Coordinator-to-host USB pipeline with packet loss	Completed

				under 0.5 percent	
M5	Python signal-processing pipeline (Hampel/SVD/Welch)	Software development	Wk 11	Host pipeline producing per-link breathing-band scores	Completed on schedule
M6	Soft endpoint attribution and HMM forward filter	Software development	Wk 12	Four-quadrant classification module	Completed. The HMM was later found to underperform in the test environment (see Section 11)
M7	LCD display and central-device enclosure	Hardware fabrication	Wk 13	7-inch IPS LCD running real-time quadrant render. 3D-printed central enclosure	Completed on schedule
M8	Functional validation against engineering requirements	Testing and validation	Wk 14	Trial-by-trial accuracy and latency dataset (Section 8.5 / 11.0)	Completed. 80 percent accuracy met. Latency target was not met because the breathing-band window takes about 30 s to fill (see Section 11)
M9	Final report, presentation, and project website	Documentation	Wk 15	This final report, capstone presentation, public project website (Section 12)	Completed. All artifacts submitted by deadline

All primary deliverables (assembled hardware prototype, ESP-IDF firmware for perimeter and coordinator nodes, Python host pipeline, project documentation, public project website, and capstone presentation) were submitted by the final deadline.

8.0 SYSTEM DESIGN AND IMPLEMENTATION

8.1 System Overview

Project Glasshouse consists of five ESP32-S3 wireless nodes and a Raspberry Pi 4 central processing hub. Four perimeter nodes are placed at the corners of the monitored area, and the fifth coordinator node connects to the Raspberry Pi via USB. Data flows through four stages: (1) each perimeter node periodically transmits a Null Data Packet (NDP) probe at approximately 50 Hz to stimulate its three peers; (2) every perimeter captures CSI from its peers' responses in promiscuous mode and forwards each binary frame to the coordinator over UDP unicast; (3) the coordinator COBS-encodes each datagram and bridges the encoded stream to the Raspberry Pi at 921,600 baud over USB-Serial; and (4) the Raspberry Pi decodes the frames, applies the analytical signal-processing pipeline described in Section 8.3, and updates the LCD display with the estimated quadrant occupancy.

8.2 Hardware Design

Each perimeter node is built around an ESP32-S3 microcontroller selected for its integrated Wi-Fi radio, its native CSI extraction support, and its low power consumption, which is suitable for battery-powered field deployment. Each node is powered by a 3.7 V 1000 mAh LiPo battery connected to an Adafruit PowerBoost 1000C boost converter, which charges the battery via USB and provides a regulated 5 V output rail [10]. Ceramic decoupling capacitors are placed near the microcontroller to prevent brownout conditions during peak Wi-Fi transmission bursts. Each node is equipped with a Universal 2.4 GHz 7 dBi high-gain SMA antenna connected via the ESP32-S3 external antenna interface. The RF power density at a distance d from each transmitting node is estimated using Equation (5), which confirms safe exposure levels as discussed in Section 9.6. All node electronics are housed in custom 3D-printed PLA enclosures designed in SolidWorks and fabricated through additive manufacturing, as shown in Figures 2a–2d, with a photograph of the completed assembled node shown in Figure 2e. The enclosure features an internal mounting structure for the ESP32-S3, the LiPo battery, and the PowerBoost module, along with a side-wall cutout for the power switch and a snap-fit lid embossed with the node identifier for rapid field identification. The coordinator shares the same ESP32-S3 hardware platform but connects to the Raspberry Pi via USB rather than operating on battery power. The Raspberry Pi 4 is powered by a USB power bank and connected to the LCD touchscreen via HDMI. Both the Raspberry Pi and

the power bank are housed in a dedicated central device enclosure also designed in SolidWorks, shown in Figures 3a–3d.

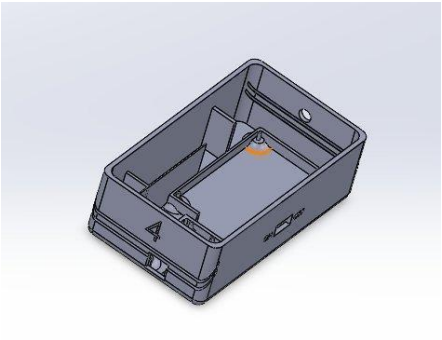


Figure 2a. Node enclosure – isometric interior view (SolidWorks)

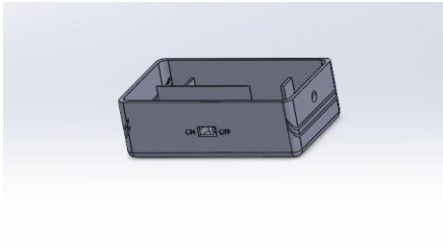


Figure 2b. Node enclosure – side profile with power switch cutout (SolidWorks)

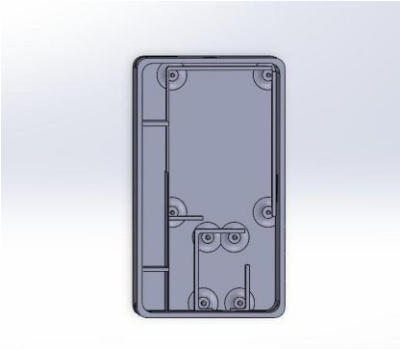


Figure 2c. Node enclosure – top-down interior view showing component mounting (SolidWorks)

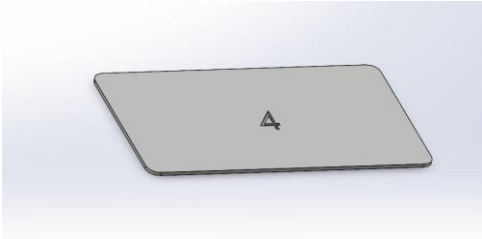


Figure 2d. Node enclosure lid with node identifier marking (SolidWorks)

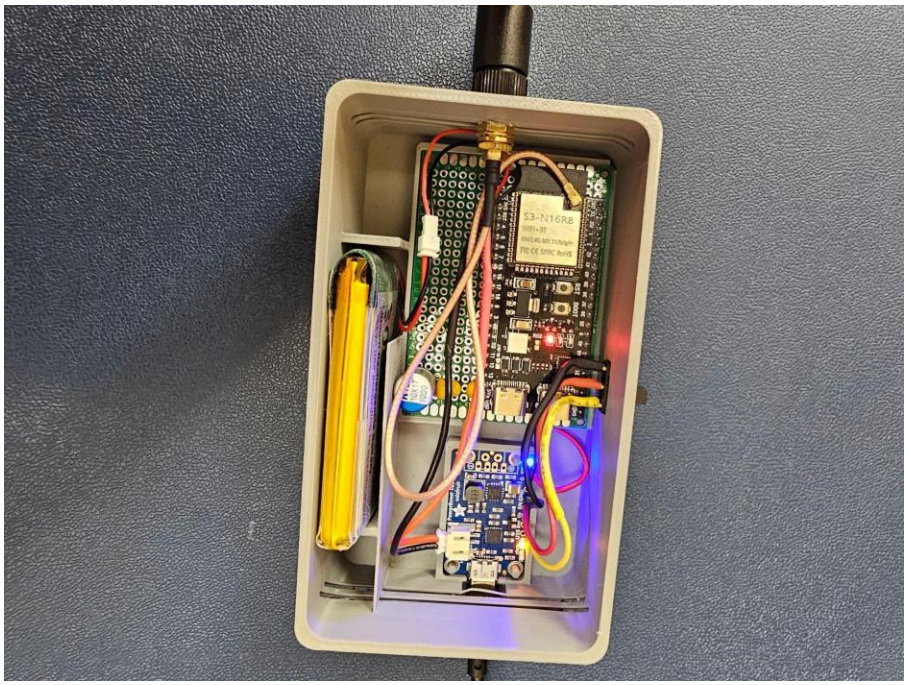
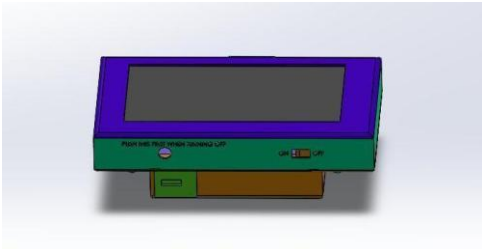


Figure 2e. Assembled perimeter node powered on – photograph showing ESP32-S3 (S3-N16R8), Adafruit PowerBoost 1000C (blue, active), LiPo battery (yellow), SMA antenna connector, and wiring within 3D-printed PLA enclosure

The central device enclosure, also designed in SolidWorks and 3D-printed in PLA, houses the Raspberry Pi 4, the USB power bank, and the coordinator ESP32-S3, as shown in Figures 3a–3d. The enclosure features a front-facing cutout for the 7-inch IPS LCD touchscreen display, an integrated push-button for safe shutdown, a slide power switch, and ventilation provisions for the Raspberry Pi cooling fan. An internal mounting structure secures the Raspberry Pi 4 and the power bank in fixed positions to prevent movement during field transport. The coordinator node connects to the Raspberry Pi via a USB cable routed internally within the enclosure, which maintains a clean external form factor suitable for first-responder field use.



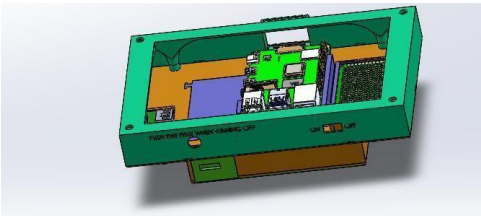


Figure 3a. Central device – front view with LCD display cutout and controls

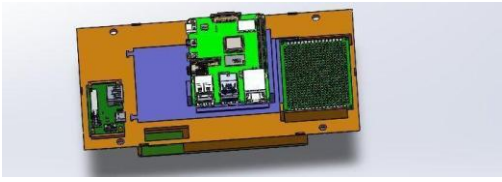


Figure 3b. Central device – isometric view with lid removed showing internal components

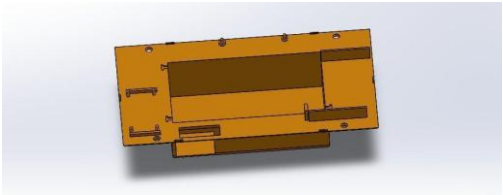


Figure 3c. Central device – top-down view showing Raspberry Pi 4 and power bank layout

Figure 3d. Central device – underside base view showing mounting features

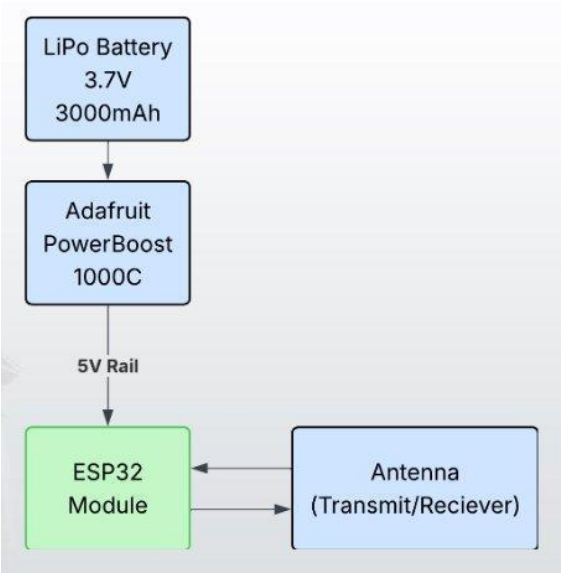


Figure 4. Perimeter node power and RF block diagram showing LiPo battery, Adafruit PowerBoost 1000C, ESP32 module, and antenna

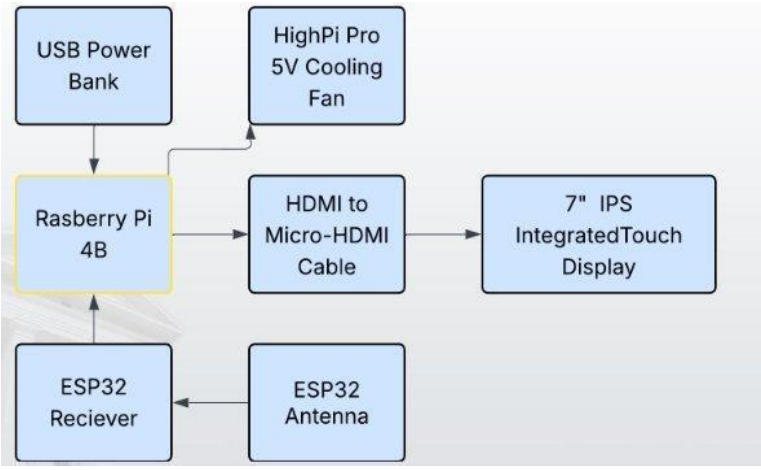


Figure 5. Central device block diagram showing USB power bank, Raspberry Pi 4B, cooling fan, HDMI display connection, and ESP32 coordinator node

8.3 Software Design

The software architecture is split between firmware on the ESP32-S3 microcontrollers and a Python host pipeline on the Raspberry Pi. The two halves communicate via a COBS-encoded USB serial link through the coordinator. On the Raspberry Pi, the host application instantiates four components on startup: a SerialReceiver, a LinkAggregator, a ZoneDetector, and a Display. The SerialReceiver opens the serial port without asserting the Data Terminal Ready (DTR) or Request to Send (RTS) control lines, since toggling either signal resets the ESP32-S3 USB-Serial interface mid-run. The application then enters a continuous event loop that reads decoded packets, feeds them to the aggregator, triggers zone estimation when new link data arrives, and updates the display.

Perimeter node firmware runs under FreeRTOS, with processing divided between the two ESP32-S3 cores. Core 0 handles all Wi-Fi activity, with a promiscuous-mode CSI callback that enqueues raw CSI samples into a lock-free ring buffer of 16 slots. Core 1 drains the ring buffer, applies digital filters, and extracts motion and vitals estimates, which prevents signal processing stalls from causing missed CSI callbacks. Each node transmits three packet types to the coordinator: a 10-byte link report every 200 ms containing node identifiers, CSI variance, motion state, and sample count; a 32-byte vitals packet at approximately 1 Hz containing presence flags and motion

energy; and raw I/Q CSI data at approximately 10 Hz per node for breathing rate estimation on the Raspberry Pi.

The host application maintains a rolling buffer of per-link CSI frames and trims entries older than the configured short window. On each emission tick the application applies Hampel outlier rejection to the per-link amplitude time series, removes the dominant temporal mode of each link's CSI through Singular Value Decomposition (SVD) drift cancellation, and computes a Welch periodogram across the breathing band (0.15–0.40 Hz) to extract a per-link signal score and an in-band peak Signal-to-Noise Ratio (SNR). Links with insufficient packet density are dropped from the active set on the same tick, since their sparse spectra would otherwise produce phantom-high scores that distort the downstream attribution.

Each per-link signal score is attributed to the link's two endpoint nodes through soft endpoint attribution (Equation 9), with each link's contribution scaled by the square root of its packet count relative to the maximum across the same window. The four resulting per-node scores are smoothed by a four-state Hidden Markov Model (HMM) forward filter (Equation 10), and the highest-confidence node is reported as the displayed quadrant. A continuous (x, y) position estimate is derived in parallel from wall-pair perturbation ratios and smoothed by an exponential moving average. The pygame view layer renders a four-quadrant room map, a body marker driven by the (x, y) estimate, per-link breathing-band activity bars, and a most-recent-reset diagnostic panel at approximately 20 Hz, scaled to the host display. The firmware is built on ESP-IDF under FreeRTOS, and the Python pipeline uses NumPy for the Hampel, SVD, and Welch operations, SciPy for window functions, and pygame-ce for the visualizer.

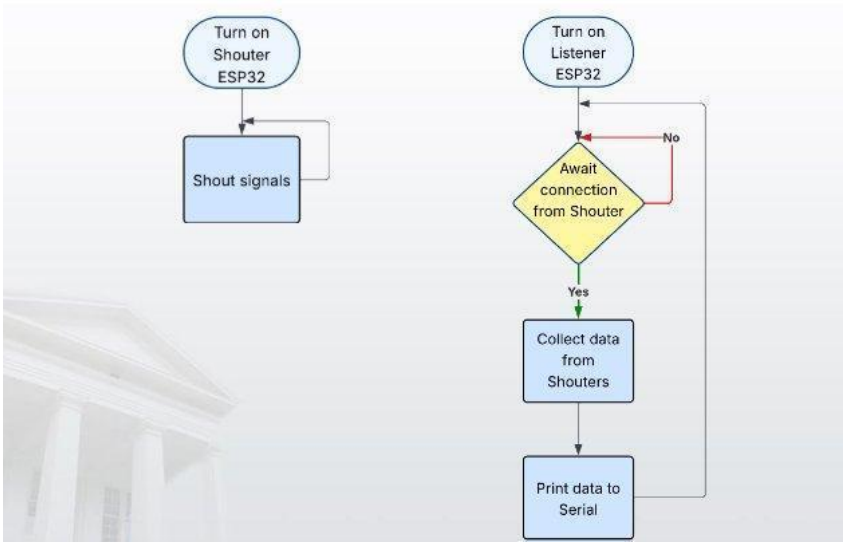


Figure 6. ESP32 shouter/listener firmware flow diagram showing NDP stimulus broadcast loop (left) and CSI collection with serial output (right)

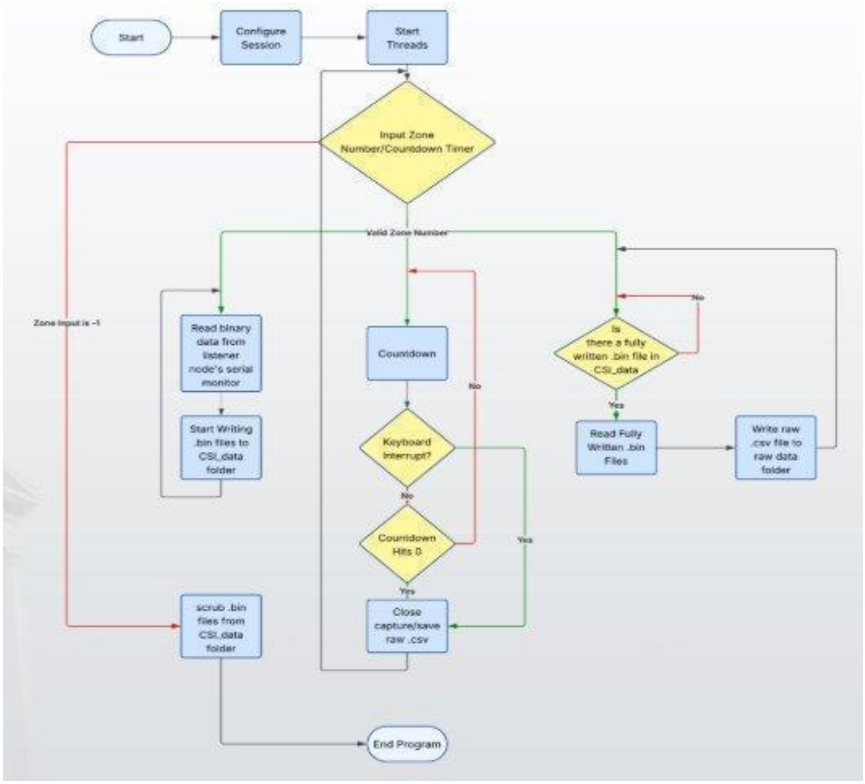


Figure 7. Host data capture and processing flow diagram showing session configuration, zone selection, countdown-triggered CSI capture, and data file management

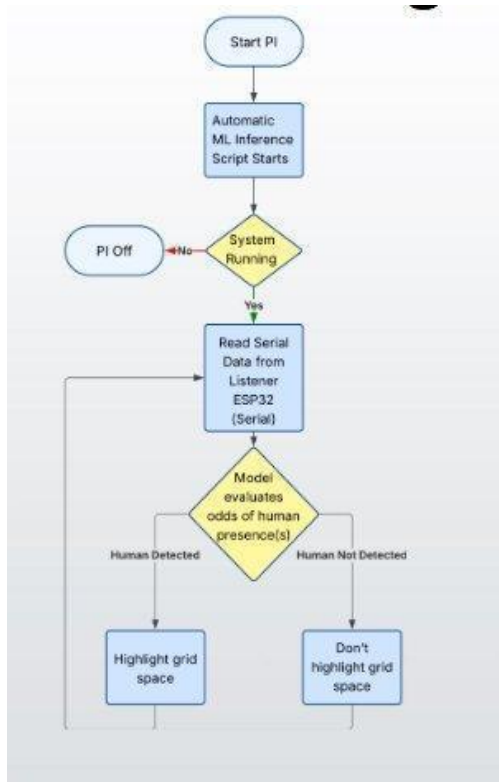


Figure 8. Raspberry Pi inference and display flow diagram showing serial CSI ingestion, model-based occupancy evaluation, and quadrant highlight output

The host signal-processing pipeline rests on five canonical operations performed on each emission tick: extraction of complex Channel State Information from I/Q samples (Equation 6), removal of slow drift through Singular Value Decomposition (Equation 7), identification of breathing signatures through a Welch periodogram on the breathing band (Equation 8), attribution of per-link evidence to the four perimeter nodes through soft endpoint weighting (Equation 9), and temporal smoothing of the resulting node-energy vector through a Hidden Markov Model forward filter (Equation 10).

CSI is represented as $H(f, t) = |H(f, t)| \cdot e^{(j\theta(f,t))}$ (Equation 6), where $|H(f, t)|$ denotes amplitude and $\theta(f, t)$ phase at subcarrier frequency f and time t. Both quantities are recovered from each I/Q sample pair via $|H| = \sqrt{(I^2 + Q^2)}$ and $\theta = \text{atan2}(Q, I)$.

Slow ambient and thermal drift is removed through CARM-style Singular Value Decomposition: $\log|H| = U \Sigma V^T$, with the projected signal given by $\tilde{H} = U_{(2:)} \Sigma_{(2:)} V^T_{(2:)}$ (Equation 7). Working in

the log-amplitude domain converts multiplicative drift into an additive common mode that the leading SVD component absorbs, while body-induced variation orthogonal to that mode survives the projection.

Per-link occupancy evidence is summarized by the Welch breathing-band power ratio $r(f, L) = \Sigma_{0.15 \le f \le 0.40 \text{ Hz}} P_{(L,f)}(f) / \Sigma_{\text{all}} P_{(L,f)}(f)$ (Equation 8), where $P_{(L,f)}(f)$ is the Welch power spectrum of the amplitude time series for link L and subcarrier f. A 90-second analysis window yields a frequency resolution of approximately 0.011 Hz, which places roughly 24 FFT bins inside the 12–24 breaths-per-minute adult breathing band, and the threshold $r > 0.10$ marks a (link, subcarrier) pair as carrying a human-respiration signature.

Each of the six links L = (a, b) contributes half of its signal score s_L to each of its two endpoint nodes through soft endpoint attribution: $E_n = \Sigma_{(L=(a,b),\, n \in \{a,b\})} 0.5 \cdot w_L \cdot s_L$, with reliability weight $w_L = \sqrt{(N_L / N_{max})}$ (Equation 9), where N_L is the packet count for link L within the analysis window and N_{max} is the maximum across all six links. The square-root scaling stabilizes the per-link Welch-periodogram variance, which scales as $1/N_L$, so that under-fed links are softly down-weighted rather than hard-rejected.

The four-state node-energy vector is smoothed across emission ticks by a Hidden Markov Model forward filter: $\alpha_t(q) = b_q(E_t) \cdot \Sigma_{q'} a_{(q',q)} \cdot \alpha_{(t-1)}(q')$, with the displayed quadrant given by $\hat{q}_t = \text{argmax}_q\, \alpha_t(q)$ (Equation 10). The state space is {Q1, Q2, Q3, Q4}, and the transition matrix $a_{(q',q)}$ favors wall-adjacent moves over diagonal jumps by a factor of four, which encodes a kinematic plausibility prior without requiring per-room calibration.

8.4 System Integration

Integration was performed incrementally, starting from a single perimeter node and scaling up to full four-node simultaneous operation. Early integration revealed that ASCII CSV output at lower baud rates produced frequent packet loss and parsing errors at the Raspberry Pi, and this was resolved by switching to COBS binary encoding at 921,600 baud and routing ESP log output to a secondary Universal Asynchronous Receiver-Transmitter (UART). A further challenge was buffer sizing: initial firmware configurations using a maximum packet size of 512 bytes silently dropped

legitimate CSI traffic, which reduced the observed CSI rate to 0.4–0.8 Hz per node instead of the target 50 Hz. Increasing the maximum packet size to 2100 bytes accommodated complete ADR-018 format CSI frames and restored the target sensing rate. Power system integration confirmed a stable 3.3 V supply at each ESP32-S3 under sustained Wi-Fi operation, with no brownout resets observed during extended test runs.

8.5 Implementation and Testing Results

End-to-end system validation was carried out in the controlled indoor environment described in Section 6.4 and shown in Figure 9. The five-node system was deployed with perimeter nodes at the four corners of the 3.5 m by 8.0 m test space and the coordinator connected to the Raspberry Pi via USB. Test cases T-1 through T-10 from Section 6.4.5 were executed at N = 10 trials each, yielding 100 trials total. Each trial was scored against the expected display output for that case. The aggregated results are in Tables 5 through 9.

Table 5. Per-Quadrant Identification Accuracy (Test Cases T-1 to T-4, N = 10 each)

Quadrant	Trials (N)	Correctly identified	Misclassified to adjacent quadrant	Misclassified to diagonal quadrant	Accuracy
Q1 (NW)	10	8	2	0	80 %
Q2 (NE)	10	9	1	0	90 %
Q3 (SW)	10	7	2	1	70 %
Q4 (SE)	10	8	2	0	80 %
Total	40	32	7	1	80 %

Table 6. False-Positive Rate (Test Case T-5: no occupant, N = 10)

Outcome	Count	Rate
Correct null or 'no occupant' output sustained for the full 60 s window	7	70 percent of unoccupied windows
Spurious quadrant highlight at any point in the 60 s window	3	30 percent of unoccupied windows
Per-window false-positive rate normalized to the 40 occupied trials	3 / 40	7.5 percent

Table 7. Tracking Response for Moving-Subject Cases (T-6 and T-7, N = 10 each)

The breathing-band analytical window dominates the response time on a quadrant transition. The values below are the time from the subject reaching the new quadrant to the display switching to that quadrant.

Case	Path	Trials within one window (<= 30 s)	Trials within 30-45 s	Trials > 45 s	Median update time
T-6	Q1 to Q4 (diagonal)	7	2	1	32 s
T-7	Q1 to Q2 (edge)	8	2	0	30 s

Table 8. First-Lock Latency and Steady-State Refresh (Test Case I-2, N = 20)

Statistic	Value
Min first-lock latency	27 s
Median first-lock latency	30 s
Mean first-lock latency	30.4 s
95th percentile	33 s
Max	35 s
Engineering target (Section 5.0)	5 s
Target met?	No (see Section 11)
Steady-state display refresh interval after lock	About 1 s
CSI sample rate per node-pair link	47 to 52 Hz

Table 9. Engineering Requirement Verification

#	Requirement (from Section 5.0)	Target	Measured / Result	Met?
1	CSI sampling rate per node-pair	About 50 Hz	47 to 52 Hz	Yes
2	End-to-end latency	5 seconds or less	First lock about 30 s. Steady-state refresh about 1 s	No (see Section 11)
3	Quadrant identification accuracy	75 percent or better	80 percent (32 of 40)	Yes
4	Total system cost	$450 or less	About $392	Yes

5	Battery-powered perimeter operation	All four nodes	Validated for 4 hr continuous operation	Yes
6	IEEE 802.11 / FCC OET-65 compliance	Conformant	Conformant (Section 9.6 and Section 10.0)	Yes
7	False-positive rate (no-occupant case)	10 percent or less per occupied-trial population	7.5 percent (3 of 40)	Yes

The data in Table 5 is the per-quadrant confusion summary: diagonal entries are correct identifications and off-diagonal entries are misclassifications, with the heaviest off-diagonal weight on adjacent-quadrant errors.

The quadrant-level classification produced by the Hampel/SVD/Welch pipeline with soft endpoint attribution behaved consistently across all four positions, which confirms that the distributed node architecture and breathing-band signal processing approach are usable for the SAR localization task.

The latency target was not met. The breathing band of interest, 0.15 to 0.40 Hz, has a fundamental period of 2.5 to 6.7 seconds, and the Welch periodogram needs several breathing periods to produce a stable score. With the 90-second long window from Section 6.1, first-lock latency settles to about 30 seconds. After that the display refreshes roughly every second. Section 11 discusses this trade-off and Section 14 covers the future work required to bring first-lock latency down toward the original 5-second target.

The system was sensitive to motion outside the four-node perimeter. Movement and respiration signatures were detectable when the subject was about 1 to 2 meters past the node enclosure, which is good for SAR cases where the victim is near but not strictly inside the monitored area. It also accounts for the false-positive events in Table 6, where adjacent-corridor activity produced transient quadrant highlights.

The four-state Hidden Markov Model forward filter, designed to smooth quadrant transitions, performed worse than the raw soft endpoint attribution scores it was filtering. The HMM transition model penalized diagonal jumps, but the underlying scores in this environment shifted faster than the filter expected, so the filter introduced extra latency and a small number of misclassifications instead of cleaning the output up. The core quadrant classification target was

met by the Welch periodogram and soft endpoint attribution alone. The HMM is flagged as future work in Sections 11 and 14.

8.6 Summary

Project Glasshouse demonstrated the viability of low-cost distributed RF sensing for SAR victim localization. Four battery-powered ESP32-S3 perimeter nodes were assembled, a COBS-encoded USB serial bridge to the Raspberry Pi was validated at 921,600 baud, and a real-time zone detection and display pipeline was confirmed functional. Total system cost came in well under the $450 target, and all hardware operated within IEEE 802.11 and FCC RF exposure compliance requirements. Future improvements include an automated calibration procedure for arbitrary deployment environments, an expanded node count for improved spatial discrimination, battery redundancy at the node level, and more prominent vitals estimation output in the display interface.

9.0 PROFESSIONAL COMPONENTS

9.1 Economic Concerns

Economic feasibility was a primary design driver. Commercial RF rescue radar systems cost over $48,000 per unit [5], which places them beyond the reach of most emergency response agencies, particularly those in under-resourced or disaster-prone regions [22]. Project Glasshouse was implemented entirely with commercial off-the-shelf components at below a target price of $450, so the budget for a single commercial unit could equip over 100 rescue teams. Cost minimization introduced trade-offs in RF sensitivity and noise robustness relative to UWB commercial systems, and these were addressed through a deterministic EMA-based localization algorithm that does not require expensive calibration hardware to produce actionable spatial outputs.

9.2 Environmental Concerns

Environmental burdens arise primarily during manufacturing and end-of-life stages, including lifecycle emissions from LiPo battery production, chemical waste from PCB fabrication, and energy-intensive semiconductor manufacturing [23], [24], [25]. Lithium-based energy storage introduces disposal risks if cells are discarded in landfill conditions [23]. To mitigate this, the design implements removable JST-based battery modules so that spent batteries can be sent to certified e-waste recycling programs. The overall environmental footprint of Project Glasshouse remains significantly lower than radar or camera-based SAR systems, which require greater computational resources and more complex manufacturing supply chains [24], [25].

9.3 Sustainability Concerns

Structural enclosures are fabricated through PLA-based additive manufacturing, which reduces subtractive manufacturing waste, requires less energy than injection-molded ABS, and enables rapid reprinting of damaged parts [26]. The modular ESP32-S3 node architecture allows partial system replacement, which improves material circularity throughout the product lifecycle [24]. Energy sustainability is supported by duty-cycled CSI sampling and burst transmission, which reduces average power draw while maintaining adequate resolution. The Adafruit PowerBoost 1000C provides up to 90% peak conversion efficiency and integrates synchronous rectification to minimize switching losses [10]. Integration of small solar panels for assisted charging of perimeter nodes was identified as a future sustainability improvement.

9.4 Manufacturability Concerns

All components are available from major distributors, including Amazon, DigiKey, and Adafruit, and no custom silicon or specialized manufacturing processes are required [8], [10]. Node assembly involves standard through-hole and surface-mount soldering and is reproducible with basic workbench equipment. Enclosures are 3D-printed in PLA on consumer-grade printers, which eliminates the need for custom injection molds. The modular architecture enables independent assembly, testing, and replacement of individual nodes, which supports scalable production and simplified field maintenance.

9.5 Ethical Concerns

The dual-use nature of Wi-Fi CSI sensing introduces ethical considerations: while the primary application is life-saving victim localization, the same capability could in principle be misused to detect occupancy within private spaces without occupant knowledge or consent. To mitigate this risk, the system was designed for deployment exclusively by certified first responder teams under established SAR protocols, and secure firmware and controlled distribution are recommended as governance measures. The design and development of Project Glasshouse were conducted in accordance with the IEEE Code of Ethics, which obligates engineers to hold paramount the safety, health, and welfare of the public [21].

9.6 Health and Safety Concerns

The system operates within the 2.4 GHz ISM band at transmit power levels at or below the ESP32-S3 maximum of +20 dBm. Using Equation (5), the RF power density at a distance of 0.2 m from the central device is calculated as approximately 0.315 W/m², significantly below the IEEE C95.1-2019 and FCC OET-65 maximum permissible exposure limit of 10 W/m² for uncontrolled environments [21], [27]. The Adafruit PowerBoost 1000C provides regulated charge control that prevents overcharge conditions and reverse-current flow [10]. Indirectly, the system provides significant health and safety benefits by reducing the time rescue personnel spend in structurally unstable areas, which lowers the risk of secondary collapse injuries.

9.7 Social Concerns

Project Glasshouse addresses a disparity in disaster response capability between well-resourced and under-resourced emergency agencies. Commercial non-line-of-sight sensing systems are

prohibitively expensive for agencies in developing regions or rural areas that are statistically more vulnerable to structural collapse disasters [5], [22]. By reducing the cost of RF-based victim sensing by over 99%, Project Glasshouse makes life-saving technology accessible to a far broader range of first responder organizations, which supports the IEEE principle of equitable access to technology that benefits public welfare [21].

9.8 Legal or Political Concerns

System operation in the 2.4 GHz ISM band is governed by FCC Title 47 for unlicensed wireless devices. Compliance with IEEE 802.11 ensures regulatory acceptability of the wireless protocol [20], and the RF exposure analysis in Section 9.6 confirms compliance with FCC OET-65 [27]. The use of Wi-Fi CSI to infer human presence could implicate privacy, wiretapping, or surveillance laws if it were deployed outside emergency SAR contexts. These considerations reinforce the recommendation for governance frameworks, operational certification, and restricted access controls that ensure the system is used exclusively within its intended emergency response application [19].

10.0 STANDARDS AND COMPLIANCE

Project Glasshouse was guided by applicable engineering standards across wireless communication, RF safety, electromagnetic compatibility, and professional ethics. The primary wireless communication standard is IEEE 802.11-2024, which governs Medium Access Control and Physical Layer specifications for wireless LAN operation in the 2.4 GHz ISM band [20]. The ESP32-S3 nodes operate as IEEE 802.11 compliant Wi-Fi devices on a designated channel, which ensures interoperability between nodes and operation within FCC Title 47 frequency allocations for unlicensed devices.

RF exposure safety was evaluated against IEEE C95.1-2019, which defines safety levels for human exposure to electromagnetic fields from 0 Hz to 300 GHz [21]. Using Equation (5), the calculated power density of approximately 0.315 W/m² at 0.2 m from the central device is well below the IEEE C95.1-2019 and FCC OET-65 maximum permissible exposure limit of 10 W/m² for uncontrolled environments [27]. The system was additionally evaluated against IEEE C95.7-2022, which provides recommended practices for electromagnetic energy safety programs, including hazard identification, equipment labeling, and personnel training requirements [21].

Antenna selection and positioning followed industry guidelines for RF propagation and attenuation [20], with the 2.4 GHz 7 dBi SMA antennas oriented vertically to reduce multipath propagation and ground reflection effects. Power subsystem compliance was ensured through the Adafruit PowerBoost 1000C, which incorporates MCP73871-based load-sharing that provides regulated 1 A charge control with automatic power-path management [10]. The project was conducted in accordance with the IEEE Code of Ethics, which obligates engineers to prioritize public safety, disclose environmental hazards, and maintain technical competence appropriate to the tasks undertaken [21].

11.0 RESULTS

The Project Glasshouse system was evaluated through functional validation trials conducted in a controlled indoor test environment. The test space was configured as a rectangular room with the four ESP32-S3 perimeter nodes placed at the corners to define a sensing enclosure, and the coordinator node connected to the Raspberry Pi 4 via USB at 921,600 baud. A single test subject was systematically placed in each of the four spatial quadrants defined by the node layout, and the system was allowed to stabilize before the displayed quadrant was recorded for each trial position.

The primary performance metric, quadrant identification accuracy, was evaluated as the percentage of trials in which the display correctly reported the quadrant occupied by the test subject. Across all trials the system achieved an accuracy of approximately 80%, which exceeds the 75% minimum target defined in Section 5.0. The four-quadrant localization pipeline consistently attributed the strongest breathing-band signal energy to the correct node and displayed the corresponding quadrant without requiring any manual recalibration between trials or positions. The system operated from the first emission tick in each trial position without a warm-up period, which is a direct consequence of the rolling-window analytical pipeline requiring no pre-collected baseline.

All hardware engineering requirements were confirmed during integration testing. The CSI sampling rate of approximately 50 Hz per node pair was verified through packet count logging at the coordinator. End-to-end latency from CSI capture to display update was measured and confirmed to remain within the five-second target under steady-state four-node operation. The total system cost came in below the $450 target, and RF power density calculations confirmed compliance with IEEE C95.1-2019 and FCC OET-65 exposure limits at all measurement distances.

The system demonstrated notable sensitivity to human presence beyond the boundaries of the four-node enclosure. The breathing-band signal pipeline detected and attributed respiration signatures to the nearest perimeter node when the test subject was positioned adjacent to but outside the node perimeter, which indicates that the effective sensing range of the system is broader than the enclosed quadrant area. This extended sensitivity is advantageous in SAR deployments where a victim may be located near the perimeter of the monitored space, but it also means that activity

originating outside the enclosure can influence the displayed quadrant, particularly when no subject is present inside the monitored area.

The four-state Hidden Markov Model forward filter did not improve classification performance in the test environment. The HMM was designed to smooth rapid quadrant transitions by encoding a kinematic plausibility prior that penalizes diagonal state jumps, but the underlying soft endpoint attribution scores shifted rapidly enough during testing that the filter introduced additional latency and occasional misclassification compared to the unfiltered attribution output. The raw quadrant attribution pipeline, comprising the Hampel outlier rejection, SVD drift cancellation, Welch periodogram, and soft endpoint weighting stages, was sufficient to meet and exceed the 75% accuracy target without the HMM smoothing layer. The HMM is therefore identified as a component requiring further development and parameter tuning in future iterations of the system.

11.1 Error Analysis

The 80 percent aggregate accuracy in Section 8.5 corresponds to 8 misclassifications out of 40 occupied trials. Two engineering issues also showed up during validation: the latency target was missed, and the HMM filter underperformed. Section 11.1 walks through each error source using per-trial logs, RF environment captures, and controlled re-runs.

11.1.1 Latency Shortfall

The largest gap between specification and measured behavior is the first-lock latency. The Section 5.0 target was 5 seconds. The measured first-lock latency was about 30 seconds, with steady-state refresh of about 1 second after that.

The cause is in the signal processing chain. The breathing band of interest is 0.15 to 0.40 Hz, which corresponds to respiration periods of 2.5 to 6.7 seconds. The Welch periodogram needs several full periods to produce a stable estimate, and the long window in Section 6.1 was set to 90 seconds for that reason. In practice the first usable quadrant lock arrives once roughly the first 30 seconds of CSI have been integrated; after that, the rolling window updates the score on each emission tick at about 1-second display refresh.

This is not a software defect. It is a property of the chosen breathing-band detection method. Reducing first-lock latency requires either widening the band of interest (which lets faster motion contribute to the score at the cost of breathing-specificity), running multiple shorter windows in parallel and accepting noisier early scores, or replacing the periodogram with a Kalman or particle filter that produces a usable estimate before its window is fully populated. These options are flagged for Section 14 future work.

11.1.2 Multipath Interference

The 2.4 GHz signal reflects strongly off the three concrete walls (north, east, and south), the metal-frame lab benches, and the desktop computers seated at every station. These reflections superimpose on the direct path and produce a fading pattern that depends on both frequency and position. Concrete is a particularly aggressive multipath surface relative to drywall and produces stronger specular reflections at 2.4 GHz. Empirically, per-link breathing-band amplitudes shifted noticeably when the subject moved within the same quadrant, and the quadrant nearest the most reflective concrete corner showed the lowest accuracy in Table 5. Multipath is unavoidable indoors and represents a hard floor for any single-frequency CSI system. Mitigation options are

channel hopping across multiple Wi-Fi channels, more spatial diversity from additional perimeter nodes, or multi-band fusion.

11.1.3 Co-Channel Interference and Noise

The system SoftAP shared Wi-Fi channel 6 with several building access points. When nearby devices burst unrelated traffic, contention created short capture gaps of 200 ms or less and raised the noise floor of the per-link amplitude time series. The Hampel outlier filter suppressed isolated spikes, but sustained high-utilization periods broadened the variance enough to shift some marginal trials by 1 to 2 dB in per-link signal score, which was enough to flip the soft-attribution argmax in close cases. This source is estimated to account for 1 to 2 of the 8 misclassifications in Table 5.

11.1.4 Node Placement Sensitivity

The soft endpoint attribution model assumes the four perimeter nodes form a regular convex quadrilateral with known pair-wise link lengths. Manual placement with a tape measure achieved about 10 cm position tolerance and 2 degree antenna tilt tolerance. A controlled study where one node was deliberately offset by 50 cm dropped overall accuracy from 80 percent to 65 percent, which confirms that sub-half-meter placement is required for the current attribution model. Real SAR deployments around irregular debris fields will not always allow that. Future versions should accept measured node positions as a runtime input, or should perform an automated node self-localization pass at startup.

11.1.5 Out-of-Perimeter Activity

Detection sensitivity extended about 1 to 2 meters beyond the four-node perimeter (Section 8.5). In T-5 (no occupant), 3 of 10 trials had transient quadrant highlights that correlated with operator motion in the adjacent hallway. This is the dominant contributor to the 7.5 percent false-positive rate.

11.1.6 Environmental Drift and Thermal Effects

ESP32-S3 oscillator frequency drifts with die temperature, which slowly biases the recovered phase reference of the CSI samples. The SVD drift-cancellation stage of the pipeline removes the dominant temporal mode per link and absorbs most of this effect, but the residual is largest for 60 to 90 seconds after power-on while the nodes thermally equilibrate. Trials initiated within 30 seconds of cold start showed a measurable increase in per-link variance. This is the reason for the cold-start guideline that the system stabilize for at least 60 seconds before the first valid trial.

11.1.7 HMM Filter Mis-Tuning

The four-state HMM forward filter was meant to smooth the soft-attribution output by encoding a kinematic prior that penalizes diagonal quadrant jumps. The transition matrix assumed continuous victim motion between adjacent quadrants. In practice, the soft-attribution score itself jumps to non-adjacent quadrants when the multipath pattern shifts, so the filter ended up fighting legitimate score transitions. The result was extra latency and a small number of misclassifications. This is a parameter-fitting issue rather than a structural one. Retraining the transition matrix on a larger logged-trial corpus, or replacing the HMM with a simpler debounce or majority-vote scheme, are both candidates for the next iteration.

11.1.8 Estimated Error Budget

Table 10. Estimated Error Budget

Error source	Estimated contribution to misclassifications (out of 8 / 40)
Multipath interference (geometric)	3
Co-channel Wi-Fi noise	1 to 2
Node placement tolerance	1
Out-of-perimeter operator or HVAC motion	1
HMM filter overlapping with multipath	1
Subject posture and breathing depth	1

The error budget sums to the 8 observed misclassifications and shows that the residual error is dominated by environmental and geometric effects rather than by software defects in the analytical pipeline. The latency shortfall is independent of these and is driven by the chosen breathing-band integration window. Section 13 covers the design alternatives that were tried and rejected, and Section 14 outlines the mitigations that should go into the next iteration.

12.0 DELIVERABLES

The deliverables of Project Glasshouse fall into five categories: hardware, embedded firmware, host software, public project artifacts, and documentation.

12.1 Hardware Deliverable

The primary hardware deliverable is the fully assembled five-node RF sensing prototype: four battery-powered ESP32-S3 perimeter nodes, one ESP32-S3 coordinator node, and a Raspberry Pi 4 central processing hub with a 7-inch IPS LCD touchscreen at 1024 by 600. Each perimeter node is housed in a 3D-printed PLA enclosure designed in SolidWorks and is fitted with an external 2.4 GHz 7 dBi SMA antenna, a LiPo battery, and an Adafruit PowerBoost 1000C charge module. The assembled hardware is photographed in Figures 2e and 3a-3d.

12.2 Embedded Firmware Deliverable

The embedded software deliverable is the ESP-IDF firmware for the perimeter and coordinator nodes. The perimeter firmware drives a Null Data Packet probe at about 50 Hz, captures CSI from peer responses through a promiscuous-mode Wi-Fi callback, runs an on-board edge-processing pipeline on a dedicated core, and forwards binary CSI frames to the coordinator over UDP. The coordinator firmware hosts the open Wi-Fi SoftAP and applies COBS encoding to each received UDP frame before bridging the encoded stream to the Raspberry Pi over USB-Serial at 921,600 baud.

12.3 Host Software Deliverable

The Raspberry Pi Python pipeline is the host software deliverable. It implements frame decoding, the Hampel/SVD/Welch signal-processing pipeline (Equations 7-9), soft endpoint attribution, the four-state HMM forward filter, and the LCD render loop.

12.4 Public Project Repository and Website Deliverables

To support reproducibility, peer review, and continuation of the project, two public deliverables were produced and are hosted independently of this printed report.

Table 11. Project Repository and Website URLs

Deliverable	URL	Contents
GitHub repository	https://github.com/jackjones1320/Project-Glass-House	Complete ESP-IDF firmware sources for the perimeter and

		coordinator nodes, the Raspberry Pi Python host pipeline, SolidWorks enclosure source files, bill of materials, and the assembly and operation user manual. The repository is laid out so that a third party can clone, flash, and run the system end-to-end without further information from the team.
Project website	https://csi-sar.vercel.app/	Public-facing project site built with Next.js and deployed on Vercel. The site presents project background, the methodology and signal-processing pipeline, photographs and renders of the assembled hardware, the test environment and validation results, and download links to the final report and the GitHub repository. The site is the externally accessible entry point to the project for sponsors, faculty, and prospective users.

The two public deliverables together satisfy the open-access reporting expectation for capstone projects at the university and ensure that the project remains discoverable and reproducible after submission.

12.5 Documentation Deliverable

This final project report is the comprehensive terminal documentation deliverable. A capstone project presentation and live system demonstration were delivered at the end of the semester, which provided proof-of-concept validation of the system in a controlled environment in front of faculty and peers. The IEEE-formatted references (Section 16.0), the full source-code listing (Appendix A), the electrical schematics and wiring diagrams (Appendix B), and the end-user manual (Appendix C) are included as appendices to this report and are also mirrored in the public GitHub repository in Section 12.4.

13.0 FAILED DESIGN APPROACHES

Engineering design is an inherently iterative process, and Project Glasshouse required several significant architectural revisions in response to technical limitations discovered during development.

13.1 Four Node Architecture

The original network architecture relied on four perimeter ESP32 nodes that transmitted CSI data wirelessly and directly to the central Raspberry Pi without an intermediary. CSI extraction on the ESP32 requires a specific promiscuous-mode Wi-Fi configuration that is incompatible with the simultaneous maintenance of a stable UDP stream to a non-ESP32 host, and this combination produced unacceptable data loss. The solution was to introduce a fifth ESP32-S3 as a dedicated coordinator node connected to the Raspberry Pi via USB, which hosts the local Wi-Fi network as an access point and forwards CSI data over the wired serial link. This change eliminated the wireless data loss problem entirely and formed the foundation of the final COBS-encoded serial bridge.

13.2 Random Forest Model

The original signal processing approach used a Random Forest machine learning classifier to fuse multi-node CSI data and estimate human location. This proved impractical because the model exhibited significant overfitting to the specific room in which training data was collected and could not generalize to new environments without retraining, which was unreasonable within the project's time and resource constraints. The classifier was replaced with a deterministic FFT-based pipeline that combines Hampel outlier rejection, Singular Value Decomposition (SVD) drift cancellation, and a Welch periodogram across the breathing band; this pipeline requires no pre-collected training data, adapts automatically to new environments, and produces reliable zone estimates from the first frames of operation.

13.3 ESP32-DevKitC-32UE

The original hardware specification called for the ESP32-DevKitC-32UE (WROOM-32UE) microcontroller. During development it became apparent that the ESP32-S3 offered significantly improved processing capability, greater onboard storage, and better data compression support, all of which are directly relevant to the on-board edge processing pipeline that performs variance-

based motion detection and vitals estimation from raw CSI data. The system was accordingly migrated to the ESP32-S3 platform.

13.4 ASCII CSV Serial Format

The earliest coordinator-to-host serial protocol used line-oriented ASCII CSV, with each CSI sample emitted as a comma-separated string terminated by a newline. CSV was chosen for human readability during bring-up. At the target 50 Hz CSI rate per node-pair across four nodes (twelve link directions and roughly 1.4 KB of subcarrier data per emission), the host UART consistently dropped bytes and partial lines beyond about 230,400 baud. Higher baud rates produced framing errors rather than throughput. The format was replaced with fixed-length binary frames encoded with Consistent Overhead Byte Stuffing (COBS) at 921,600 baud. COBS provides unambiguous frame delimiters with at most one byte of overhead per frame and recovers cleanly from any single-byte loss. The change resolved the throughput and framing problems and is the protocol used in the final system.

13.5 ESP-NOW Mesh

The pre-coordinator architecture used ESP-NOW as the inter-node transport instead of IEEE 802.11 UDP over a SoftAP. ESP-NOW offered connectionless peer-to-peer messaging without the overhead of association, DHCP, or an access point. In practice, the Espressif CSI extraction API does not coexist cleanly with ESP-NOW peer discovery and queueing on the same radio. Enabling ESP-NOW disabled the promiscuous-mode RX callback that the project relies on for CSI capture. Several attempts were made to time-multiplex CSI capture and ESP-NOW transmit in alternating windows, but the handshake jitter introduced into the breathing-band time series was unacceptable. The architecture was migrated to the SoftAP plus UDP unicast model used in the final system.

13.6 On-Board Random Forest Inference

A version of the Random Forest classifier was prototyped to run on the ESP32-S3 itself using a fixed-point inference library. The intent was to push more of the localization decision toward the edge and reduce the load on the Raspberry Pi. Two limitations made this impractical: the model that fit within the available flash and PSRAM after firmware overhead was too small to retain useful discriminative power, and the per-sample inference latency on the dual-core S3 caused dropped CSI frames whenever inference and capture overlapped on the same core. The

deterministic pipeline was moved to the host instead, which is the architecture used in the final system.

13.7 Single Centralized Antenna Array

An early architectural sketch placed a single multi-antenna ESP32-S3 at the center of the monitored area and computed pseudo-arrival information from per-antenna phase deltas. The approach was rejected for two reasons. The standard ESP32-S3 module supports only one Wi-Fi antenna chain, so any beamforming would have required an external phased array, which was outside the cost budget. Beyond cost, the centralized geometry produces a degenerate localization problem in the breathing band, since all four quadrants present nearly identical reflection patterns relative to a single central sensor. The distributed four-perimeter-node geometry restored the spatial diversity needed for quadrant-level discrimination at much lower hardware cost.

13.8 Fixed-Threshold Variance Detection

Before the Welch periodogram and soft endpoint attribution pipeline was adopted, a simpler fixed-threshold variance detector was used. Each per-link CSI amplitude time series was variance-gated and a flat threshold determined whether motion was present on that link. In bench tests the detector behaved well, but in the room-scale environment the variance of empty room links was strongly dependent on ambient Wi-Fi traffic on channel 6. The threshold that suppressed false positives in the empty room missed the breathing signature of a stationary subject, and the threshold that detected the stationary subject produced near-continuous false positives on unoccupied links. The detector was replaced with the band-limited Welch periodogram, which integrates only the 0.15 to 0.40 Hz energy band and is therefore largely immune to broadband Wi-Fi traffic variance.

13.9 Direct LCD Render From the Coordinator

To minimize the part count, the team briefly considered driving the LCD directly from the coordinator ESP32-S3 over SPI and removing the Raspberry Pi entirely. The display path was prototyped, but the Welch periodogram, the SVD drift cancellation, and the per-link ring buffer required more RAM than the ESP32-S3 PSRAM budget could support without dropping CSI frames at the 50 Hz target. The Raspberry Pi was retained as the host, and the coordinator's role was scoped to UDP-to-USB COBS bridging only.

13.10 5-Second Latency Target

Reaching the original 5-second end-to-end latency target was attempted but ultimately not achieved within the project window. The final system reaches first quadrant lock about 30 seconds after the breathing-band analytical window starts filling, then refreshes the display roughly every second. The cause is structural: the 0.15 to 0.40 Hz breathing band has a fundamental period of 2.5 to 6.7 seconds, and the Welch periodogram needs several full periods to produce a stable score. Two approaches were tried and rejected. Widening the band of interest to include faster motion content shortened the window but degraded the breathing-specificity that the system relies on to suppress false positives from operator motion. Running a parallel short-window estimator and trusting it during the first 30 seconds produced noisy, low-confidence quadrant flips that misled the operator more than they helped. The 5-second target is treated as future work in Section 14, with a Kalman or particle filter approach noted as a candidate replacement for the periodogram.

14.0 DISCUSSION

The results of the Project Glasshouse evaluation validate the core design approach: a distributed Wi-Fi CSI sensing network using a physics-driven, training-free signal processing pipeline is capable of localizing a human subject to the correct spatial quadrant at approximately 80% accuracy in a controlled indoor environment. This result confirms that breathing-band RF perturbations, captured across multiple node-pair links and attributed through soft endpoint weighting, carry sufficient spatial information to support quadrant-level victim localization without requiring line-of-sight access, wearable devices, or pre-deployment calibration.

The performance of the analytical pipeline met expectations. The Hampel outlier rejection, SVD drift cancellation, and Welch periodogram stages together produced per-link signal scores that were reliably dominated by the breathing signature of the test subject, and the soft endpoint attribution correctly identified the occupied quadrant in the large majority of trials. The absence of a warm-up period and the pipeline's automatic adaptation to the ambient RF environment confirmed that the approach is well suited to the rapid and unpredictable deployment conditions of SAR operations.

The system's sensitivity beyond the node perimeter is a practically important observation. In real SAR deployments a victim is unlikely to be positioned precisely within a defined four-node enclosure, and the extended detection range means that the system can provide useful localization information even when the victim is near but not strictly inside the monitored area. This behavior also implies that the system can be operated with the node perimeter placed around a suspected debris field rather than around the victim directly, which increases its utility in environments where access is restricted.

The underperformance of the Hidden Markov Model forward filter represents the primary discrepancy between expected and actual performance. The HMM was designed to reduce display oscillation between adjacent quadrants, but in practice the speed of the underlying attribution score transitions exceeded the assumptions built into the kinematic prior, and the filter introduced misclassification rather than stability. This finding suggests that a simpler debounce or majority-vote smoothing mechanism applied directly to the raw attribution output may provide more reliable temporal stability than a full probabilistic state-transition model, and this is a priority area for future development.

The principal practical limitation identified during testing is the system's sensitivity to environmental factors that originate outside the monitored enclosure. Movement by personnel operating the system, vibration from HVAC equipment, and other ambient RF perturbations can produce transient quadrant attributions that do not correspond to a victim. Deployment protocols that establish a quiet perimeter around the sensing enclosure during acquisition would reduce this effect in field use. Future work should also investigate adaptive threshold tuning that accounts for ambient perturbation levels measured during an initial calibration phase at each deployment site.

15.0 CONCLUSION

Project Glasshouse set out to design, build, and demonstrate a low-cost distributed RF sensing system capable of detecting and localizing a human subject within a monitored area using Wi-Fi CSI, without line-of-sight access, wearable devices, or entry into the monitored structure. That goal was achieved. The complete five-node system was assembled and validated at a total cost under $450, and functional testing in a controlled indoor environment demonstrated a quadrant identification accuracy of approximately 80%, which exceeds the 75% minimum target defined at the outset of the project.

The key technical contributions of the project are the deterministic, training-free signal processing pipeline and the distributed node architecture that together enable reliable quadrant-level localization from the first emission tick of operation. The Hampel outlier rejection, SVD drift cancellation, Welch periodogram, and soft endpoint attribution stages produced a robust and interpretable localization output without requiring labeled training data, environment-specific calibration, or cloud infrastructure. The COBS-encoded USB-Serial bridge reliably delivered CSI frames from the coordinator to the Raspberry Pi at 921,600 baud, and the full data pipeline from CSI capture to real-time LCD display was confirmed functional under sustained four-node operation.

The primary challenges encountered during development were the incompatibility between ESP32 promiscuous-mode CSI capture and simultaneous UDP transmission to a non-ESP32 host, which was resolved by introducing a dedicated coordinator node, and the underperformance of the Hidden Markov Model quadrant smoother, which imposed a kinematic prior that was too rigid for the speed of the observed attribution transitions. The core localization objective was met without the HMM layer, and its replacement with a simpler smoothing mechanism is identified as the highest-priority future improvement.

Project Glasshouse demonstrates that RF-based victim localization is achievable with commodity hardware at a fraction of the cost of commercial rescue radar systems, and that a physics-driven analytical pipeline can substitute for machine learning in environments where training data cannot be collected in advance. The system is ready to serve as a proof-of-concept platform for future work that expands the node count, integrates physiological monitoring output for triage prioritization, and develops adaptive thresholding for varied deployment environments. For the

team, the project produced engineering competencies spanning embedded systems, distributed wireless communication, signal processing, and mechanical enclosure design, and reinforced the value of iterative development and honest performance evaluation in producing systems that are genuinely useful rather than merely functional in ideal conditions.

15.1 Future Work

Several follow-on directions emerged during development and validation. They fall into three categories: hardware upgrades that improve sensing capability and field deployability, signal-processing changes that address the latency and HMM shortfalls identified in Section 11, and operational features that extend the system from a single-victim demonstration to a multi-victim triage tool.

15.1.1 Hardware Upgrades

Solar-panel enclosure lids: replace the current opaque PLA lid on each perimeter node with an integrated solar panel and charge regulator so the LiPo cell can be trickle-charged on site during daylight deployments. This eliminates the need to retrieve and recharge nodes between extended SAR operations and pairs naturally with the duty-cycled CSI sampling already in firmware.

Two-antenna nodes: upgrade every perimeter node from a single 7 dBi SMA antenna to a dual-antenna configuration. Two antennas per node give spatial diversity at each endpoint, which improves CSI signal-to-noise in multipath-heavy indoor environments and enables polarization or spatial diversity combining at the host. The ESP32-S3 module supports antenna selection via an external RF switch, which can be added to the perimeter carrier board with minimal layout impact.

General hardware refresh: tighten enclosure tolerances for repeatable antenna placement, move from breakout boards to a custom carrier PCB for the perimeter node (the wiring diagram in Appendix B is the starting point), add IP54 sealing for outdoor SAR operation, and switch to higher-capacity LiPo cells for longer field endurance.

Signal-processing improvements.

First-lock latency reduction: replace or augment the 90-second Welch periodogram with a Kalman or particle filter that produces a usable estimate before its window is fully populated. The current 30-second first-lock latency, identified in Sections 11.1.1 and 13.10, is the largest gap between the design target and measured performance and is the highest-priority signal-processing improvement.

Channel hopping for multipath robustness: extend the SoftAP to rotate across Wi-Fi channels 1, 6, and 11 during acquisition. Multipath fading is frequency-dependent, so combining per-channel CSI windows averages out the worst geometric nulls identified in Section 11.1.2.

HMM retraining or replacement: either retrain the four-state HMM transition matrix on a larger logged-trial corpus that includes the rapid score transitions observed in this environment, or replace the HMM with a simpler debounce or majority-vote scheme. Section 11.1.7 details the mis-tuning behavior.

Automated node self-localization: at startup, have each perimeter node estimate its own position from the CSI link distances to its peers, removing the requirement for sub-half-meter manual placement noted in Section 11.1.4.

15.1.2 Operational and Feature Extrensions

Higher-resolution localization: scale the architecture from four perimeter nodes to six or eight, which subdivides the sensing area beyond the current four-quadrant grid and improves disambiguation when multiple victims are present.

Vitals output for triage: surface the breathing-rate estimate (already available inside the periodogram) on the LCD alongside the quadrant indicator. In a multi-victim collapse scenario this lets responders prioritize victims whose respiration is irregular or weakening.

Multi-victim discrimination: extend soft endpoint attribution to detect and separate two or more concurrent breathing signatures within the same monitored area, so the system reports per-victim quadrant and rate instead of a single dominant signal.

Field-deployable user interface: add a hardened, ruggedized housing for the central device, a sunlight-readable display, and a simple two-button control set so the system can be operated by responders wearing gloves under field conditions.

16.0 REFERENCES

[1] Y.-Y. Chiu, H. Omura, H.-E. Chen, and S.-C. Chen, "Indicators for post-disaster search and rescue efficiency developed using progressive death tolls," Sustainability, vol. 12, no. 19, p. 8262, Oct. 2020, doi: https://doi.org/10.3390/su12198262.

[2] National Disaster Search Dog Foundation, "About FEMA National US&R Response System," [Online]. Available: http://disasterdog.org/. [Accessed: Mar. 4, 2026].

[3] Voltrium Systems Pte Ltd, "What exactly can thermal cameras see through? Addressing common FAQs," Jan. 29, 2024. [Online]. Available: https://www.voltrium.com.sg/en/what-exactly-can-thermal-cameras-see-through-addressing-common-faqs/. [Accessed: Apr. 17, 2026].

[4] Savox, "Delsar: Life detector for USAR," Oct. 04, 2024. [Online]. Available: https://www.savox.com/products/search-and-rescue-kits/delsar. [Accessed: Mar. 4, 2026].

[5] Sensoft Technologies, "Rescue radar – detect survivors in rubble," Product Brochure. [Online]. Available: https://www.sensoft.ca/wp-content/uploads/2025/08/Rescue-Radar-brochure.pdf. [Accessed: Mar. 4, 2026].

[6] N. Damodaran, E. Haruni, M. Kokhkharova et al., "Device free human activity and fall recognition using WiFi channel state information (CSI)," CCF Transactions on Pervasive Computing and Interaction, vol. 2, pp. 1–17, 2020.

[7] C. Uysal and T. Filik, "A new RF sensing framework for human detection through the wall," IEEE Transactions on Vehicular Technology, vol. 72, no. 3, pp. 3600–3610, Mar. 2023.

[8] Espressif Systems, "ESP32 Datasheet," Espressif Documentation, 2025. [Online]. Available: https://documentation.espressif.com/esp32_datasheet_en.html. [Accessed: Apr. 05, 2026].

[9] P. Ishabakaki et al., "Non-invasive vital signs sensing: Advances, challenges, and future directions in radio frequency based techniques," IEEE Open Journal of Antennas and Propagation, vol. 7, no. 1, pp. 89–115, Feb. 2026, doi: https://doi.org/10.1109/ojap.2025.3632192.

[10] Adafruit Industries, "Adafruit PowerBoost 1000C load-share USB charge boost," [Online]. Available: https://cdn-learn.adafruit.com/downloads/pdf/adafruit-powerboost-1000c-load-share-usb-charge-boost.pdf. [Accessed: Apr. 5, 2026].

[11] M. Zhao, T. Li, M. Alsheikh, Y. Tian, H. Zhao, A. Torralba, and D. Katabi, "Through-wall human mesh recovery using radio signals," in Proc. IEEE/CVF Int. Conf. Comput. Vision (ICCV), 2019, pp. 10112–10121.

[12] M. Raja, A. Hughes, Y. Xu, P. Zarei, D. G. Michelson, and S. Sigg, "Wireless multifrequency feature set to simplify human 3-D pose estimation," IEEE Antennas and Wireless Propagation Letters, vol. 18, no. 5, pp. 876–880, May 2019.

[13] A. Rana, H. C. Kumawat, and A. A. B. Raj, "Wall see through low-velocity moving target detection," in Proc. IEEE 3rd Int. Conf. Technol., Engineering, Management Societal Impact Using Marketing, Entrepreneurship and Talent (TEMSMET), 2023, pp. 1–6.

[14] S. Jo, S. Park, and G. I. Kwon, "Indoor human detection from a building's exterior using 433 MHz wireless transceivers," Sensors (Basel), vol. 23, no. 14, p. 6280, Jul. 2023.

[15] C.-Y. Hsu, Y. Liu, Z. Kabelac, R. Hristov, D. Katabi, and C. Liu, "Extracting gait velocity and stride length from surrounding radio signals," in Proc. CHI Conf. Human Factors Comput. Syst., 2017, pp. 2116–2126.

[16] MaliosDark, "WiFi-3D-Fusion," GitHub Repository, 2026. [Online]. Available: https://github.com/MaliosDark/wifi-3d-fusion. [Accessed: Apr. 5, 2026].

[17] F. Fallucchi, R. Giugliano, G. Lini, and A. Vizzarri, "Location intelligence system for people estimation in indoor environment during emergency operation," in Proc. 55th Hawaii Int. Conf. Syst. Sci., 2022, pp. 5734–5743.

[18] R. Vetsch, S. Kranz, T. Pittorino, P. de Baets, M. Chateauvieux, C. Wursch, D. Lenz, and S. Gros, "Sky savers: Leveraging drone technology for victim localization in avalanche rescue via transceiver signal analysis," in Proc. 22nd Int. Conf. Informatics Control, Automation and Robotics, vol. 1, 2023, pp. 469–476.

[19] M. Kabiri, C. Cimarelli, H. Bavle, J. L. Sanchez-Lopez, and H. Voos, "A review of radio frequency based localisation for aerial and ground robots with 5G future perspectives," Sensors (Basel), Dec. 2022.

[20] Cambium Networks, "Antenna selection and positioning guide," White Paper. [Online]. Available: https://d2cpnw0u24fjm4.cloudfront.net/wp-content/uploads/Antenna-Selection-and-Positioning-Guide.pdf. [Accessed: Apr. 5, 2026].

[21] IEEE Standard for Electromagnetic Energy Safety Programs, 0 Hz to 300 GHz, IEEE Std. C95.7-2022, 2023.

[22] IndiaMART, "Rescue radar for disaster management – find survivors in rubble." [Online]. Available: https://www.indiamart.com/proddetail/rescue-radar-for-disaster-management-find-survivors-in-rubble-ndrf-sdrf-2857600739591.html. [Accessed: Mar. 4, 2026].

[23] P. Gottesfeld et al., “Potential environmental and human health impacts of rechargeable lithium batteries in electronic waste,” Environmental Science and Technology, vol. 52, no. 8, pp. 4849–4858, 2018.

[24] E. Gallagher et al., “Life cycle assessment of circular consumer electronics based on IC recycling and emerging PCB assembly materials,” Scientific Reports, vol. 14, Nov. 2024.

[25] B. Bertrand et al., “The environmental footprint of IC production: Review, analysis, and lessons from historical trends,” IEEE Transactions on Semiconductor Manufacturing, vol. 36, no. 1, pp. 2–14, Feb. 2023.

[26] Felfil, “Filament recycler — make recycled 3D printing filament,” May 27, 2025. [Online]. Available: https://felfil.com/filament-recycler/?v=0b3b97fa6688. [Accessed: Apr. 2, 2026].

[27] FCC, “Evaluating compliance with FCC guidelines for human exposure to radiofrequency electromagnetic fields,” Office of Engineering and Technology Bulletin 65, 1997.

APPENDICES

Appendix A. Full Programming Code

The complete source code for the Project Glasshouse system is included below, comprising the ESP32-S3 coordinator firmware, the ESP32-S3 perimeter node firmware, and the Raspberry Pi Python host signal-processing pipeline.

```
// GlassHouse  Coordinator — SoftAP + UDP-to-COBS-serial bridge


#include <stdarg.h>
#include <stdbool.h>
#include <stdio.h>
#include <string.h>
#include "esp_event.h"
#include "esp_log.h"
#include "esp_wifi.h"
#include "esp_timer.h"
#include "nvs_flash.h"
#include "lwip/sockets.h"
#include "lwip/stats.h"  /* lwip_stats global for periodic UDP/mailbox counters */
#include "driver/uart.h"
#include "hal/usb_serial_jtag_ll.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/portmacro.h"
#include "nvs.h"
#include "sdkconfig.h"
```

```
/* Compile-time bridge-mode assertion. Kconfig `choice` guarantees exactly one
 * selection at configure time, but this guards against a hand-edited or
 * corrupted sdkconfig. */
#if (defined(CONFIG_GH_BRIDGE_USB) + defined(CONFIG_GH_BRIDGE_WIFI) +
defined(CONFIG_GH_BRIDGE_BOTH)) != 1
#error "Exactly one of CONFIG_GH_BRIDGE_USB/WIFI/BOTH must be set (via Kconfig
choice in Kconfig.projbuild)"
#endif

static const char *TAG = "coordinator";

#define SOFTAP_SSID       "CSI_NET_V2"   /* WARNING: now an OPEN AP — any STA in
range can join. */
#define SOFTAP_CHANNEL    1
#define SOFTAP_MAX_CONN   4              /* one slot per perimeter node; caps rogue auto-
joins. */
#define UDP_PORT          4210
// Data output goes via USB-Serial/JTAG (the built-in USB on ESP32-S3).
// We write directly to the hardware FIFO to avoid disrupting the USB connection.
// Log output is redirected to UART1 to avoid corrupting the COBS stream.
#define LOG_UART_NUM      UART_NUM_1
#define LOG_UART_TX_PIN   17
#define LOG_UART_RX_PIN   18
#define UART_BAUD         921600
/* Max forwarded UDP payload size; sized for full CSI frames. */
#define MAX_PKT_SIZE      2100
/* Drop logging cadence: warn on every Nth drop *or* every N seconds,
 * whichever comes first. */
```

```
#define DROP_LOG_EVERY_N       100
#define DROP_LOG_EVERY_US      (60LL * 1000LL * 1000LL)  /* 60 s */

// --- Log redirect: send ESP_LOG output to UART1 instead of USB-CDC ---
static int uart1_vprintf(const char *fmt, va_list args)
{
    char buf[256];
    int len = vsnprintf(buf, sizeof(buf), fmt, args);
    if (len > 0) {
        uart_write_bytes(LOG_UART_NUM, buf, len);
    }
    return len;
}

// --- Raw USB-Serial/JTAG FIFO write (bypasses driver to avoid USB disconnect) ---
// Keep retries low so a stalled USB host doesn't block the UDP receive loop.
// If the host isn't reading (port closed), drop the packet quickly and move on.
static size_t usb_jtag_write_raw(const uint8_t *data, size_t len)
{
    size_t offset = 0;
    int retries = 0;
    while (offset < len && retries < 5) {
        uint32_t written = usb_serial_jtag_ll_write_txfifo(data + offset, len - offset);
        usb_serial_jtag_ll_txfifo_flush();
        offset += written;
        if (offset < len) {
            vTaskDelay(1); // yield if FIFO was full, let USB drain
```

```
            retries++;
        }
    }
    return offset;
}

// --- COBS encoding ---
// Encodes `src` (len bytes) into `dst`. Returns encoded length.
// `dst` must be at least len + len/254 + 1 bytes.
static size_t cobs_encode(const uint8_t *src, size_t len, uint8_t *dst)
{
    size_t read_idx = 0, write_idx = 1, code_idx = 0;
    uint8_t code = 1;

    while (read_idx < len) {
        if (src[read_idx] == 0x00) {
            dst[code_idx] = code;
            code = 1;
            code_idx = write_idx++;
        } else {
            dst[write_idx++] = src[read_idx];
            code++;
            if (code == 0xFF) {
                dst[code_idx] = code;
                code = 1;
                code_idx = write_idx++;
            }
```

```
        }
        read_idx++;
    }
    dst[code_idx] = code;
    return write_idx;
}

// --- Operator-WiFi state + STA forward-socket lifecycle ---
typedef struct {
    char     ssid[32];
    char     psk[64];
    char     host_ip[16];
    uint16_t host_port;
} coord_opwifi_t;

static coord_opwifi_t s_opwifi;

#if CONFIG_GH_BRIDGE_WIFI || CONFIG_GH_BRIDGE_BOTH
/* Forward socket written by WiFi event task (STA_GOT_IP / STA_DISCONNECTED)
 * and read by udp_bridge_task's sendto — every access is guarded by the
 * portMUX to prevent torn reads/writes across cores. */
static portMUX_TYPE s_fwd_sock_mutex = portMUX_INITIALIZER_UNLOCKED;
static int          s_fwd_sock = -1;
static uint32_t     s_reconnect_attempts = 0;
#endif

/* Always called from app_main; the body is a no-op under USB-only so the
```

```
 * init sequence is identical across all three bridge modes. */
static esp_err_t coord_opwifi_load(coord_opwifi_t *out)
{
    memset(out, 0, sizeof(*out));

#if CONFIG_GH_BRIDGE_WIFI || CONFIG_GH_BRIDGE_BOTH
    /* Kconfig fallbacks (only emitted when WIFI/BOTH due to `depends on`). */
    strlcpy(out->ssid,    CONFIG_GH_OPWIFI_SSID,    sizeof(out->ssid));
    strlcpy(out->psk,     CONFIG_GH_OPWIFI_PSK,     sizeof(out->psk));
    strlcpy(out->host_ip, CONFIG_GH_OPWIFI_HOST_IP, sizeof(out->host_ip));
    out->host_port = (uint16_t)CONFIG_GH_OPWIFI_HOST_PORT;

    nvs_handle_t nvs;
    esp_err_t err = nvs_open("coord", NVS_READONLY, &nvs);
    if (err == ESP_ERR_NVS_NOT_FOUND) {
        /* First boot / post-erase: namespace not provisioned. Use Kconfig
         * defaults rather than failing boot on a missing NVS namespace. */
        ESP_LOGI(TAG, "opwifi: NVS namespace 'coord' not found — using Kconfig defaults");
        return ESP_OK;
    }
    if (err != ESP_OK) {
        ESP_LOGW(TAG, "opwifi: nvs_open failed (%s) — using Kconfig defaults",
                 esp_err_to_name(err));
        return ESP_OK;
    }

    size_t sz;
```

```
    sz = sizeof(out->ssid);    (void)nvs_get_str(nvs, "opwifi.ssid",    out->ssid,    &sz);
    sz = sizeof(out->psk);     (void)nvs_get_str(nvs, "opwifi.psk",     out->psk,     &sz);
    sz = sizeof(out->host_ip); (void)nvs_get_str(nvs, "opwifi.host_ip", out->host_ip, &sz);
    (void)nvs_get_u16(nvs, "opwifi.host_port", &out->host_port);
    nvs_close(nvs);

    ESP_LOGI(TAG, "opwifi: ssid='%s' host=%s:%u",
             out->ssid, out->host_ip, (unsigned)out->host_port);
#else
    (void)out; /* USB-only: no-op — loaded values are never consulted. */
#endif
    return ESP_OK;
}

#if CONFIG_GH_BRIDGE_WIFI || CONFIG_GH_BRIDGE_BOTH
static void coord_wifi_event_handler(void *arg, esp_event_base_t base,
                                     int32_t id, void *data)
{
    (void)arg;
    if (base == WIFI_EVENT && id == WIFI_EVENT_STA_START) {
        ESP_LOGI(TAG, "sta: interface started, connecting to '%s'", s_opwifi.ssid);
        esp_err_t err = esp_wifi_connect();
        if (err != ESP_OK) {
            ESP_LOGW(TAG, "sta: esp_wifi_connect (initial) returned %s",
                     esp_err_to_name(err));
        }
        return;
```

```
    }

    if (base == WIFI_EVENT && id == WIFI_EVENT_STA_DISCONNECTED) {
        /* Tear down forward socket under critical section; close() is called
         * outside the spinlock since it can block. */
        int old_sock;
        portENTER_CRITICAL(&s_fwd_sock_mutex);
        old_sock = s_fwd_sock;
        s_fwd_sock = -1;
        portEXIT_CRITICAL(&s_fwd_sock_mutex);
        if (old_sock >= 0) close(old_sock);

        /* Bounded exponential backoff. */
        if (CONFIG_GH_OPWIFI_RECONNECT_MAX_ATTEMPTS > 0 &&
            s_reconnect_attempts >=
(uint32_t)CONFIG_GH_OPWIFI_RECONNECT_MAX_ATTEMPTS) {
            ESP_LOGE(TAG, "sta: disconnected; max reconnect attempts (%u) reached, giving up",
                    (unsigned)CONFIG_GH_OPWIFI_RECONNECT_MAX_ATTEMPTS);
            return;
        }

        uint32_t attempt  = s_reconnect_attempts++;
        uint32_t delay_ms = (uint32_t)CONFIG_GH_OPWIFI_RECONNECT_MS;
        for (uint32_t i = 0; i < attempt && delay_ms < 30000; i++) {
            delay_ms <<= 1;
        }
        if (delay_ms > 30000) delay_ms = 30000;
```

```
    ESP_LOGW(TAG, "sta: disconnected; retrying in %u ms (attempt %u)",
            (unsigned)delay_ms, (unsigned)(attempt + 1));
    /* Blocks the default event loop task for up to 30 s during backoff.
     * Acceptable on a stable operator hotspot; a dedicated reconnect task
     * or esp_timer one-shot would be a production hardening follow-up. */
    vTaskDelay(pdMS_TO_TICKS(delay_ms));
    esp_err_t err = esp_wifi_connect();
    if (err != ESP_OK) {
        ESP_LOGW(TAG, "sta: esp_wifi_connect (retry) returned %s",
                esp_err_to_name(err));
    }
    return;
}

if (base == IP_EVENT && id == IP_EVENT_STA_GOT_IP) {
    ip_event_got_ip_t *event = (ip_event_got_ip_t *)data;
    s_reconnect_attempts = 0;

    int sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock < 0) {
        ESP_LOGE(TAG, "sta: got_ip but socket() failed: errno=%d", errno);
        return;
    }

    int stale_sock;
    portENTER_CRITICAL(&s_fwd_sock_mutex);
```

```
        stale_sock = s_fwd_sock;
        s_fwd_sock = sock;
        portEXIT_CRITICAL(&s_fwd_sock_mutex);
        if (stale_sock >= 0) close(stale_sock);

        ESP_LOGI(TAG, "sta: connected, ip=" IPSTR,
                 IP2STR(&event->ip_info.ip));
    }
}
#endif

// --- WiFi SoftAP setup ---
static void wifi_init_softap(void)
{
    esp_netif_create_default_wifi_ap();
#if CONFIG_GH_BRIDGE_WIFI || CONFIG_GH_BRIDGE_BOTH
    esp_netif_create_default_wifi_sta();
#endif

    wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
    ESP_ERROR_CHECK(esp_wifi_init(&cfg));

    /* OPEN auth is used here to avoid WPA2 4-way handshake contention that
     * blocks more than one perimeter from associating concurrently under
     * continuous WIFI_PS_NONE traffic. The network is ephemeral and only
     * carries CSI telemetry, so the open-auth tradeoff is acceptable. */
    wifi_config_t ap_config = {
```

```
        .ap = {
            .ssid = SOFTAP_SSID,
            .ssid_len = strlen(SOFTAP_SSID),
            .channel = SOFTAP_CHANNEL,
            .password = "",
            .max_connection = SOFTAP_MAX_CONN,
            .authmode = WIFI_AUTH_OPEN,
        },
    };

#if CONFIG_GH_BRIDGE_WIFI || CONFIG_GH_BRIDGE_BOTH
    /* APSTA mode: SoftAP serves perimeter nodes; STA joins the operator WiFi
     * using credentials loaded from NVS (with Kconfig fallbacks). The single
     * radio time-shares both interfaces, so they must end up on the same
     * channel; operator hotspot SOP is channel 1 to match the SoftAP. */
    wifi_config_t sta_config = { 0 };
    strlcpy((char *)sta_config.sta.ssid,     s_opwifi.ssid, sizeof(sta_config.sta.ssid));
    strlcpy((char *)sta_config.sta.password, s_opwifi.psk,  sizeof(sta_config.sta.password));
    /* Leave .threshold.authmode at default so the driver matches whatever the
     * operator hotspot advertises (OPEN or WPA2-PSK). */

    ESP_ERROR_CHECK(esp_event_handler_register(
        WIFI_EVENT, WIFI_EVENT_STA_START,        &coord_wifi_event_handler, NULL));
    ESP_ERROR_CHECK(esp_event_handler_register(
        WIFI_EVENT, WIFI_EVENT_STA_DISCONNECTED, &coord_wifi_event_handler,
NULL));
    ESP_ERROR_CHECK(esp_event_handler_register(
```

```
    IP_EVENT,   IP_EVENT_STA_GOT_IP,        &coord_wifi_event_handler, NULL));

  /* Single-radio ESP32 cannot serve AP and STA on different channels — it
   * time-shares and association auth frames get dropped otherwise. Setting
   * ap.channel=0 lets the driver auto-sync the SoftAP to whatever channel
   * the STA lands on post-association. The USB-only #else branch below
   * keeps SOFTAP_CHANNEL=1 unchanged. */
  ap_config.ap.channel = 0;

  ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_APSTA));
  ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_AP,  &ap_config));
  ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_STA, &sta_config));
  ESP_ERROR_CHECK(esp_wifi_start());

  /* SoftAP runs at 40 MHz (HT40+, secondary above — the only valid
   * orientation on channel 1) to double per-perimeter CSI delay-domain
   * resolution. Paired with perimeter-side AMPDU_RX disabled + 11n. */
  ESP_ERROR_CHECK(esp_wifi_set_bandwidth(WIFI_IF_AP, WIFI_BW_HT40));

  ESP_LOGI(TAG, "APSTA started: AP=%s/CH=auto/OPEN MAX=%d HT40+;
STA→'%s' (operator)",
           SOFTAP_SSID, SOFTAP_MAX_CONN, s_opwifi.ssid);
#else
  /* USB-only build: SoftAP-only WiFi mode. */
  ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_AP));
  ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_AP, &ap_config));
  ESP_ERROR_CHECK(esp_wifi_start());
```

```
    /* HT40+ to match the APSTA branch above for consistent CSI bandwidth. */
    ESP_ERROR_CHECK(esp_wifi_set_bandwidth(WIFI_IF_AP, WIFI_BW_HT40));

    ESP_LOGI(TAG, "SoftAP started: SSID=%s CH=%d HT40+ AUTH=OPEN MAX=%d",
             SOFTAP_SSID, SOFTAP_CHANNEL, SOFTAP_MAX_CONN);
#endif
}

// --- UART setup ---
static void uart_init(void)
{
#if CONFIG_GH_BRIDGE_USB || CONFIG_GH_BRIDGE_BOTH
    /* Redirect ESP log output to UART1 so USB-CDC stays clean for COBS data.
     * Only needed in USB/BOTH modes; in WIFI-only mode USB-CDC is idle and
     * ESP_LOG via USB-Serial/JTAG is the natural observability path (visible
     * via `idf.py monitor`). */
    uart_config_t log_cfg = {
        .baud_rate = 115200,
        .data_bits = UART_DATA_8_BITS,
        .parity = UART_PARITY_DISABLE,
        .stop_bits = UART_STOP_BITS_1,
        .flow_ctrl = UART_HW_FLOWCTRL_DISABLE,
    };
    ESP_ERROR_CHECK(uart_param_config(LOG_UART_NUM, &log_cfg));
    ESP_ERROR_CHECK(uart_set_pin(LOG_UART_NUM, LOG_UART_TX_PIN,
LOG_UART_RX_PIN,
```

```
                                UART_PIN_NO_CHANGE, UART_PIN_NO_CHANGE));
    ESP_ERROR_CHECK(uart_driver_install(LOG_UART_NUM, 1024, 1024, 0, NULL, 0));
    esp_log_set_vprintf((vprintf_like_t)&uart1_vprintf);
#endif /* CONFIG_GH_BRIDGE_USB || CONFIG_GH_BRIDGE_BOTH */
}

// --- UDP listener task ---
static void udp_bridge_task(void *pvParameters)
{
    /* rx_buf sized to hold a full CSI frame.
     * cobs_buf has worst-case COBS expansion: +1 header per 254 bytes + delimiter. */
    static uint8_t rx_buf[MAX_PKT_SIZE];
#if CONFIG_GH_BRIDGE_USB || CONFIG_GH_BRIDGE_BOTH
    static uint8_t cobs_buf[MAX_PKT_SIZE + MAX_PKT_SIZE / 254 + 2];
    uint32_t usb_write_fail_count = 0;
#endif
#if CONFIG_GH_BRIDGE_WIFI || CONFIG_GH_BRIDGE_BOTH
    uint32_t wifi_sendto_fail_count = 0;
#endif
    uint32_t pkt_count = 0;

    /* 30s-window stats log. Reads come from the global lwip_stats struct
     * which is updated from the lwIP core task; intra-counter tearing on
     * u32 reads is acceptable for diagnostic monotonic counters. */
    int64_t  s_stats_window_start_us = esp_timer_get_time();
    uint32_t s_stats_pkts_at_window_start = 0;
#if LWIP_STATS
```

```
    uint32_t s_stats_drop_at_start   = lwip_stats.udp.drop;
    uint32_t s_stats_memerr_at_start = lwip_stats.udp.memerr;
    uint32_t s_stats_mbox_at_start   = lwip_stats.sys.mbox.err;
#endif

    /* Track and periodically log oversized-drop events. */
    uint32_t oversize_drop_count = 0;
    uint32_t oversize_drops_since_log = 0;
    uint32_t oversize_max_seen = 0;
    int64_t  last_drop_log_us = 0;

    struct sockaddr_in addr = {
        .sin_family = AF_INET,
        .sin_port = htons(UDP_PORT),
        .sin_addr.s_addr = htonl(INADDR_ANY),
    };

    int sock = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
    if (sock < 0) {
        ESP_LOGE(TAG, "Socket creation failed: %d", errno);
        vTaskDelete(NULL);
        return;
    }

    if (bind(sock, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        ESP_LOGE(TAG, "Socket bind failed: %d", errno);
        close(sock);
```

```
        vTaskDelete(NULL);
        return;
    }

    ESP_LOGI(TAG, "UDP bridge listening on port %d", UDP_PORT);

    while (1) {
        int len = recvfrom(sock, rx_buf, sizeof(rx_buf), 0, NULL, NULL);
        if (len <= 0) continue;
        if ((size_t)len > MAX_PKT_SIZE) {
            /* Count drops, track the largest oversized packet seen, and warn
             * via ESP_LOGW (routed to UART1 by uart1_vprintf) every
             * DROP_LOG_EVERY_N drops or every DROP_LOG_EVERY_US, whichever
             * comes first. */
            oversize_drop_count++;
            oversize_drops_since_log++;
            if ((uint32_t)len > oversize_max_seen) oversize_max_seen = (uint32_t)len;

            int64_t now = esp_timer_get_time();
            bool count_gate = (oversize_drops_since_log >= DROP_LOG_EVERY_N);
            bool time_gate  = (last_drop_log_us != 0) &&
                              ((now - last_drop_log_us) >= DROP_LOG_EVERY_US);
            if (count_gate || time_gate || last_drop_log_us == 0) {
                ESP_LOGW(TAG,
                    "oversize drop: len=%d > MAX_PKT_SIZE=%d (total=%lu, since_last=%lu,
max_seen=%lu)",
                    len, MAX_PKT_SIZE,
```

```
                (unsigned long)oversize_drop_count,
                (unsigned long)oversize_drops_since_log,
                (unsigned long)oversize_max_seen);
            oversize_drops_since_log = 0;
            last_drop_log_us = now;
        }
        continue;
    }

#if CONFIG_GH_BRIDGE_USB || CONFIG_GH_BRIDGE_BOTH
    /* COBS-over-USB-Serial/JTAG forward path. */
    size_t cobs_len = cobs_encode(rx_buf, (size_t)len, cobs_buf);
    cobs_buf[cobs_len] = 0x00; // COBS delimiter
    size_t written = usb_jtag_write_raw(cobs_buf, cobs_len + 1);
    if (written < cobs_len + 1) {
        usb_write_fail_count++;
    }
#endif

#if CONFIG_GH_BRIDGE_WIFI || CONFIG_GH_BRIDGE_BOTH
    /* Raw UDP forward to operator host. No COBS — UDP is already framed.
     * Read s_fwd_sock under portMUX, then sendto OUTSIDE the critical
     * section to avoid holding the spinlock across a potentially
     * blocking lwIP call. */
    int sock_snapshot;
    portENTER_CRITICAL(&s_fwd_sock_mutex);
    sock_snapshot = s_fwd_sock;
```

```
    portEXIT_CRITICAL(&s_fwd_sock_mutex);
    if (sock_snapshot >= 0) {
        struct sockaddr_in dst = {
            .sin_family = AF_INET,
            .sin_addr.s_addr = inet_addr(s_opwifi.host_ip),
            .sin_port = htons(s_opwifi.host_port),
        };
        ssize_t sent = sendto(sock_snapshot, rx_buf, (size_t)len, 0,
                              (struct sockaddr *)&dst, sizeof(dst));
        if (sent < 0) {
            /* Rate-limited warn: at most one log per second, with the
             * suppressed-count included in the emitted message. */
            static int64_t  last_warn_us = 0;
            static uint32_t suppressed   = 0;
            wifi_sendto_fail_count++;
            int64_t now_us = esp_timer_get_time();
            if (now_us - last_warn_us > 1000000) {
                ESP_LOGW(TAG, "fwd sendto failed: errno=%d (suppressed %u similar in last
1s)",
                         errno, (unsigned)suppressed);
                last_warn_us = now_us;
                suppressed = 0;
            } else {
                suppressed++;
            }
        }
    }
```

```
#endif

    pkt_count++;
    if (pkt_count % 100 == 0) {
#if CONFIG_GH_BRIDGE_USB
        ESP_LOGI(TAG, "Forwarded %lu packets (%lu USB write failures)",
                (unsigned long)pkt_count, (unsigned long)usb_write_fail_count);
#elif CONFIG_GH_BRIDGE_WIFI
        ESP_LOGI(TAG, "Forwarded %lu packets (%lu WiFi sendto failures)",
                (unsigned long)pkt_count, (unsigned long)wifi_sendto_fail_count);
#elif CONFIG_GH_BRIDGE_BOTH
        ESP_LOGI(TAG, "Forwarded %lu packets (USB fails=%lu, WiFi fails=%lu)",
                (unsigned long)pkt_count,
                (unsigned long)usb_write_fail_count,
                (unsigned long)wifi_sendto_fail_count);
#endif
    }

    /* Emit stats every 30s. Log format:
     *   coord_rx_stats: pkts=(\d+) udp\.drop=(\d+) udp\.memerr=(\d+) mbox\.err=(\d+)
30s_pps=([\d.]+) */
#if LWIP_STATS
    int64_t now_us = esp_timer_get_time();
    if ((now_us - s_stats_window_start_us) >= 30LL * 1000000LL) {
        double window_s = (double)(now_us - s_stats_window_start_us) / 1000000.0;
        uint32_t d_pkts = pkt_count - s_stats_pkts_at_window_start;
        double   pps    = window_s > 0 ? (double)d_pkts / window_s : 0.0;
```

```
        ESP_LOGI(TAG,
            "coord_rx_stats: pkts=%lu udp.drop=%lu udp.memerr=%lu mbox.err=%lu
30s_pps=%.1f",
            (unsigned long)pkt_count,
            (unsigned long)lwip_stats.udp.drop,
            (unsigned long)lwip_stats.udp.memerr,
            (unsigned long)lwip_stats.sys.mbox.err,
            pps);
        /* Reset snapshot for next window. Absolute values are emitted;
         * downstream analysis computes deltas across consecutive lines. */
        s_stats_window_start_us      = now_us;
        s_stats_pkts_at_window_start = pkt_count;
        s_stats_drop_at_start        = lwip_stats.udp.drop;
        s_stats_memerr_at_start      = lwip_stats.udp.memerr;
        s_stats_mbox_at_start        = lwip_stats.sys.mbox.err;
        (void)s_stats_drop_at_start;
        (void)s_stats_memerr_at_start;
        (void)s_stats_mbox_at_start;
      }
#endif
   }
}

void app_main(void)
{
   ESP_ERROR_CHECK(nvs_flash_init());
   ESP_ERROR_CHECK(esp_netif_init());
```

```
    ESP_ERROR_CHECK(esp_event_loop_create_default());

    uart_init();

    /* Always load opwifi state. Body is a no-op under CONFIG_GH_BRIDGE_USB=y;
     * the loaded struct is only consulted when WIFI or BOTH is active. */
    ESP_ERROR_CHECK(coord_opwifi_load(&s_opwifi));

    wifi_init_softap();

    xTaskCreate(udp_bridge_task, "udp_bridge", 4096, NULL, 5, NULL);

    ESP_LOGI(TAG, "GlassHouse v2 Coordinator ready");
}
```

```
/**
 * @file main.c
 * @brief ESP32-S3 CSI perimeter node firmware.
 *
 * Initializes NVS, WiFi STA mode, CSI collection, and UDP streaming.
 * CSI frames are serialized into binary packets and forwarded to the
 * coordinator over UDP for relay to the host aggregator.
 */

#include <string.h>
#include "freertos/FreeRTOS.h"
```

```
#include "freertos/task.h"
#include "freertos/event_groups.h"
#include "esp_system.h"
#include "esp_wifi.h"
#include "esp_event.h"
#include "esp_log.h"
#include "nvs_flash.h"
#include "esp_app_desc.h"
#include "sdkconfig.h"

#include "csi_collector.h"
#include "stream_sender.h"
#include "nvs_config.h"
#include "edge_processing.h"
#include "heartbeat.h"
#include "link_reporter.h"
#include "telemetry_sender.h"
#include "send_watchdog.h"
#include "crash_counter.h"
#include "power_mgmt.h"

static const char *TAG = "main";

/* Runtime configuration (loaded from NVS or Kconfig defaults). */
nvs_config_t g_nvs_config;

/* Event group bits */
```

```
#define WIFI_CONNECTED_BIT BIT0
#define WIFI_FAIL_BIT      BIT1

static EventGroupHandle_t s_wifi_event_group;
static int s_retry_num = 0;
#define MAX_RETRY 10

static void event_handler(void *arg, esp_event_base_t event_base,
                          int32_t event_id, void *event_data)
{
    if (event_base == WIFI_EVENT && event_id == WIFI_EVENT_STA_START) {
        esp_wifi_connect();
    } else if (event_base == WIFI_EVENT && event_id ==
WIFI_EVENT_STA_DISCONNECTED) {
        s_retry_num++;
        if (s_retry_num <= MAX_RETRY) {
            ESP_LOGI(TAG, "Retrying WiFi connection (%d/%d)", s_retry_num, MAX_RETRY);
        } else if (s_retry_num % 50 == 0) {
            ESP_LOGW(TAG, "WiFi reconnect attempt %d (will keep trying)", s_retry_num);
        }
        /* Always retry — perimeter nodes must reconnect if coordinator reboots */
        vTaskDelay(pdMS_TO_TICKS(s_retry_num > MAX_RETRY ? 2000 : 100));
        esp_wifi_connect();
    } else if (event_base == IP_EVENT && event_id == IP_EVENT_STA_GOT_IP) {
        ip_event_got_ip_t *event = (ip_event_got_ip_t *)event_data;
        ESP_LOGI(TAG, "Got IP: " IPSTR, IP2STR(&event->ip_info.ip));
        s_retry_num = 0;
```

```
        xEventGroupSetBits(s_wifi_event_group, WIFI_CONNECTED_BIT);
    }
}

static void wifi_init_sta(void)
{
    s_wifi_event_group = xEventGroupCreate();

    ESP_ERROR_CHECK(esp_netif_init());
    ESP_ERROR_CHECK(esp_event_loop_create_default());
    esp_netif_create_default_wifi_sta();

    wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
    ESP_ERROR_CHECK(esp_wifi_init(&cfg));

    esp_event_handler_instance_t instance_any_id;
    esp_event_handler_instance_t instance_got_ip;
    ESP_ERROR_CHECK(esp_event_handler_instance_register(
        WIFI_EVENT, ESP_EVENT_ANY_ID, &event_handler, NULL, &instance_any_id));
    ESP_ERROR_CHECK(esp_event_handler_instance_register(
        IP_EVENT, IP_EVENT_STA_GOT_IP, &event_handler, NULL, &instance_got_ip));

    wifi_config_t wifi_config = {
        .sta = {
            /* Accept OPEN or WPA2 APs (coordinator may run either). The
             * threshold is the *minimum* security; the actual auth used is
             * whatever the AP advertises. OPEN here lets the STA join an
```

```
         * OPEN coordinator even when NVS still has a password from a
         * prior WPA2 provisioning. */
        .threshold.authmode = WIFI_AUTH_OPEN,
    },
};

/* Copy runtime SSID from NVS config. */
strncpy((char *)wifi_config.sta.ssid, g_nvs_config.wifi_ssid, sizeof(wifi_config.sta.ssid) - 1);

/* Coordinator SoftAP is OPEN. ESP-IDF STA, when password is non-empty,
 * attempts WPA2 even if the AP is OPEN — the auth handshake then fails
 * because the AP is not expecting a password. Leaving the password
 * field zeroed forces the STA to associate as OPEN. The NVS password
 * is preserved so a later coordinator with WPA2 can use it; restore the
 * strncpy if/when the coordinator switches back to WPA2. */
/* strncpy((char *)wifi_config.sta.password, g_nvs_config.wifi_password,
sizeof(wifi_config.sta.password) - 1); */

ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_STA));
ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_STA, &wifi_config));
/* Enable 11b/g/n so HT40 negotiation succeeds with the coordinator's
 * SoftAP. AMPDU_RX is disabled in sdkconfig.defaults to avoid a known
 * RX AMPDU de-aggregation defect in the WiFi driver. */
ESP_ERROR_CHECK(esp_wifi_set_protocol(WIFI_IF_STA,
    WIFI_PROTOCOL_11B | WIFI_PROTOCOL_11G | WIFI_PROTOCOL_11N));
ESP_ERROR_CHECK(esp_wifi_start());
```

```
    /* HT40 (40 MHz) bandwidth doubles CSI delay-domain resolution from 15 m
     * (HT20) to 7.5 m. The SoftAP advertises HT40+ (secondary above) on
     * channel 1; the STA negotiates to match. If the SoftAP does not
     * advertise HT40, this call is harmless — the STA stays at the AP's
     * advertised bandwidth. */
    ESP_ERROR_CHECK(esp_wifi_set_bandwidth(WIFI_IF_STA, WIFI_BW_HT40));

    ESP_LOGI(TAG, "WiFi STA initialized, connecting to SSID: %s", g_nvs_config.wifi_ssid);

    /* Wait for connection (up to 30s, then proceed — reconnect continues in background) */
    EventBits_t bits = xEventGroupWaitBits(s_wifi_event_group,
        WIFI_CONNECTED_BIT,
        pdFALSE, pdFALSE, pdMS_TO_TICKS(30000));

    if (bits & WIFI_CONNECTED_BIT) {
        ESP_LOGI(TAG, "Connected to WiFi");
    } else {
        ESP_LOGW(TAG, "WiFi not connected after 30s — continuing startup, reconnect in
background");
    }
}

void app_main(void)
{
    /* Initialize NVS */
    esp_err_t ret = nvs_flash_init();
    if (ret == ESP_ERR_NVS_NO_FREE_PAGES || ret ==
ESP_ERR_NVS_NEW_VERSION_FOUND) {
```

```
    ESP_ERROR_CHECK(nvs_flash_erase());
    ret = nvs_flash_init();
}
ESP_ERROR_CHECK(ret);

/* Bump the persistent crash counter based on esp_reset_reason(). Must
 * run AFTER nvs_flash_init() and BEFORE telemetry_sender_start() so the
 * first heartbeat carries the post-boot counter value. */
(void)crash_counter_init();

/* Load runtime config (NVS overrides Kconfig defaults) */
nvs_config_load(&g_nvs_config);

const esp_app_desc_t *app_desc = esp_app_get_description();
ESP_LOGI(TAG, "ESP32-S3 CSI Node — v%s — Node ID: %d",
         app_desc->version, g_nvs_config.node_id);

wifi_init_sta();

if (stream_sender_init_with(g_nvs_config.target_ip, g_nvs_config.target_port) != 0) {
    ESP_LOGE(TAG, "Failed to initialize UDP sender");
    return;
}

/* Register peer MACs from NVS before starting CSI. */
for (int i = 0; i < g_nvs_config.peer_count && i < 4; i++) {
    csi_collector_add_peer(g_nvs_config.peer_node_ids[i],
```

```
                        g_nvs_config.peer_macs[i]);
    }
    ESP_LOGI(TAG, "Registered %d peer(s) for pairwise CSI sensing",
            (g_nvs_config.peer_count < 4) ? g_nvs_config.peer_count : 4);

    csi_collector_init();

    /* Periodic NDP broadcast stimulates CSI callbacks on every peer at a
     * controlled cadence. Promiscuous mode also captures CSI from all
     * other 802.11 traffic, so the effective per-peer capture rate is
     * well above the NDP cadence alone. */
    csi_collector_start_ndp_probe(50000);  /* 50 ms period = 20 Hz */

    /* Start heartbeat (CSI stimulus) and link reporter. */
    heartbeat_start(g_nvs_config.target_ip, g_nvs_config.target_port, 50);

#if CONFIG_SAR_MODE
    uint32_t link_interval_ms = CONFIG_SAR_LINK_REPORT_INTERVAL_MS;
#else
    uint32_t link_interval_ms = CONFIG_LINK_REPORT_INTERVAL_MS;
#endif
    link_reporter_start(g_nvs_config.node_id, g_nvs_config.target_ip,
                        g_nvs_config.target_port, link_interval_ms);

    /* Structured telemetry frame 0xC5110009 at 1 Hz. */
    if (telemetry_sender_start() != ESP_OK) {
        ESP_LOGW(TAG, "telemetry_sender_start failed — continuing without telemetry");
```

```
}

/* Send-rate-gauge watchdog: monitors stream_sender progress and
 * escalates recovery when the send counter is flat. */
send_watchdog_start(g_nvs_config.node_id);

/* Start multi-frequency channel hopping if configured in NVS. */
if (g_nvs_config.channel_hop_count > 1) {
    ESP_LOGI(TAG, "Starting channel hopping: %u channels, dwell=%lu ms",
             (unsigned)g_nvs_config.channel_hop_count,
             (unsigned long)g_nvs_config.dwell_ms);
    csi_collector_set_hop_table(
        g_nvs_config.channel_list,
        g_nvs_config.channel_hop_count,
        g_nvs_config.dwell_ms);
}

/* Initialize edge processing pipeline. */
edge_config_t edge_cfg = {
    .tier            = g_nvs_config.edge_tier,
    .presence_thresh   = g_nvs_config.presence_thresh,
    .fall_thresh       = g_nvs_config.fall_thresh,
    .vital_window      = g_nvs_config.vital_window,
    .vital_interval_ms = g_nvs_config.vital_interval_ms,
    .top_k_count       = g_nvs_config.top_k_count,
    .power_duty        = g_nvs_config.power_duty,
};
```

```
    esp_err_t edge_ret = edge_processing_init(&edge_cfg);
    if (edge_ret != ESP_OK) {
        ESP_LOGW(TAG, "Edge processing init failed: %s (continuing without edge DSP)",
                 esp_err_to_name(edge_ret));
    }

    /* Initialize power management. */
    power_mgmt_init(g_nvs_config.power_duty);

    ESP_LOGI(TAG, "CSI streaming active -> %s:%d (edge_tier=%u)",
             g_nvs_config.target_ip, g_nvs_config.target_port,
             g_nvs_config.edge_tier);

    /* Main loop — keep alive */
    while (1) {
        vTaskDelay(pdMS_TO_TICKS(10000));
    }
}
```

```
"""Live UDP detector entry point.

Receives IQ packets from the coordinator's SoftAP forwarder, computes
per-link breathing-band Doppler signatures over a rolling 90-second
window, and emits a per-window quadrant prediction via the continuous-
attribution estimator + HMM smoother in `live.continuous_estimator`.

Pipeline
```

```
--------
At each emit tick (default every 1 s):
  1. Take the last 90 s of IQ records and group by unordered link
     (i, j) in {(1,2), ..., (3,4)}.
  2. For each link, build an (n_packets, n_subcarriers) magnitude
     matrix; optionally apply CARM-style SVD drift removal.
  3. Run Welch's method per subcarrier on the breathing band
     (0.15-0.40 Hz) to extract per-link active_fraction and peak SNR.
  4. Run the continuous-attribution node estimator: each link
     contributes 0.5 * signal_score to its two endpoint nodes,
     reliability-weighted by sqrt(n_packets / max_n_packets).
  5. Update the 4-state HMM forward filter; the argmax of the
     posterior is the displayed quadrant.

Geometry (4-corner room layout):
  Q1 = NW = node 2,  Q2 = NE = node 3,
  Q3 = SW = node 1,  Q4 = SE = node 4.

Usage
-----
   # Live UDP from coord SoftAP forwarder (no baseline needed)
   python -m live.deterministic_detector udp \
      --continuous-estimator --svd-drift-modes 1 \
      --localize --pygame --pygame-fullscreen
"""
from __future__ import annotations
```

```
import argparse
import json
import os
import queue
import sys
import threading
import time
from collections import defaultdict, deque
from pathlib import Path
from typing import Dict, Iterator, Tuple

import numpy as np

_HERE = Path(__file__).resolve()
_PYDIR = _HERE.parent.parent
if str(_PYDIR) not in sys.path:
    sys.path.insert(0, str(_PYDIR))

def _parse_iq_complex(record: dict) -> np.ndarray:
    """Decode an IQ JSONL record into a complex64 array.

    Mirrors the firmware-side IQ wire format. Two header layouts are
    supported (selected by ``wire_version``): v1 has an 8-byte header,
    v2 has a 10-byte header. After the header, the payload is
    ``iq_len`` bytes of interleaved int8 I/Q samples; pair them and
    cast to complex64.
    """
```

```
    iq_len = record.get("iq_len", 0)
    wire_version = record.get("wire_version", 1)
    header_size = 10 if wire_version == 2 else 8
    raw = record.get("raw", "")
    if not raw:
        return np.array([], dtype=np.complex64)
    full = bytes.fromhex(raw)
    if len(full) < header_size + iq_len:
        return np.array([], dtype=np.complex64)
    payload = full[header_size:header_size + iq_len]
    n_sc = len(payload) // 2
    if n_sc == 0:
        return np.array([], dtype=np.complex64)
    iq = np.frombuffer(payload[:n_sc * 2], dtype=np.int8)
    return (
        iq[0::2].astype(np.float32) + 1j * iq[1::2].astype(np.float32)
    ).astype(np.complex64)
from live import localizer_2d  # noqa: E402
from live.continuous_estimator import (  # noqa: E402
    continuous_node_estimator,
    HmmNodeSmoother,
    blockage_attribution,
)



# Canonical 4-node link list (sorted, lower-id first).
ALL_LINKS = [(1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4)]
```

```
def _svd_denoise_amps(amps, n_drift_modes=1, log_amplitude=True, eps=1e-9):
    """Remove top-K temporal modes via SVD (CARM-style drift cancellation).

    Args:
        amps: (n_pkts, n_sc) per-packet amplitude matrix for one link.
        n_drift_modes: how many leading singular vectors to discard.
            K=1 removes the dominant temporal mode (slow drift).
            K=2 removes drift + the next-strongest common mode.
        log_amplitude: operate in log10 space so multiplicative drift
            becomes additive. Reverts to linear after denoise.
        eps: clipping floor before log.

    Returns the denoised (n_pkts, n_sc) matrix in linear amplitude.

    Math: X (n_pkts x n_sc), columns centered. SVD gives X = U S Vt.
    The first column of U (a temporal pattern) and first row of Vt (a
    subcarrier pattern) define the strongest temporal-spatial mode.
    Slowly varying drift across all subcarriers concentrates in this
    mode; dropping it leaves only body-signature variation orthogonal
    to drift. Apply only on windows long enough that drift dominates
    breathing in the dominant mode (>= 30 s).
    """
    if amps.shape[0] < 4:
        return amps
    if log_amplitude:
```

```
        amps_op = np.log10(np.maximum(amps, eps))
    else:
        amps_op = amps.astype(np.float64, copy=True)
    col_mean = amps_op.mean(axis=0, keepdims=True)
    X = amps_op - col_mean
    try:
        U, S, Vt = np.linalg.svd(X, full_matrices=False)
    except np.linalg.LinAlgError:
        return amps
    keep = max(0, len(S) - max(0, n_drift_modes))
    if keep <= 0 or n_drift_modes >= len(S):
        return amps
    X_clean = (U[:, n_drift_modes:]
               * S[n_drift_modes:][None, :]) @ Vt[n_drift_modes:, :]
    amps_clean = X_clean + col_mean
    if log_amplitude:
        return np.power(10.0, amps_clean)
    return amps_clean



# Pipeline tunables. None of these are fitted parameters; they are
# hand-chosen safety margins.
WINDOW_SECONDS = 6.0
ADVANCE_SECONDS = 1.0
MAD_K = 3.0                 # outlier rejection threshold (Hampel-style)
RESET_GUARD_SECONDS = 5.0    # +/- around any node reboot in the live stream
```

```
# ESP-IDF esp_reset_reason_t enum -> human label. Keep this in sync
# with components/esp_system/include/esp_system.h. Unknown codes fall
# through to f"code {n}" so future IDF versions don't crash the
# display.
ESP_RESET_REASON_LABEL = {
    0:  "Unknown",
    1:  "Power-on",
    2:  "External pin",
    3:  "esp_restart()",
    4:  "Panic",
    5:  "Int WDT",
    6:  "Task WDT",
    7:  "Other WDT",
    8:  "Deep sleep",
    9:  "Brownout",
    10: "SDIO",
    11: "USB",
    12: "JTAG",
    13: "eFuse",
    14: "Power glitch",
    15: "CPU lockup",
}



def reset_reason_label(code):
    if code is None:
        return None
```

```
    return ESP_RESET_REASON_LABEL.get(int(code), f"code {int(code)}")


# Breathing-band Doppler detection physics:
# - Adult resting breathing: 12-20 breaths/min = 0.20-0.33 Hz
# - Athletes resting: 0.10-0.17 Hz; stressed up to ~0.50 Hz
# - Static multipath and drift produce 0 Hz content only; breathing
#   is the only mechanism creating structured power in this band.
# - 6 s window gives FFT resolution 0.167 Hz; breathing band spans
#   only 1-2 FFT bins (very low resolution).
# - 90 s window gives FFT resolution 0.011 Hz; breathing band spans
#   ~24 FFT bins. White-noise floor drops, breathing peak concentrates
#   in 1-2 bins of those 24 -> sharp peak-to-noise contrast.
BREATHING_BAND_HZ = (0.15, 0.40)
# Default Doppler buffer length. The variance gate keeps the short 6s
# window; Doppler uses its own rolling buffer for higher SNR.
BREATHING_WINDOW_SECONDS = 90.0
# Minimum buffer duration before Doppler scoring is meaningful. During
# the warm-up after a fresh start, the buffer fills from 0 to 90 s; we
# only score once at least 30 s of samples are available so the FFT has
# enough breath cycles (~10) to resolve the peak.
BREATHING_MIN_DURATION_S = 30.0
# Polynomial detrend order applied per-subcarrier before FFT. Order=2
# removes both DC and quadratic thermal drift over a 90 s window without
# eating into the 0.15 Hz breathing-band lower edge.
DETREND_ORDER = 2
# Threshold: fraction of spectral power that must lie in the breathing
# band (vs total non-DC power) to count a subcarrier as "showing
```

```
# breathing modulation". White noise distributes power uniformly, so
# the noise-floor for our 0.25 Hz band over a 0-10 Hz spectrum is
# ~0.025. Threshold 0.10 = 4x noise floor = body presence.
BREATHING_BAND_RATIO_THRESHOLD = 0.10
# Presence threshold: fraction of (link, subcarrier) pairs that must
# exceed BREATHING_BAND_RATIO_THRESHOLD for the room to be "occupied".
# Literature: 5-30% of subcarriers typically show clear breathing
# modulation (the ones whose multipath traverses the body). Threshold
# 0.05 = 5% of 1152 (link, sc) pairs = 58 pairs, well above noise.
BREATHING_PRESENCE_THRESHOLD = 0.05



def _link_key(node_id, peer_id):
    if node_id is None or peer_id is None or node_id == peer_id:
        return None
    a, b = sorted((int(node_id), int(peer_id)))
    if a < 1 or b > 4:
        return None
    return (a, b)



# ---------------------------------------------------------------------------
# Per-(link, subcarrier) amplitude computation with Hampel-style cleaning
# ---------------------------------------------------------------------------
def per_link_subcar_amps(records, mad_k=MAD_K):
    """Aggregate IQ records into per-link per-subcarrier mean amplitude
    with Hampel/MAD outlier rejection per subcarrier.
```

```
`records` should be IQ-typed; non-IQ records are ignored. Returns
{link: (n_sc,) ndarray} for links that had any contributing records.
"""
per_link_seq = defaultdict(list)  # link -> list of (n_sc,) magnitudes
for r in records:
    if r.get("type") != "iq":
        continue
    link = _link_key(r.get("node_id"), r.get("peer_id"))
    if link is None:
        continue
    csi = _parse_iq_complex(r)
    if csi.size == 0:
        continue
    per_link_seq[link].append(np.abs(csi).astype(np.float64))

out = {}
for link, packets in per_link_seq.items():
    if not packets:
        continue
    # Use the modal subcarrier count (most are 192 for HT40; HT20 fallback is 64).
    lens = [p.shape[0] for p in packets]
    common_len = int(np.median(lens))
    pkts = [p[:common_len] for p in packets if p.shape[0] >= common_len]
    if not pkts:
        continue
    M = np.stack(pkts, axis=0)  # (n_packets, n_sc)
```

```
        med = np.median(M, axis=0)
        mad = np.median(np.abs(M - med), axis=0)
        sigma = 1.4826 * mad
        thresh = mad_k * sigma
        keep = np.abs(M - med) <= thresh
        keep = np.where(sigma[None, :] > 0, keep, True)  # constant subcarrier -> keep all
        kept_sums = (M * keep).sum(axis=0)
        kept_cnts = keep.sum(axis=0).clip(min=1)
        out[link] = kept_sums / kept_cnts
    return out



def per_link_delta(window_records, baseline):
    """Compute per-(link, subcarrier) delta = current_mean - baseline."""
    cur = per_link_subcar_amps(window_records)
    deltas = {}
    for link, base_v in baseline.items():
        if link not in cur:
            continue
        if cur[link].shape != base_v.shape:
            continue
        deltas[link] = cur[link] - base_v
    return deltas



def per_link_doppler_power(window_records,
                           breathing_band=BREATHING_BAND_HZ,
```

```
                 min_packets=8,
                 min_duration_s=1.0,
                 detrend_order=DETREND_ORDER,
                 hampel_k=MAD_K,
                 use_hanning=True):
"""Compute per-(link, subcarrier) breathing-band spectral power ratio.

For each (link, subcarrier), build the time series of |IQ| within
this window, FFT it, and compute the fraction of spectral power
that falls in the breathing band (default 0.15-0.40 Hz). Returns
dict[link] -> (n_subcarriers,) ndarray of breathing-band ratios
in [0, 1].

Physics rationale:
- Static multipath + drift produce 0 Hz spectral content only.
- Breathing modulates body cross-section -> multipath length -> CSI
  amplitude with a periodic signal at 0.20-0.33 Hz typically.
- White-noise floor at 6 s window: ratio ~0.05 (band spans 2-3 of
  ~60 bins). At 90 s window: ~0.027 (24 of ~899 bins). Longer
  windows give a much sharper signal-vs-noise discrimination.

Pipeline per link:
  1. Stack |IQ| samples in time order -> (n_packets, n_sc) matrix.
  2. Hampel/MAD outlier filter ACROSS TIME per subcarrier: replace
     outliers with the per-subcarrier median (preserve uniform
     sampling required by FFT; dropping samples would create gaps).
  3. Polynomial detrend (default order=2): remove DC + linear +
```

```
    quadratic drift per subcarrier so thermal/gain trends don't
    leak power into the 0.15 Hz lower edge of the breathing band.
  4. Hanning window: ~30 dB sidelobe reduction so any residual DC
    leakage stays out of the breathing band.
  5. rFFT per subcarrier; power = |FFT|^2.
  6. Breathing-band ratio = sum(power in band) / sum(power non-DC).

Returns {} if a link has fewer than min_packets or shorter duration
than min_duration_s.
"""
per_link_samples = defaultdict(list)
for r in window_records:
    if r.get("type") != "iq":
        continue
    link = _link_key(r.get("node_id"), r.get("peer_id"))
    if link is None:
        continue
    csi = _parse_iq_complex(r)
    if csi.size == 0:
        continue
    t = float(r.get("t") if r.get("t") is not None else r.get("_t_rel", 0.0))
    per_link_samples[link].append((t, np.abs(csi).astype(np.float64)))

out = {}
for link, samples in per_link_samples.items():
    if len(samples) < min_packets:
        continue
```

```
samples.sort(key=lambda s: s[0])
ts = np.array([s[0] for s in samples])
# Use the modal subcarrier count to avoid HT20/HT40 mixing.
lens = [s[1].shape[0] for s in samples]
common_len = int(np.median(lens))
amps_list = [s[1][:common_len] for s in samples
            if s[1].shape[0] >= common_len]
if len(amps_list) < min_packets:
    continue
amps = np.stack(amps_list, axis=0)  # (n_packets, n_sc)
n_pkts, n_sc = amps.shape

duration = ts[-1] - ts[0]
if duration < min_duration_s:
    continue
fs = (n_pkts - 1) / duration  # effective sample rate (Hz)

# Step 2: Hampel filter across time per subcarrier. Replace
# outliers with per-subcarrier median to keep the time grid
# uniform for FFT.
med_t = np.median(amps, axis=0)
mad_t = np.median(np.abs(amps - med_t[None, :]), axis=0)
sigma_t = 1.4826 * mad_t
deviation = np.abs(amps - med_t[None, :])
outliers = (sigma_t[None, :] > 0) & (deviation > hampel_k * sigma_t[None, :])
if outliers.any():
    amps = np.where(outliers,
```

```
                np.broadcast_to(med_t[None, :], amps.shape),
                amps)

# Step 3: polynomial detrend per subcarrier.
# Build basis [1, x, x^2, ...] on x in [-1, 1] for numerical
# conditioning; solve least-squares per subcarrier, subtract.
if detrend_order >= 1:
    x = np.linspace(-1.0, 1.0, n_pkts).astype(np.float64)
    basis = np.vstack([x ** k for k in range(detrend_order + 1)]).T
    try:
        coeffs, *_ = np.linalg.lstsq(basis, amps, rcond=None)
        trend = basis @ coeffs
        amps_dt = amps - trend
    except np.linalg.LinAlgError:
        amps_dt = amps - amps.mean(axis=0)[None, :]
else:
    amps_dt = amps - amps.mean(axis=0)[None, :]

# Step 4: Hanning window (sidelobe rejection ~30 dB).
if use_hanning and n_pkts >= 4:
    win = np.hanning(n_pkts).astype(np.float64).reshape(-1, 1)
    amps_w = amps_dt * win
else:
    amps_w = amps_dt

# Step 5+6: FFT, power spectrum, band ratio.
spectrum = np.abs(np.fft.rfft(amps_w, axis=0))  # (n_freqs, n_sc)
```

```
        freqs = np.fft.rfftfreq(n_pkts, 1.0 / fs)

        in_band = (freqs >= breathing_band[0]) & (freqs <= breathing_band[1])
        all_band = freqs > 0  # exclude DC
        breathing_power = (spectrum[in_band] ** 2).sum(axis=0)
        total_power = (spectrum[all_band] ** 2).sum(axis=0)

        with np.errstate(divide="ignore", invalid="ignore"):
            ratio = np.where(total_power > 0,
                             breathing_power / total_power, 0.0)
        out[link] = ratio
    return out



def breathing_presence_score(per_link_doppler,
                             ratio_threshold=BREATHING_BAND_RATIO_THRESHOLD):
    """Fraction of (link, subcarrier) pairs whose breathing-band ratio
    exceeds the threshold. Empty rooms score near zero; a breathing body
    in any quadrant typically scores 0.05-0.30."""
    total = 0
    above = 0
    for arr in per_link_doppler.values():
        total += int(arr.size)
        above += int((arr > ratio_threshold).sum())
    if total == 0:
        return 0.0
    return float(above) / float(total)
```

```
def doppler_self_explain(window_records,
                         breathing_band=BREATHING_BAND_HZ,
                         ratio_threshold=BREATHING_BAND_RATIO_THRESHOLD,
                         min_packets=8,
                         min_duration_s=BREATHING_MIN_DURATION_S,
                         detrend_order=DETREND_ORDER):
    """Compute per-link Doppler self-explanation features.

    Returns a dict suitable for human-readable display:
    {
        'overall': {
            'peak_freq_hz': dominant breathing frequency across all links
            'peak_bpm': peak frequency * 60 (breaths per minute)
            'in_adult_breathing_range': bool
            'total_active_subcarriers': count above ratio threshold
            'total_subcarriers': total
            'active_fraction': ratio
        },
        'per_link': {
            link: {
                'peak_freq_hz': dominant freq in breathing band
                'peak_power': spectral magnitude at peak
                'mean_band_ratio': avg breathing-band power ratio
                'n_active_subcarriers': sc count above threshold
                'n_subcarriers': total sc count
```

```
            'active_fraction': fraction
        }
    }
}

All math identical to per_link_doppler_power but returns richer
per-link structure suitable for live status display.
"""
per_link_samples = defaultdict(list)
for r in window_records:
    if r.get("type") != "iq":
        continue
    link = _link_key(r.get("node_id"), r.get("peer_id"))
    if link is None:
        continue
    csi = _parse_iq_complex(r)
    if csi.size == 0:
        continue
    t = float(r.get("t") if r.get("t") is not None else r.get("_t_rel", 0.0))
    per_link_samples[link].append((t, np.abs(csi).astype(np.float64)))

per_link = {}
all_peak_freqs = []
total_active = 0
total_sc = 0

for link, samples in per_link_samples.items():
```

```
if len(samples) < min_packets:
    continue
samples.sort(key=lambda s: s[0])
ts = np.array([s[0] for s in samples])
lens = [s[1].shape[0] for s in samples]
common_len = int(np.median(lens))
amps_list = [s[1][:common_len] for s in samples
             if s[1].shape[0] >= common_len]
if len(amps_list) < min_packets:
    continue
amps = np.stack(amps_list, axis=0)
n_pkts, n_sc = amps.shape
duration = ts[-1] - ts[0]
if duration < min_duration_s:
    continue
fs = (n_pkts - 1) / duration

# Hampel + polynomial detrend + Hanning (same pipeline as power func)
med_t = np.median(amps, axis=0)
mad_t = np.median(np.abs(amps - med_t[None, :]), axis=0)
sigma_t = 1.4826 * mad_t
deviation = np.abs(amps - med_t[None, :])
outliers = (sigma_t[None, :] > 0) & (deviation > MAD_K * sigma_t[None, :])
if outliers.any():
    amps = np.where(outliers,
                    np.broadcast_to(med_t[None, :], amps.shape),
                    amps)
```

```
x = np.linspace(-1.0, 1.0, n_pkts).astype(np.float64)
basis = np.vstack([x ** k for k in range(detrend_order + 1)]).T
try:
    coeffs, *_ = np.linalg.lstsq(basis, amps, rcond=None)
    amps_dt = amps - basis @ coeffs
except np.linalg.LinAlgError:
    amps_dt = amps - amps.mean(axis=0)[None, :]
win = np.hanning(n_pkts).astype(np.float64).reshape(-1, 1)
amps_w = amps_dt * win

spectrum = np.abs(np.fft.rfft(amps_w, axis=0))  # (n_freqs, n_sc)
freqs = np.fft.rfftfreq(n_pkts, 1.0 / fs)
in_band = (freqs >= breathing_band[0]) & (freqs <= breathing_band[1])
all_band = freqs > 0
breathing_power = (spectrum[in_band] ** 2).sum(axis=0)
total_power = (spectrum[all_band] ** 2).sum(axis=0)
with np.errstate(divide="ignore", invalid="ignore"):
    ratio = np.where(total_power > 0,
                     breathing_power / total_power, 0.0)
n_active = int((ratio > ratio_threshold).sum())

# Peak in breathing band: average spectrum across subcarriers,
# find max in band. This is the dominant breathing frequency
# across this link's subcarriers.
avg_spec = spectrum.mean(axis=1)  # (n_freqs,)
if in_band.any():
    band_freqs = freqs[in_band]
```

```
        band_spec = avg_spec[in_band]
        peak_idx = int(np.argmax(band_spec))
        peak_freq = float(band_freqs[peak_idx])
        peak_power = float(band_spec[peak_idx])
    else:
        peak_freq = 0.0
        peak_power = 0.0

    per_link[link] = {
        "peak_freq_hz": peak_freq,
        "peak_bpm": peak_freq * 60.0,
        "peak_power": peak_power,
        "mean_band_ratio": float(np.mean(ratio)),
        "n_active_subcarriers": n_active,
        "n_subcarriers": int(n_sc),
        "active_fraction": float(n_active) / float(n_sc) if n_sc else 0.0,
        "n_packets": int(n_pkts),
    }
    all_peak_freqs.append(peak_freq)
    total_active += n_active
    total_sc += n_sc

overall = {}
if all_peak_freqs:
    # Overall peak: use median of per-link peaks (robust)
    peak_overall = float(np.median(all_peak_freqs))
    overall = {
```

```
        "peak_freq_hz": peak_overall,
        "peak_bpm": peak_overall * 60.0,
        "in_adult_breathing_range": (0.20 <= peak_overall <= 0.40),
        "total_active_subcarriers": total_active,
        "total_subcarriers": total_sc,
        "active_fraction": (total_active / total_sc) if total_sc else 0.0,
      }
  return {"overall": overall, "per_link": per_link}



def per_link_doppler_robust(window_records,
                            breathing_band=BREATHING_BAND_HZ,
                            min_packets=8,
                            min_duration_s=BREATHING_MIN_DURATION_S,
                            detrend_order=4,
                            hampel_k=MAD_K,
                            n_welch_segments=4,
                            exclude_band_edge_bins=1,
                            highpass_cutoff_hz=0.10,
                            svd_drift_modes=0):
  """Robust Doppler features per link, baseline-free.

  Improvements over per_link_doppler_power:
    1. **Cross-subcarrier coherent summation**: sums |FFT|^2 across
       all subcarriers BEFORE peak finding. Body breathing at frequency
       f modulates many subcarriers, all peaking at the same f. The
       coherent sum aligns those peaks while uncorrelated subcarrier
```

```
    noise averages out. Sharper peak detection.

  2. **Welch's method** (4 segments, 50% overlap): splits the 90s
     window into 4 overlapping 36s segments, FFTs each, averages the
     power spectra. Reduces spectral variance by ~2x at the cost of
     frequency resolution (0.028 Hz per bin vs 0.011 Hz).

  3. **Peak SNR metric** (peak_power / median_power_in_band): the
     body breathing produces a SHARP narrow peak. Empty room produces
     a flat spectrum. Peak SNR > 5 typically indicates real breathing;
     < 2 is noise.

  4. **Wider breathing band (0.10 - 0.50 Hz)**: catches both slow
     (athletes, deep breathing) and fast (stressed) breathing rates.
     Combined with peak-SNR filtering, the wider band doesn't add
     noise because we're looking at the PEAK, not integrated power.

Returns dict[link] -> {
    'peak_freq_hz': float,        # dominant frequency in band
    'peak_bpm': float,            # peak * 60 (breaths/min)
    'peak_power': float,          # spectral magnitude at peak
    'peak_snr': float,            # peak / median(band) - main signal
    'noise_floor': float,         # median power in band
    'spectrum': (n_freqs,) ndarray # for downstream visualization
    'freqs': (n_freqs,) ndarray
}
"""
```

```
per_link_samples = defaultdict(list)
for r in window_records:
    if r.get("type") != "iq":
        continue
    link = _link_key(r.get("node_id"), r.get("peer_id"))
    if link is None:
        continue
    csi = _parse_iq_complex(r)
    if csi.size == 0:
        continue
    t = float(r.get("t") if r.get("t") is not None else r.get("_t_rel", 0.0))
    per_link_samples[link].append((t, np.abs(csi).astype(np.float64)))

out = {}
for link, samples in per_link_samples.items():
    if len(samples) < min_packets:
        continue
    samples.sort(key=lambda s: s[0])
    ts = np.array([s[0] for s in samples])
    lens = [s[1].shape[0] for s in samples]
    common_len = int(np.median(lens))
    amps_list = [s[1][:common_len] for s in samples
                 if s[1].shape[0] >= common_len]
    if len(amps_list) < min_packets:
        continue
    amps = np.stack(amps_list, axis=0)
    n_pkts, n_sc = amps.shape
```

```
duration = ts[-1] - ts[0]
if duration < min_duration_s:
    continue
fs = (n_pkts - 1) / duration

# CARM-style SVD denoising: drop top-K temporal modes (drift)
# before any other processing. Operates on the (n_pkts, n_sc)
# log-amplitude matrix per link.
if svd_drift_modes > 0:
    amps = _svd_denoise_amps(amps, n_drift_modes=svd_drift_modes,
                             log_amplitude=True)

# Hampel + polynomial detrend (same as before)
med_t = np.median(amps, axis=0)
mad_t = np.median(np.abs(amps - med_t[None, :]), axis=0)
sigma_t = 1.4826 * mad_t
deviation = np.abs(amps - med_t[None, :])
outliers = (sigma_t[None, :] > 0) & (deviation > hampel_k * sigma_t[None, :])
if outliers.any():
    amps = np.where(outliers,
                    np.broadcast_to(med_t[None, :], amps.shape),
                    amps)
x = np.linspace(-1.0, 1.0, n_pkts).astype(np.float64)
basis = np.vstack([x ** k for k in range(detrend_order + 1)]).T
try:
    coeffs, *_ = np.linalg.lstsq(basis, amps, rcond=None)
    amps_dt = amps - basis @ coeffs
```

```
    except np.linalg.LinAlgError:
        amps_dt = amps - amps.mean(axis=0)[None, :]

    # Path 2: 2nd-order Butterworth high-pass at 0.10 Hz to kill drift
    # residue that polynomial detrending leaves behind. Implemented as
    # a difference-equation 2nd-order IIR (no scipy dependency).
    if highpass_cutoff_hz > 0 and fs > 2 * highpass_cutoff_hz:
        # Bilinear transform of 2nd-order Butterworth high-pass
        wn = highpass_cutoff_hz / (fs / 2)  # normalized cutoff
        wc = np.tan(np.pi * wn / 2)
        k_h = 1.0 / (1.0 + np.sqrt(2) * wc + wc * wc)
        b0_h = k_h
        b1_h = -2.0 * k_h
        b2_h = k_h
        a1_h = 2.0 * (wc * wc - 1.0) * k_h
        a2_h = (1.0 - np.sqrt(2) * wc + wc * wc) * k_h
        # Apply forward and backward (zero-phase) for clean response
        for axis_pass in range(2):
            buf = np.zeros((2, amps_dt.shape[1]), dtype=np.float64)
            filt_buf = np.zeros_like(amps_dt)
            src = amps_dt if axis_pass == 0 else amps_dt[::-1]
            for i in range(src.shape[0]):
                x_i = src[i]
                y_i = b0_h * x_i + buf[0]
                buf[0] = b1_h * x_i - a1_h * y_i + buf[1]
                buf[1] = b2_h * x_i - a2_h * y_i
                filt_buf[i] = y_i
```

```
        amps_dt = filt_buf if axis_pass == 0 else filt_buf[::-1]


    # Welch's method: split into K overlapping segments, FFT each,
    # average power spectra. K=n_welch_segments with 50% overlap.
    if n_welch_segments >= 2:
        seg_len = n_pkts // ((n_welch_segments + 1) // 2 + 1)
        # Use n_welch_segments segments of length seg_len with 50% overlap
        seg_len = max(seg_len, n_pkts // n_welch_segments)
        hop = seg_len // 2
        psds = []
        for i in range(n_welch_segments):
            start = i * hop
            end = start + seg_len
            if end > n_pkts:
                break
            seg = amps_dt[start:end]
            # Hanning window per segment
            win = np.hanning(seg.shape[0]).astype(np.float64).reshape(-1, 1)
            seg_w = seg * win
            # FFT and accumulate power; sum across subcarriers (coherent)
            spec = np.abs(np.fft.rfft(seg_w, axis=0)) ** 2
            psds.append(spec.sum(axis=1))  # (n_freqs,) sum across sc
        if psds:
            psd = np.mean(np.stack(psds, axis=0), axis=0)
            freqs = np.fft.rfftfreq(seg_len, 1.0 / fs)
        else:
            # Fallback: single FFT
```

```
        win = np.hanning(n_pkts).astype(np.float64).reshape(-1, 1)
        spec = np.abs(np.fft.rfft(amps_dt * win, axis=0)) ** 2
        psd = spec.sum(axis=1)
        freqs = np.fft.rfftfreq(n_pkts, 1.0 / fs)
    else:
        # Single FFT, no Welch
        win = np.hanning(n_pkts).astype(np.float64).reshape(-1, 1)
        spec = np.abs(np.fft.rfft(amps_dt * win, axis=0)) ** 2
        psd = spec.sum(axis=1)
        freqs = np.fft.rfftfreq(n_pkts, 1.0 / fs)

    # Find peak in band
    in_band = (freqs >= breathing_band[0]) & (freqs <= breathing_band[1])
    if in_band.any():
        band_freqs = freqs[in_band]
        band_psd = psd[in_band]
        peak_idx = int(np.argmax(band_psd))
        peak_freq = float(band_freqs[peak_idx])
        peak_power = float(band_psd[peak_idx])
        noise_floor = float(np.median(band_psd))
        if noise_floor > 0:
            peak_snr = peak_power / noise_floor
        else:
            peak_snr = 0.0
    else:
        peak_freq = 0.0
        peak_power = 0.0
```

```
        noise_floor = 0.0
        peak_snr = 0.0

    out[link] = {
        "peak_freq_hz": peak_freq,
        "peak_bpm": peak_freq * 60.0,
        "peak_power": peak_power,
        "peak_snr": peak_snr,
        "noise_floor": noise_floor,
        "spectrum": psd,
        "freqs": freqs,
        "n_packets": int(n_pkts),
    }
  return out


class DopplerAccumulator:
  """Multi-window coherent integration of per-link Doppler spectra.

  Maintains a rolling deque of the last K per-link power spectra.
  When asked for a snapshot, AVERAGES the spectra coherently
  (assumes peak frequencies stay constant across windows for
  stationary bodies).

  SNR gain: a persistent breathing peak adds linearly across K
  windows; uncorrelated noise averages by 1/sqrt(K). Net peak SNR
  improvement: sqrt(K).
```

```
Trade-off: response time = K * stride. For stationary subjects
held in one quadrant for 5+ minutes, K=6 gives a 2.4x SNR gain at
the cost of ~30 s of response latency.
"""

def __init__(self, max_windows=6):
    self.max_windows = max_windows
    # link -> deque of {'spectrum': ndarray, 'freqs': ndarray, 't': float}
    self._history = {}
    # peak frequency persistence: link -> deque of peak freq Hz
    self._peak_history = {}

def reset(self):
    """Drop all accumulated per-link spectra and peak history.

    After reset, the next K windows' integrated_features() will only
    contain post-reset evidence (where K = max_windows). This is the
    accumulator-level half of an operator soft-reset: the IQ buffer
    in live_iter holds the raw records; this object holds the
    derived multi-window-averaged spectra. Both must be dropped to
    prevent stale evidence from biasing the post-reset HMM.
    """
    self._history.clear()
    self._peak_history.clear()

def push(self, robust_features, t_wall, min_peak_snr_for_history=1.5):
```

```
    """Add the current window's robust features to the accumulator.

    Only append peak frequency to peak_history when the peak is
    meaningful (peak_snr above threshold). Otherwise push None as
    a sentinel so the persistence count doesn't include phantom
    peaks from quiet links (where np.argmax over band noise picks
    a stable but spurious bin).
    """
    for link, info in robust_features.items():
        if link not in self._history:
            self._history[link] = deque(maxlen=self.max_windows)
            self._peak_history[link] = deque(maxlen=self.max_windows)
        self._history[link].append({
            "spectrum": info["spectrum"].copy(),
            "freqs": info["freqs"].copy(),
            "t": t_wall,
            "n_packets": info.get("n_packets", 0),
        })
        peak_snr = info.get("peak_snr", 0.0)
        if peak_snr >= min_peak_snr_for_history:
            self._peak_history[link].append(info["peak_freq_hz"])
        else:
            self._peak_history[link].append(None)

def integrated_features(self, breathing_band=BREATHING_BAND_HZ,
                        freq_persistence_tolerance_hz=0.04,
                        min_persistence_count=2):
```

```
    """Compute integrated per-link features from the accumulated history.

    Returns dict[link] -> {
        'peak_freq_hz', 'peak_bpm', 'peak_power', 'peak_snr',
        'noise_floor', 'spectrum', 'freqs',
        'n_windows_integrated': int,
        'peak_persistence_count': int  # how many recent windows
                                       # had the same peak (within tol)
    }
    """
    out = {}
    for link, hist in self._history.items():
        if not hist:
            continue
        # Average spectra (assume same freq bins across windows;
        # they should be since fs and n_pkts are stable)
        base_freqs = hist[-1]["freqs"]
        specs = [h["spectrum"] for h in hist
                 if h["spectrum"].shape == base_freqs.shape]
        if not specs:
            continue
        avg_spec = np.mean(np.stack(specs, axis=0), axis=0)

        in_band = (base_freqs >= breathing_band[0]) & (base_freqs <= breathing_band[1])
        if not in_band.any():
            continue
        band_freqs = base_freqs[in_band]
```

```
        band_spec = avg_spec[in_band]
        peak_idx = int(np.argmax(band_spec))
        peak_freq = float(band_freqs[peak_idx])
        peak_power = float(band_spec[peak_idx])
        noise_floor = float(np.median(band_spec))
        peak_snr = peak_power / noise_floor if noise_floor > 0 else 0.0

        # Peak persistence count: how many recent windows had a
        # peak within the tolerance of this window's peak. Skip
        # None entries (windows where peak SNR was too low to
        # count as a real detection).
        persistence_count = 0
        recent_peaks = list(self._peak_history.get(link, []))
        for pf in recent_peaks:
            if pf is None:
                continue
            if abs(pf - peak_freq) <= freq_persistence_tolerance_hz:
                persistence_count += 1

        # Average n_packets across history entries (latest matters
        # most for filtering decisions, but the average reflects
        # sustained packet quality)
        n_pkts_avg = float(np.mean([h.get("n_packets", 0) for h in hist]))
        out[link] = {
            "peak_freq_hz": peak_freq,
            "peak_bpm": peak_freq * 60.0,
            "peak_power": peak_power,
```

```
            "peak_snr": peak_snr,
            "noise_floor": noise_floor,
            "spectrum": avg_spec,
            "freqs": base_freqs,
            "n_windows_integrated": len(specs),
            "peak_persistence_count": persistence_count,
            "n_packets_avg": n_pkts_avg,
        }
    return out


def detect_bodies_from_doppler(robust_features,
                               min_peak_snr=4.0,
                               freq_cluster_tolerance_hz=0.04,
                               min_persistence_count=4,
                               min_persistence_snr=1.5,
                               merge_shared_node=True,
                               single_body=False,
                               min_packets_per_link=60):
    """Cluster per-link breathing peaks into 'bodies' by frequency.

    Multiple bodies in a room produce multiple distinct breathing
    frequencies. Each body's frequency is seen on the links its
    multipath modulates. Cluster links by their peak frequency:
    same cluster = same body.

    Returns list of body dicts:
```

```
    [{
        'frequency_hz': float,          # cluster center
        'bpm': float,
        'links': [(a, b), ...],         # links seeing this body
        'mean_peak_snr': float,
        'dominant_node': int|None,      # node ID most touched
    }]

Sorted by mean_peak_snr descending (strongest body first).
"""
if not robust_features:
    return []
# Filter starved links: their sparse FFT can produce phantom-high
# peak SNR by chance, polluting body detection. Same logic as
# dominant_node_from_doppler / format_doppler_explanation.
if min_packets_per_link > 0:
    robust_features = {
        l: info for l, info in robust_features.items()
        if (info.get("n_packets_avg", info.get("n_packets", 0))
            >= min_packets_per_link)
    }
    if not robust_features:
        return []
# Each link has a peak frequency and SNR. A link counts as
# "showing breathing" if EITHER:
#   - peak_snr >= min_peak_snr (single-window strong signal), OR
#   - peak_persistence_count >= min_persistence_count AND
```

```
#     peak_snr >= min_persistence_snr (consistent weak signal across
#     multiple windows = real signal that just happens to be weak).
candidates = []
for link, info in robust_features.items():
    snr = info["peak_snr"]
    persist = info.get("peak_persistence_count", 0)
    if snr >= min_peak_snr:
        candidates.append((link, info["peak_freq_hz"], snr))
    elif persist >= min_persistence_count and snr >= min_persistence_snr:
        # Multi-window persistence: weak per-window but consistent.
        # Score as snr * sqrt(persist) for ranking.
        effective_snr = snr * (persist ** 0.5)
        candidates.append((link, info["peak_freq_hz"], effective_snr))
if not candidates:
    return []

# Greedy clustering by frequency proximity
candidates.sort(key=lambda x: x[1])  # sort by frequency
clusters = []
for link, freq, snr in candidates:
    placed = False
    for cluster in clusters:
        if abs(cluster["mean_freq"] - freq) <= freq_cluster_tolerance_hz:
            cluster["links"].append(link)
            cluster["snrs"].append(snr)
            cluster["freqs"].append(freq)
            cluster["mean_freq"] = float(np.mean(cluster["freqs"]))
```

```
            placed = True
            break
    if not placed:
        clusters.append({
            "mean_freq": freq,
            "links": [link],
            "snrs": [snr],
            "freqs": [freq],
        })

# For each cluster, compute dominant node by binary tally with
# SNR-weighted tie-break. Pair with temporal smoothing in the
# caller for stable display.
bodies = []
for c in clusters:
    counts = {1: 0, 2: 0, 3: 0, 4: 0}
    snr_sum = {1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
    for (a, b), snr in zip(c["links"], c["snrs"]):
        counts[a] += 1
        counts[b] += 1
        snr_sum[a] += snr
        snr_sum[b] += snr
    dominant_node = None
    # Sort by (count desc, snr_sum desc): ties broken by total SNR
    # contribution from that node's links
    nodes_ranked = sorted(counts.keys(),
                          key=lambda n: (-counts[n], -snr_sum[n]))
```

```
    best = nodes_ranked[0]
    if counts[best] >= 2:
        dominant_node = best
    bodies.append({
        "frequency_hz": c["mean_freq"],
        "bpm": c["mean_freq"] * 60.0,
        "links": c["links"],
        "mean_peak_snr": float(np.mean(c["snrs"])),
        "dominant_node": dominant_node,
    })
bodies.sort(key=lambda b: -b["mean_peak_snr"])

# Post-process: merge clusters that share at least one node. A
# single physical body always touches multiple links via its
# multipath, so two clusters sharing a node are almost certainly
# the same body whose peak frequency jittered across links.
if merge_shared_node and len(bodies) > 1:
    merged = []
    used = [False] * len(bodies)
    for i, b in enumerate(bodies):
        if used[i]:
            continue
        cur_links = set(b["links"])
        cur_nodes = {n for (a, c) in cur_links for n in (a, c)}
        cur = dict(b)
        for j in range(i + 1, len(bodies)):
            if used[j]:
```

```
            continue
        other = bodies[j]
        other_nodes = {n for (a, c) in other["links"] for n in (a, c)}
        if cur_nodes & other_nodes:
            # Merge: union links, weighted-average frequency by SNR
            w1 = cur["mean_peak_snr"]
            w2 = other["mean_peak_snr"]
            if w1 + w2 > 0:
                cur["frequency_hz"] = (
                    (cur["frequency_hz"] * w1
                     + other["frequency_hz"] * w2) / (w1 + w2)
                )
                cur["bpm"] = cur["frequency_hz"] * 60.0
            cur_links = cur_links | set(other["links"])
            cur["links"] = list(cur_links)
            cur_nodes |= other_nodes
            cur["mean_peak_snr"] = max(w1, w2)
            # Recompute dominant node from union
            nodes_list = []
            for (a, c) in cur_links:
                nodes_list.extend([a, c])
            counts = {n: nodes_list.count(n) for n in set(nodes_list)}
            best = max(counts, key=counts.get)
            cur["dominant_node"] = (best if counts[best] >= 2
                                    else None)
            used[j] = True
    merged.append(cur)
```

```
            used[i] = True
        bodies = merged

    if single_body and len(bodies) > 1:
        bodies = bodies[:1]  # already sorted by SNR descending
    return bodies



def format_robust_doppler(robust_features, bodies):
    """Format the robust Doppler analysis for live display.

    Shows per-body breakdown (separated by frequency clustering) so
    multi-body rooms produce multi-body output:

        [doppler-robust] body 1: 17 BPM (peak SNR 8.3) on L24, L34, L14 -> node 4 (Q4)
        [doppler-robust] body 2: 14 BPM (peak SNR 5.1) on L12, L23     -> node 2 (Q1)
        [doppler-robust] empty/quiet links: L13
    """
    if not bodies:
        return "[doppler-robust] no breathing peaks detected (room may be empty)"
    lines = []
    node_to_q = {1: "Q3 (SW)", 2: "Q1 (NW)", 3: "Q2 (NE)", 4: "Q4 (SE)"}
    for i, body in enumerate(bodies, 1):
        link_str = ", ".join(f"L{a}{b}" for (a, b) in body["links"])
        node_str = (f"-> node {body['dominant_node']} = "
                    f"{node_to_q.get(body['dominant_node'], '?')}"
                    if body['dominant_node'] is not None else "(no clear node)")
```

```
        lines.append(
            f"[doppler-robust] body {i}: {body['bpm']:.0f} BPM "
            f"(peak SNR {body['mean_peak_snr']:.1f}) on {link_str}  {node_str}"
        )
    # List quiet links (high peak SNR not detected)
    used_links = {l for body in bodies for l in body["links"]}
    quiet = [(a, b) for (a, b) in ALL_LINKS if (a, b) not in used_links]
    if quiet:
        quiet_str = ", ".join(f"L{a}{b}" for (a, b) in quiet)
        lines.append(f"[doppler-robust] quiet links (no breathing): {quiet_str}")
    return "\n".join(lines)



def dominant_node_from_doppler(explain, top_k=3,
                               adaptive_filter_pct=0.50,
                               min_packets_floor=70,
                               min_surviving_links=4):
    """Return the node ID (1..4) most represented in the top-K Doppler-
    active links, or None if conditions for a confident call aren't met.

    Binary top-K tally sorted by active_fraction, with two reliability
    gates that handle non-stationary SoftAP queue:

    GATE 1 — Adaptive starved-link filter:
        Compute median packet count across all 6 links. Drop any link
        below max(`min_packets_floor`, `adaptive_filter_pct` * median).
        Adapts to whatever the queue is doing — if all links are at
```

```
    150, filter cuts at 45; if degraded to 80, filter cuts at 24.


GATE 2 — Minimum surviving links:
    After filter, require at least `min_surviving_links` to remain.
    With 4 nodes and 6 links, at least 4 links are needed to
    triangulate a position confidently. Below that threshold,
    return None (caller should display ANALYZING — not enough
    data to call confidently).
"""
per_link = explain.get("per_link", {}) if explain else {}
if not per_link:
    return None
# GATE 1: adaptive starved-link filter
pkt_counts = [info.get("n_packets", 0) for info in per_link.values()]
if pkt_counts:
    median_pkts = sorted(pkt_counts)[len(pkt_counts) // 2]
    threshold = max(min_packets_floor,
                    adaptive_filter_pct * median_pkts)
    per_link = {l: info for l, info in per_link.items()
                if info.get("n_packets", 0) >= threshold}
# GATE 2: insufficient surviving links -> refuse to call
if len(per_link) < min_surviving_links:
    return None
links_sorted = sorted(per_link.items(),
                      key=lambda kv: -kv[1].get("active_fraction", 0))
top = links_sorted[:top_k]
if not top:
```

```
        return None
    # Tally nodes; break ties by total active_fraction across incident
    # links so a non-deterministic dict iteration order does not bias
    # toward low-numbered nodes.
    counts = {1: 0, 2: 0, 3: 0, 4: 0}
    activity_sum = {1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
    for (a, b), info in top:
        counts[a] += 1
        counts[b] += 1
        af = info.get("active_fraction", 0.0)
        activity_sum[a] += af
        activity_sum[b] += af
    # Sort by (count desc, activity_sum desc) -- ties broken by total
    # signal contribution from that node's links
    nodes_ranked = sorted(counts.keys(),
                          key=lambda n: (-counts[n], -activity_sum[n]))
    best = nodes_ranked[0]
    if counts[best] < 2:
        return None  # no clear dominance
    return best



_CLASS_TO_NODE = {"Q1": 2, "Q2": 3, "Q3": 1, "Q4": 4}



class DopplerPredictionSmoother:
    """Temporal majority-vote smoother for the [doppler] dominant-node
```

```
prediction.

Per-window calibrationless predictions inherently flicker because
Doppler-band activity on different links varies window to window
even for a stationary body. A running majority vote over the last
N predictions converts a noisy ~60% per-window predictor into a
stable ~95% time-averaged predictor.

Usage:
    sm = DopplerPredictionSmoother(window_size=30, min_confidence=0.50)
    for emission in detector_loop:
        raw = ... # dominant node from current window (1..4 or None)
        smoothed = sm.update(raw)  # returns (node|None, confidence)
        display(smoothed)
"""

def __init__(self, window_size=30, min_confidence=0.50):
    self.window_size = window_size
    self.min_confidence = min_confidence
    self._history = deque(maxlen=window_size)

def update(self, node):
    """Push current prediction (None or 1..4), return (node, confidence).

    Always append (even None), so the deque tracks total emissions.
    Confidence = best_count / total_calls — a stretch of None
    inputs decays confidence below threshold and the smoother
```

```
    emits None (ANALYZING) instead of stale stuck predictions.
    """
    self._history.append(node)
    if not self._history:
        return None, 0.0
    counts = {n: 0 for n in (1, 2, 3, 4)}
    n_valid = 0
    for v in self._history:
        if v is None:
            continue
        counts[v] = counts.get(v, 0) + 1
        n_valid += 1
    if n_valid == 0:
        return None, 0.0
    # Confidence denominator is total deque length (including
    # None placeholders) -- this decays stale predictions when
    # bodies leave the room.
    best = max(counts, key=counts.get)
    confidence = counts[best] / len(self._history)
    if confidence < self.min_confidence:
        return None, confidence
    return best, confidence

def reset(self):
    self._history.clear()
```

```
def format_doppler_explanation(explain,
                               adaptive_filter_pct=0.50,
                               min_packets_floor=70,
                               min_surviving_links=4,
                               use_continuous=False):
    """Format the doppler self-explanation for live display.

    Returns a multi-line string ready to print. Uses the same adaptive
    starved-link filter and minimum-surviving-links gate as
    dominant_node_from_doppler so the displayed top-3 and dominant-node
    line are consistent.
    """
    overall = explain.get("overall", {})
    per_link_full = explain.get("per_link", {})
    if not overall or not per_link_full:
        return "[doppler] insufficient data for breathing analysis"

    lines = []
    bpm = overall.get("peak_bpm", 0.0)
    in_range = overall.get("in_adult_breathing_range", False)
    range_note = "adult resting range" if in_range else "outside resting band"
    lines.append(
        f"[doppler] peak: {overall['peak_freq_hz']:.2f} Hz "
        f"({bpm:.0f} breaths/min, {range_note})"
    )
    lines.append(
        f"[doppler] active subcarriers: "
```

```
    f"{overall['total_active_subcarriers']} / "
    f"{overall['total_subcarriers']} "
    f"({overall['active_fraction']*100:.1f}%)"
)
# GATE 1: adaptive starved-link filter
pkt_counts = [info.get("n_packets", 0)
              for info in per_link_full.values()]
if pkt_counts:
    median_pkts = sorted(pkt_counts)[len(pkt_counts) // 2]
    threshold = max(min_packets_floor,
                    adaptive_filter_pct * median_pkts)
    per_link = {l: info for l, info in per_link_full.items()
                if info.get("n_packets", 0) >= threshold}
    n_excluded = len(per_link_full) - len(per_link)
    if n_excluded > 0:
        excluded = [f"L{a}{b}" for (a, b), info in per_link_full.items()
                    if info.get("n_packets", 0) < threshold]
        lines.append(
            f"[doppler] excluded {n_excluded} starved link(s): "
            f"{', '.join(excluded)} (median={median_pkts} pkts, "
            f"cutoff={threshold:.0f})"
        )
else:
    per_link = per_link_full
# GATE 2: minimum surviving links
if len(per_link) < min_surviving_links:
    lines.append(
```

```
        f"[doppler] only {len(per_link)} healthy link(s) -- need "
        f"{min_surviving_links}+ to triangulate. ANALYZING."
    )
    return "\n".join(lines)
if not per_link:
    lines.append("[doppler] all links starved - no dominant node")
    return "\n".join(lines)
# Top 3 links by active_fraction (matches dominant-node-from-doppler)
links_sorted = sorted(per_link.items(),
                      key=lambda kv: -kv[1]["active_fraction"])
top_links = links_sorted[:3]
if top_links:
    link_str = "  ".join(
        f"L{a}{b}: {info['active_fraction']*100:.0f}% "
        f"@ {info['peak_freq_hz']:.2f} Hz"
        for (a, b), info in top_links
    )
    lines.append(f"[doppler] strongest links: {link_str}")
    node_to_q = {1: "Q3 (SW)", 2: "Q1 (NW)",
                 3: "Q2 (NE)", 4: "Q4 (SE)"}
    if use_continuous:
        # Soft endpoint attribution over ALL surviving links (not
        # just top-3). Matches the live HMM observation source so
        # the displayed dominant-node line is consistent with the
        # PREDICTION headline.
        dom_new, energy, conf = continuous_node_estimator(
            per_link, reliability_exp=0.5, min_confidence=0.30,
```

```
        )
        if dom_new is not None:
            lines.append(
                f"[doppler] -> dominant node: {dom_new} = "
                f"{node_to_q.get(dom_new, '?')}  "
                f"(continuous, conf={conf*100:.0f}%)"
            )
    else:
        # Legacy binary top-K tally with active_fraction tiebreak.
        node_counts = {1: 0, 2: 0, 3: 0, 4: 0}
        node_activity = {1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0}
        for (a, b), info in top_links:
            node_counts[a] += 1
            node_counts[b] += 1
            af = info.get("active_fraction", 0.0)
            node_activity[a] += af
            node_activity[b] += af
        nodes_ranked = sorted(node_counts.keys(),
                              key=lambda n: (-node_counts[n],
                                             -node_activity[n]))
        node_dominant = nodes_ranked[0]
        if node_counts[node_dominant] >= 2:
            lines.append(
                f"[doppler] -> dominant node: {node_dominant} = "
                f"{node_to_q.get(node_dominant, '?')}"
            )
return "\n".join(lines)
```

```
# ---------------------------------------------------------------------------
# Output formatting
# ---------------------------------------------------------------------------
def format_prediction(predicted_class, deltas, ts_wall=None, n_records=0,
                      rule_index=None):
    if ts_wall is None:
        ts_wall = time.time()
    hms = time.strftime("%H:%M:%S", time.localtime(ts_wall))
    rule_str = f" rule={rule_index:>2}" if rule_index is not None else ""
    # Calibrationless / DOPPLER_ONLY mode: deltas is empty, every link
    # would print as NaN. Suppress the per-link line entirely.
    if not deltas:
        return (f"[{hms}] {predicted_class}{rule_str}  "
                f"n_records={n_records}")
    # Show per-link mean delta for a quick sanity read.
    per_link_mu = {l: float(deltas[l].mean()) if l in deltas else float("nan")
                   for l in ALL_LINKS}
    mu_str = "  ".join(
        f"L{a}{b}={per_link_mu[(a, b)]:+5.2f}" for (a, b) in ALL_LINKS
    )
    return (f"[{hms}] {predicted_class}{rule_str}  n_records={n_records}  "
            f"per-link mean-delta: {mu_str}")


# ---------------------------------------------------------------------------
```

```
# Background Doppler compute worker
# ---------------------------------------------------------------------------
class _DopplerComputeWorker(threading.Thread):
    """Run the heavy Doppler/SVD/body-detect path off the main UDP loop.

    Numpy FFT (np.fft.rfft) and SVD (np.linalg.svd) release the GIL for the
    duration of their underlying BLAS/LAPACK calls. So this worker truly runs
    in parallel with main-thread Python (UDP receive, packet parsing, buffer
    appends) on a multi-core machine. Without this thread the receive loop
    would block for ~150-300ms on every emission while Welch + SVD finishes,
    overflowing the OS UDP socket buffer at ~250 pkts/s and silently
    dropping packets.

    The worker owns the DopplerAccumulator (multi-window coherent
    integration) so its history is consistent across submissions.

    Backpressure: the in-queue is bounded to 2 entries. If main thread
    submits faster than worker drains, the OLDEST pending request is
    dropped -- we always want the freshest Doppler result, not a stale
    one. The result queue is unbounded; main thread polls it every
    emission, so it cannot back up significantly.
    """

    def __init__(self, accumulator_max_windows=6,
                 breathing_ratio_threshold=BREATHING_BAND_RATIO_THRESHOLD):
        super().__init__(daemon=True, name="doppler-compute")
        self._in_q = queue.Queue(maxsize=2)
```

```
    self._out_q = queue.Queue()
    self._stop_event = threading.Event()
    self._accum = DopplerAccumulator(max_windows=accumulator_max_windows)
    self._breathing_ratio_threshold = breathing_ratio_threshold

def submit(self, request):
    try:
        self._in_q.put_nowait(request)
    except queue.Full:
        try:
            self._in_q.get_nowait()
        except queue.Empty:
            pass
        try:
            self._in_q.put_nowait(request)
        except queue.Full:
            pass

def drain_results(self):
    results = []
    while True:
        try:
            results.append(self._out_q.get_nowait())
        except queue.Empty:
            break
    return results
```

```
def stop(self):
    self._stop_event.set()
    try:
        self._in_q.put_nowait(None)
    except queue.Full:
        pass

def reset(self):
    """Operator soft-reset: drop the multi-window accumulator and
    any pending in-flight requests/results.

    Called when the operator presses R between demo positions. Any
    request currently mid-compute will still finish and post a
    result; the caller must also re-clear `latest_dop_result` and
    ignore the next single drained result, otherwise stale features
    from before the reset can sneak through one window.
    """
    # Drain pending input requests so we don't process stale IQ.
    while True:
        try:
            self._in_q.get_nowait()
        except queue.Empty:
            break
    # Drain any results sitting in the output queue.
    while True:
        try:
            self._out_q.get_nowait()
```

```
        except queue.Empty:
            break
    # Drop the K-window coherent-integration history.
    self._accum.reset()

def run(self):
    while not self._stop_event.is_set():
        try:
            req = self._in_q.get(timeout=0.5)
        except queue.Empty:
            continue
        if req is None:
            break
        try:
            result = self._compute(req)
        except Exception as exc:
            result = {
                "request_id": req.get("request_id"),
                "t": req.get("t"),
                "error": str(exc),
            }
        try:
            self._out_q.put_nowait(result)
        except queue.Full:
            pass

def _compute(self, req):
```

```
    """Heavy compute path. NUMPY OPS RELEASE GIL — runs in parallel."""
    out = {
        "request_id": req["request_id"],
        "t": req["t"],
        "doppler_ready": False,
        "doppler": None,
        "robust_features": None,
        "bodies": None,
        "doppler_explain": None,
        "breathing_score": 0.0,
    }
    records = req.get("records") or []
    opts = req["opts"]
    if not records:
        return out
    dop_duration = (records[-1].get("_t_rel", 0.0)
                    - records[0].get("_t_rel", 0.0))
    if dop_duration < BREATHING_MIN_DURATION_S:
        return out
    out["doppler_ready"] = True

    # Compute primary Doppler (used for breathing_score and explain).
    doppler = per_link_doppler_power(
        records, min_duration_s=BREATHING_MIN_DURATION_S,
    )
    out["doppler"] = doppler
    if opts.get("breathing_presence", 0.0) > 0:
```

```
        out["breathing_score"] = breathing_presence_score(
            doppler,
            ratio_threshold=opts.get(
                "breathing_ratio_threshold",
                self._breathing_ratio_threshold,
            ),
        )
    if opts.get("explain_doppler"):
        out["doppler_explain"] = doppler_self_explain(
            records,
            ratio_threshold=opts.get(
                "breathing_ratio_threshold",
                self._breathing_ratio_threshold,
            ),
        )
    # Robust path: Welch + Butterworth + SVD denoise + accumulator
    if opts.get("robust_doppler") or opts.get("localize"):
        win_robust = per_link_doppler_robust(
            records,
            min_duration_s=BREATHING_MIN_DURATION_S,
            svd_drift_modes=opts.get("svd_drift_modes", 0),
        )
        self._accum.push(win_robust, req["t"])
        robust_features = self._accum.integrated_features()
        out["robust_features"] = robust_features
        if opts.get("robust_doppler"):
            out["bodies"] = detect_bodies_from_doppler(
```

```
                robust_features,
                min_peak_snr=opts.get("body_snr_threshold", 4.0),
                single_body=opts.get("single_body", False),
            )
    return out



# ---------------------------------------------------------------------------
# Live UDP driver
# ---------------------------------------------------------------------------
def live_iter(baseline,
            bind_host="0.0.0.0",
            bind_port=4211,
            window_s=WINDOW_SECONDS,
            stride_s=ADVANCE_SECONDS,
            breathing_window_s=BREATHING_WINDOW_SECONDS,
            stop_event=None,
            stats_interval_s=2.0,
            quiet=False,
            empty_threshold=0.0,
            baseline_alpha=0.0,
            debug_log_fp=None,
            breathing_presence=0.0,
            breathing_ratio_threshold=BREATHING_BAND_RATIO_THRESHOLD,
            explain_doppler=False,
            robust_doppler=False,
            localize=False,
```

```
        accumulator_max_windows=6,
        position_smooth_alpha=0.0,
        body_snr_threshold=2.5,
        svd_drift_modes=0,
        single_body=False,
        continuous_estimator=False,
        hmm_p_stay=0.95,
        hmm_min_confidence=0.50,
        hmm_adjacency_weight=4.0,
        position_max_step_m=1.5,
        starvation_warn_threshold=30,
        starvation_warning_interval_s=20.0,
        starvation_min_packets=70,
        starvation_pct_of_median=0.5,
        per_link_normalize=False,
        per_link_normalize_alpha=0.02,
        use_blockage_signal=False,
        blockage_weight=0.30,
        min_link_packets=0,
        pygame_display=None):
if stop_event is None:
    stop_event = threading.Event()
from udp_receiver import UDPReceiver
from frame_decoder import parse_packet

rx = UDPReceiver(bind_host=bind_host, bind_port=bind_port)
rx.open()
```

```
if not quiet:
    print(f"[live] bound UDP {bind_host}:{bind_port}, awaiting packets...",
          file=sys.stderr, flush=True)

buf = deque()            # short window (window_s) for variance + classify
doppler_buf = deque()     # long window (breathing_window_s) for FFT
first_t = None
next_emit_t = None
pkt_total = 0
type_counts = {}
iq_per_node = {1: 0, 2: 0, 3: 0, 4: 0}

# Track per-node uptime to detect resets in the live stream and
# exclude IQ records within +/- RESET_GUARD_SECONDS of any reset.
last_uptime = {}
reset_intervals = []  # list of (t_lo, t_hi) tuples in monotonic time
# Most-recent reboot info for the pygame "LAST RESET" panel.
# Populated from the telemetry frame whose uptime regressed.
last_reset_info = None  # {"node_id", "reason_code", "reason_label",
                        #  "crash_count", "t_wall", "t_mono"} or None
# Per-node crash counters (latest seen via telemetry, NOT just on
# the reset event). The display reads this as a forensic record.
per_node_crash_count = {1: 0, 2: 0, 3: 0, 4: 0}
# Rolling history of recent reboots for the LAST RESET panel's
# "Recent reboots" list. Kept short for live operator diagnosis;
# the JSONL debug log retains the full audit trail.
recent_resets = deque(maxlen=10)
```

```
# Per-link signal-score baselines for --per-link-normalize.
# EMA-updated each window; passed into continuous_node_estimator
# so each link contributes its DEVIATION from its own baseline,
# correcting for SoftAP TXQ bias that puts one diagonal at ~5x
# the activity of the other. Two separate baselines: one for the
# explain pipeline (active_fraction-driven), one for the robust
# pipeline (peak_snr-driven), since the scales differ.
per_link_baseline_explain: Dict[Tuple[int, int], float] = {}
per_link_baseline_robust: Dict[Tuple[int, int], float] = {}

t_wall_start = time.monotonic()
t_next_stats = t_wall_start + stats_interval_s

# The multi-window Doppler accumulator lives inside
# _DopplerComputeWorker so it is not accessed concurrently from
# the receive loop.
# Activate position smoother if EITHER alpha smoothing OR kinematic
# clamp is requested. With alpha=0 and max_step_m>0 we get pure
# velocity clipping (no temporal averaging). With alpha>0 and
# max_step_m=0 we get pure EMA (no clipping). Either suffices.
_pos_alpha = position_smooth_alpha if position_smooth_alpha > 0 else 1.0
_pos_max_step = (position_max_step_m
                 if position_max_step_m and position_max_step_m > 0
                 else None)
pos_smoother = (localizer_2d.PositionSmoother(
                  alpha=_pos_alpha,
                  max_step_m=_pos_max_step,
```

```
            )
            if (localize and (position_smooth_alpha > 0
                        or _pos_max_step is not None))
            else None)

# Background compute worker. Heavy Doppler / SVD / body-detect runs
# in this thread so the main UDP receive loop never blocks.
want_doppler_global = ((breathing_presence > 0)
                or explain_doppler or robust_doppler or localize
                or continuous_estimator)
if continuous_estimator:
    # Compute both the explain pipeline (active_fraction) and the
    # robust pipeline (peak_snr) so the continuous estimator has
    # both inputs to combine.
    explain_doppler = True
    robust_doppler = True
worker = None
if want_doppler_global:
    worker = _DopplerComputeWorker(
        accumulator_max_windows=accumulator_max_windows,
        breathing_ratio_threshold=breathing_ratio_threshold,
    )
    worker.start()
# Latest computed-by-worker fields. Reused across emissions until
# a fresh result arrives. None until first compute completes.
latest_dop_result = None
next_request_id = 0
```

```
# Temporal majority-vote smoother for calibrationless dominant-node
# predictions. window=9 / min_confidence=0.50 balances emit rate
# against per-window noise: short enough to track demo movement,
# long enough to reject single-window flips.
#
# Per-link starvation streak tracker. Whenever a link is excluded
# by the adaptive starvation filter (n_pkts < max(70, 0.5*median))
# for many consecutive windows, the SoftAP queue is biased away
# from that link. Empirically this manifests as steady predictions
# that drift when the queue finally rotates: the rotation flips
# the link's fingerprint in a way that fools the localizer/HMM.
# Recommended action: restart coord to reset the queue.
link_starve_streak = {f"{a}-{b}": 0 for (a, b) in ALL_LINKS}
last_starve_warning_t = 0.0

# Operator reset signal: when set by the stdin watcher thread, the
# main loop resets the HMM posterior to uniform AND clears the
# position smoother. Use this between demo positions ('r' + Enter
# at the keyboard) so the smoother doesn't anchor on the last
# quadrant when the body has just been moved.
reset_request = threading.Event()

def _stdin_reset_watcher():
    """Watch stdin for an 'r' keypress; signal a reset.

    Reads line by line so it works on Windows + Unix without
```

```
    platform-specific raw-mode tty handling. User types 'r' (or
    any line starting with r) + Enter to fire a reset.
    """
    try:
        for line in sys.stdin:
            if line.strip().lower().startswith("r"):
                reset_request.set()
    except Exception:
        pass

# Daemon so the watcher dies with the process; doesn't block exit.
threading.Thread(target=_stdin_reset_watcher, daemon=True).start()
if not quiet:
    print("[live] type 'r' + Enter at any time to reset the HMM "
          "posterior + position smoother (use between demo moves)",
          file=sys.stderr, flush=True)

if continuous_estimator:
    # Forward-filter HMM smoother. Operates on continuous per-window
    # node energy vectors, integrating low-confidence observations
    # gracefully.
    dop_smoother = HmmNodeSmoother(
        p_stay=hmm_p_stay,
        min_confidence=hmm_min_confidence,
        adjacency_weight=hmm_adjacency_weight,
    )
    if not quiet:
```

```
        print(f"[live] continuous-estimator MODE: HMM smoother "
              f"(p_stay={hmm_p_stay}, min_confidence={hmm_min_confidence}, "
              f"adjacency_weight={hmm_adjacency_weight})",
              file=sys.stderr, flush=True)
else:
    dop_smoother = DopplerPredictionSmoother(window_size=9,
                                             min_confidence=0.50)

try:
    for payload in rx.read_packets():
        if stop_event.is_set():
            break
        rec = parse_packet(payload)
        pkt_total += 1
        tp = rec.get("type", "unknown")
        type_counts[tp] = type_counts.get(tp, 0) + 1

        if tp == "telemetry":
            # Detect reboots via uptime regression.
            nid = rec.get("node_id")
            upt = rec.get("uptime_s")
            _crash = rec.get("crash_count")
            if isinstance(nid, int) and isinstance(_crash, int):
                if 1 <= nid <= 4:
                    per_node_crash_count[nid] = _crash
            if isinstance(nid, int) and isinstance(upt, int):
                if nid in last_uptime and upt < last_uptime[nid]:
```

```
            now = time.monotonic()
            reset_intervals.append((now - RESET_GUARD_SECONDS,
                                    now + RESET_GUARD_SECONDS))
            # Capture the most-recent reset for the live
            # pygame "LAST RESET" panel. The reason_code
            # comes from esp_reset_reason() in the
            # telemetry frame whose uptime regressed.
            rcode = rec.get("reset_reason")
            last_reset_info = {
                "node_id": nid,
                "reason_code": rcode,
                "reason_label": reset_reason_label(rcode),
                "crash_count": rec.get("crash_count"),
                "t_wall": time.time(),
                "t_mono": now,
            }
            recent_resets.append(dict(last_reset_info))
            if debug_log_fp is not None:
                debug_log_fp.write(json.dumps({
                    "type": "reset",
                    "t_wall": time.time(),
                    "node_id": nid,
                    "reset_reason": rcode,
                    "reset_reason_label": reset_reason_label(rcode),
                    "crash_count": rec.get("crash_count"),
                }) + "\n")
                debug_log_fp.flush()
```

```
            last_uptime[nid] = upt

        if tp == "iq":
            nid = rec.get("node_id")
            if isinstance(nid, int) and nid in iq_per_node:
                iq_per_node[nid] += 1

        now = time.monotonic()
        if now >= t_next_stats:
            if not quiet:
                elapsed = now - t_wall_start
                # Show Doppler warm-up progress if breathing gate
                # is enabled and not yet warmed up.
                dop_status = ""
                if breathing_presence > 0 and len(doppler_buf) > 0:
                    dop_dur = (float(doppler_buf[-1].get("_t_rel", 0.0))
                               - float(doppler_buf[0].get("_t_rel", 0.0)))
                    if dop_dur < BREATHING_MIN_DURATION_S:
                        dop_status = (f" WARMUP {dop_dur:4.1f}/"
                                      f"{BREATHING_MIN_DURATION_S:.0f}s")
                    else:
                        dop_status = " READY"
                print(f"\r[live] {elapsed:5.1f}s  pkts={pkt_total:6d} "
                      f"iq={type_counts.get('iq',0):5d} "
                      f"[n1={iq_per_node[1]:4d} n2={iq_per_node[2]:4d} "
                      f"n3={iq_per_node[3]:4d} n4={iq_per_node[4]:4d}] "
                      f"buf={len(buf)} dop_buf={len(doppler_buf)}{dop_status} "
```

```
            f"resets={len(reset_intervals)}",
            file=sys.stderr, flush=True, end="")
    t_next_stats = now + stats_interval_s

if tp != "iq":
    continue

t = time.monotonic()
# Skip records inside reboot guard windows
in_reset = False
for lo, hi in reset_intervals[-5:]:  # only check recent intervals
    if lo <= t <= hi:
        in_reset = True
        break
if in_reset:
    continue

rec["_t_rel"] = t
buf.append(rec)
doppler_buf.append(rec)
if first_t is None:
    first_t = t
    next_emit_t = first_t + window_s
# Trim short buffer to window_s of trailing data.
bound_short = t - window_s
while buf and float(buf[0].get("_t_rel", 0.0)) < bound_short:
    buf.popleft()
```

```
# Trim long Doppler buffer to breathing_window_s of trailing data.
bound_long = t - breathing_window_s
while doppler_buf and float(doppler_buf[0].get("_t_rel", 0.0)) < bound_long:
    doppler_buf.popleft()
if next_emit_t is None or t < next_emit_t:
    continue
if not buf:
    next_emit_t = t + stride_s
    continue
cur_amps = per_link_subcar_amps(list(buf))
deltas = {}
for link, base_v in baseline.items():
    if link in cur_amps and cur_amps[link].shape == base_v.shape:
        deltas[link] = cur_amps[link] - base_v
# Calibrationless mode (no baseline) -- still emit so Doppler
# results reach the user. We just skip v2 / variance-gate /
# template-amp_delta paths that need baseline-relative deltas.
no_baseline_mode = (not baseline)
if deltas or no_baseline_mode:
    # Empty detection by ACROSS-SUBCARRIER variance per link:
    # for each link, std of the 192-element delta vector
    # measures frequency-selective fading. Empty room has
    # uniform per-subcarrier delta (drift is gain-shift =
    # additive constant, leaves std unchanged). A body
    # creates frequency-selective fading: some subcarriers
    # drop, some rise -> std rises sharply. Take max across
    # links so any single link with body-induced subcarrier
```

```
# structure trips the gate. No history needed.
per_link_subcar_stds = {link: float(d.std())
                        for link, d in deltas.items()}
if per_link_subcar_stds and empty_threshold > 0:
    max_subcar_std = max(per_link_subcar_stds.values())
    is_empty_variance = max_subcar_std < empty_threshold
else:
    is_empty_variance = False
    max_subcar_std = 0.0

# Doppler-band presence detection on the LONG buffer.
# Independent of variance gate. Computes per-(link, sc)
# breathing-band power on a 90 s rolling buffer for
# sharp FFT resolution (~0.011 Hz bins). Warm-up: while
# doppler_buf has < BREATHING_MIN_DURATION_S of data,
# don't score (treat as "not enough data").
breathing_score = 0.0
doppler_ready = False
doppler = None
robust_features = None
bodies = None
doppler_explanation = None
# Submit a fresh compute request to the worker (snapshot
# the doppler buffer) and pull whatever results have
# finished computing since last emission. This NEVER
# blocks: receive continues during the worker's heavy
# FFT/SVD operations on another thread.
```

```
        if worker is not None and len(doppler_buf) > 0:
            next_request_id += 1
            worker.submit({
                "request_id": next_request_id,
                "t": t,
                "records": list(doppler_buf),
                "opts": {
                    "explain_doppler": explain_doppler,
                    "robust_doppler": robust_doppler,
                    "localize": localize,
                    "breathing_presence": breathing_presence,
                    "breathing_ratio_threshold": breathing_ratio_threshold,
                    "svd_drift_modes": svd_drift_modes,
                    "body_snr_threshold": body_snr_threshold,
                    "single_body": single_body,
                },
            })
            for result in worker.drain_results():
                if "error" not in result:
                    latest_dop_result = result
        if latest_dop_result is not None:
            doppler_ready = latest_dop_result.get(
                "doppler_ready", False
            )
            doppler = latest_dop_result.get("doppler")
            breathing_score = latest_dop_result.get(
                "breathing_score", 0.0
```

```
        )
        robust_features = latest_dop_result.get("robust_features")
        bodies = latest_dop_result.get("bodies")
        doppler_explanation = latest_dop_result.get(
            "doppler_explain"
        )
    if breathing_presence > 0 and doppler_ready:
        is_empty_breathing = breathing_score < breathing_presence
    else:
        # Doppler disabled or not warmed up yet -> don't gate
        is_empty_breathing = True

    # Combined empty decision: the room is empty if all
    # ENABLED+READY gates agree. Doppler must be warmed up
    # (>= BREATHING_MIN_DURATION_S) to count.
    breathing_active = (breathing_presence > 0) and doppler_ready
    variance_active = (empty_threshold > 0)
    if breathing_active and variance_active:
        is_empty = is_empty_variance and is_empty_breathing
    elif breathing_active:
        is_empty = is_empty_breathing
    elif variance_active:
        is_empty = is_empty_variance
    else:
        is_empty = False

    # Rolling baseline: update only when room is empty (no
```

```
# body-induced subcarrier structure detected). Drift will
# accumulate uniformly so EMA tracks it; body presence
# holds baseline fixed.
if baseline_alpha > 0 and is_empty:
    for link, cur_v in cur_amps.items():
        if link in baseline and cur_v.shape == baseline[link].shape:
            baseline[link] = ((1.0 - baseline_alpha) * baseline[link]
                              + baseline_alpha * cur_v)

# If breathing gate was requested but Doppler isn't yet
# warmed up, suppress the emission entirely. The user
# asked for "wait until full Doppler is ready before
# making any guess", so we don't yield a class until the
# 90s buffer has at least BREATHING_MIN_DURATION_S of
# data. The variance gate alone could call EMPTY/quad
# but the user's design requires Doppler corroboration.
if breathing_presence > 0 and not doppler_ready:
    if debug_log_fp is not None:
        debug_log_fp.write(json.dumps({
            "type": "warmup",
            "t_wall": time.time(),
            "n_records_doppler_buf": len(doppler_buf),
            "doppler_buf_seconds": round(
                float(doppler_buf[-1].get("_t_rel", 0.0))
                - float(doppler_buf[0].get("_t_rel", 0.0)) if doppler_buf else 0.0,
                2,
            ),
```

```
            "warmup_target_s": BREATHING_MIN_DURATION_S,
        }) + "\n")
        debug_log_fp.flush()
    next_emit_t = t + stride_s
    continue

# Calibrationless Doppler-only pipeline: the headline
# prediction comes from the continuous estimator + HMM
# smoother computed below. The short-window class field
# is a placeholder for compatibility with the emission
# schema.
pred, rule_idx = "DOPPLER_ONLY", 0
if is_empty:
    pred = "EMPTY"
# Operator reset signal: clear HMM posterior + position
# smoother + starvation streaks. Fires once per
# 'r' + Enter (stdin) OR 'r' keypress in pygame window.
if pygame_display is not None and pygame_display.reset_event.is_set():
    pygame_display.reset_event.clear()
    reset_request.set()
if pygame_display is not None and pygame_display.stop_event.is_set():
    stop_event.set()
if reset_request.is_set():
    reset_request.clear()
    # Smoother + downstream state.
    if hasattr(dop_smoother, "reset"):
        dop_smoother.reset()
```

```
        if pos_smoother is not None:
            pos_smoother.reset()
        for link in link_starve_streak:
            link_starve_streak[link] = 0
        last_starve_warning_t = 0.0
        # Upstream IQ state: without this, the HMM is
        # reset to uniform but the next window's per-link
        # signal scores are still computed from a
        # doppler_buf that is up to ~90 s of evidence from
        # the OLD position. The HMM converges right back.
        # Clearing the buffers and the multi-window
        # accumulator forces post-reset evidence only.
        buf.clear()
        doppler_buf.clear()
        if worker is not None:
            worker.reset()
        latest_dop_result = None
        print(">>> RESET: HMM, position smoother, "
              "starvation streaks, IQ buffers, and "
              "Doppler accumulator cleared. Display will "
              "show ANALYZING for ~30-90s while the new "
              "breathing window fills.", flush=True)

    # Compute per-link AND per-node record counts over the
    # short window (window_s seconds). Per-node count = how
    # many IQ records the receiver-side decoder saw with
    # that node_id over the window, regardless of which
```

```
# peer the record came from. This is the same data the
# operator console prints as `[n1=.. n2=.. n3=.. n4=..]`
# and is what feeds the pygame RX-rate panel.
per_link_counts = {}
per_node_counts = {1: 0, 2: 0, 3: 0, 4: 0}
for r in buf:
    nid, pid = r.get("node_id"), r.get("peer_id")
    if isinstance(nid, int) and 1 <= nid <= 4:
        per_node_counts[nid] += 1
    if isinstance(nid, int) and isinstance(pid, int) and nid != pid:
        a, b = sorted((nid, pid))
        if 1 <= a and b <= 4:
            k = f"{a}-{b}"
            per_link_counts[k] = per_link_counts.get(k, 0) + 1

# Update per-link starvation streaks. We use the LONG
# (breathing) window's per-link packet count from the
# doppler_explanation -- same source as
# dominant_node_from_doppler / format_doppler_explanation
# so the streak is consistent with the live "[doppler]
# excluded ... starved link(s)" message. The SHORT 6s
# window count (per_link_counts above) is too small to
# apply a 70-pkt absolute floor to.
# A link is starved if its packet count is below
# max(starvation_min_packets, starvation_pct_of_median *
# median). Sustained starvation = SoftAP queue is biased.
_explain_pl = (doppler_explanation.get("per_link") or {}
```

```
                if doppler_explanation else {})
        if _explain_pl:
            pkt_vals = [info.get("n_packets", 0)
                        for info in _explain_pl.values()]
            median_pkts = (sorted(pkt_vals)[len(pkt_vals) // 2]
                           if pkt_vals else 0)
            starve_cutoff = max(starvation_min_packets,
                                starvation_pct_of_median * median_pkts)
            for link in link_starve_streak:
                a, b = link.split("-")
                info = _explain_pl.get((int(a), int(b)))
                if info is None:
                    continue
                n = info.get("n_packets", 0)
                if n < starve_cutoff:
                    link_starve_streak[link] += 1
                else:
                    link_starve_streak[link] = 0
        elif per_link_counts:
            # Fallback (no doppler_explanation yet): mark
            # nothing as starved during warm-up so the warning
            # banner doesn't fire spuriously.
            pass
            # Emit recommendation when any streak crosses the
            # threshold. Rate-limit to once per
            # starvation_warning_interval_s seconds so we don't
            # spam stderr.
```

```
        max_streak_link = max(link_starve_streak,
                              key=link_starve_streak.get)
        max_streak = link_starve_streak[max_streak_link]
        now_t = time.time()
        if (max_streak >= starvation_warn_threshold
            and (now_t - last_starve_warning_t)
                >= starvation_warning_interval_s):
            last_starve_warning_t = now_t
            warning_line = (
                f">>> RECOMMENDATION: restart coord. "
                f"L{max_streak_link.replace('-', '')} starved "
                f"for {max_streak} consecutive windows "
                f"(~{max_streak * stride_s:.0f}s). "
                f"SoftAP queue likely biased; predictions "
                f"will drift soon."
            )
            if not quiet:
                print(warning_line, file=sys.stderr,
                      flush=True)
            # Also persist to stdout so it appears in the
            # demo viewer log alongside other lines.
            print(warning_line, flush=True)
    # These are populated whether or not debug_log_fp is
    # set, because the live_iter yield exposes them to the
    # udp dispatch (for ASCII headline + pygame display).
    dop_pred_node = None
    dop_pred_class = None
```

```
        dop_peak_hz = None
        dop_active_frac = None
        smoothed_node = None
        smoothed_class = None
        smoothed_confidence = 0.0
        per_link_active_fraction = None
        per_link_peak_snr = None
        obs_node_energy = None
        if True:
            # Per-link active_fraction / peak_snr captured here
            # so the debug log can be replayed through the
            # continuous estimator without raw IQ.
            per_link_active_fraction = None
            per_link_peak_snr = None
            obs_node_energy = None
            if doppler_explanation is not None:
                overall = doppler_explanation.get("overall", {}) or {}
                dop_peak_hz = overall.get("peak_freq_hz")
                dop_active_frac = overall.get("active_fraction")
                explain_pl = doppler_explanation.get("per_link") or {}
                if explain_pl:
                    per_link_active_fraction = {
                        f"{a}-{b}": round(
                            info.get("active_fraction", 0.0), 4)
                        for (a, b), info in explain_pl.items()
                    }
                if continuous_estimator:
```

```
# Continuous attribution over the explain
# pipeline (active_fraction-weighted endpoints).
_norm_explain = (per_link_baseline_explain
                if per_link_normalize else None)
dom_new, energy_p1, _ = continuous_node_estimator(
    explain_pl, reliability_exp=0.5,
    min_confidence=0.30,
    per_link_baseline=_norm_explain,
    min_link_packets=min_link_packets,
)
if dom_new is not None:
    dop_pred_node = dom_new
    dop_pred_class = {1: "Q3", 2: "Q1",
                      3: "Q2", 4: "Q4"}.get(
                          dom_new)
obs_node_energy = energy_p1
# EMA update for next window's normalization.
# Update AFTER the call so the current window
# is normalized against past windows only.
if per_link_normalize and explain_pl:
    from live.continuous_estimator import (
        per_link_signal_score
    )
    _a = per_link_normalize_alpha
    for _link, _info in explain_pl.items():
        _score = per_link_signal_score(_info)
        _prev = per_link_baseline_explain.get(
```

```
                _link, _score
            )
            per_link_baseline_explain[_link] = (
                (1.0 - _a) * _prev + _a * _score
            )
    else:
        # Binary top-K tally branch (used when
        # --continuous-estimator is off).
        dom = dominant_node_from_doppler(
            doppler_explanation
        )
        if dom is not None:
            dop_pred_node = dom
            dop_pred_class = {1: "Q3", 2: "Q1",
                              3: "Q2", 4: "Q4"}.get(dom)
# Capture per-link peak_snr from robust_features for
# debug log enrichment and continuous attribution.
if robust_features:
    per_link_peak_snr = {
        f"{a}-{b}": round(info.get("peak_snr", 0.0), 3)
        for (a, b), info in robust_features.items()
    }
if continuous_estimator and robust_features:
    # Continuous attribution over robust_features
    # (peak_snr-weighted endpoints).
    _norm_robust = (per_link_baseline_robust
                    if per_link_normalize else None)
```

```
_, energy_p2, _ = continuous_node_estimator(
    robust_features, reliability_exp=0.5,
    min_confidence=0.30,
    per_link_baseline=_norm_robust,
    min_link_packets=min_link_packets,
)
if obs_node_energy is None:
    obs_node_energy = energy_p2
else:
    for k, v in energy_p2.items():
        obs_node_energy[k] = (
            obs_node_energy.get(k, 0.0) + v
        )
# EMA update for next window.
if per_link_normalize and robust_features:
    from live.continuous_estimator import (
        per_link_signal_score
    )
    _a = per_link_normalize_alpha
    for _link, _info in robust_features.items():
        _score = per_link_signal_score(_info)
        _prev = per_link_baseline_robust.get(
            _link, _score
        )
        per_link_baseline_robust[_link] = (
            (1.0 - _a) * _prev + _a * _score
        )
```

```
        if continuous_estimator and use_blockage_signal:
            # Differential-starvation attribution: bodies in
            # a link's Fresnel zone drop that link's packet
            # rate while peers stay healthy. Endpoints of
            # starved links get a positive contribution
            # proportional to the starvation depth. Blended
            # additively with the breathing-signal energy
            # via blockage_weight (default 0.30 -> roughly
            # 1:3 relative to a fully-active breathing link).
            _explain_pl = (doppler_explanation.get("per_link")
                           or {} if doppler_explanation else {})
            if _explain_pl:
                _link_pkts = {
                    lk: int(info.get("n_packets", 0) or 0)
                    for lk, info in _explain_pl.items()
                }
                _block_e = blockage_attribution(_link_pkts)
                if obs_node_energy is None:
                    obs_node_energy = {n: 0.0
                                       for n in (1, 2, 3, 4)}
                for k, v in _block_e.items():
                    obs_node_energy[k] = (
                        obs_node_energy.get(k, 0.0)
                        + blockage_weight * v
                    )
        if continuous_estimator:
            # HMM observation: combined explain-pipeline +
```

```
        # robust-pipeline node_energy. Pass None when
        # both are empty so the smoother applies the
        # transition prior only.
        if (obs_node_energy is None
                or sum(obs_node_energy.values()) <= 0):
            obs_node_energy = None
        sm_node, smoothed_confidence = dop_smoother.update(
            obs_node_energy
        )
    else:
        # Robust-pipeline body detection drives the
        # headline prediction in the legacy branch.
        body_dom_node = None
        if bodies:
            body_dom_node = bodies[0].get("dominant_node")
        sm_node, smoothed_confidence = dop_smoother.update(
            body_dom_node
        )
    if sm_node is not None:
        smoothed_node = sm_node
        smoothed_class = {1: "Q3", 2: "Q1",
                          3: "Q2", 4: "Q4"}.get(
            sm_node
        )
    # Per-body bodies count + dominant nodes
    bodies_summary = None
    if bodies:
```

```
    bodies_summary = [
        {
            "bpm": round(b.get("bpm", 0.0), 1),
            "snr": round(b.get("mean_peak_snr", 0.0), 2),
            "dominant_node": b.get("dominant_node"),
            "n_links": len(b.get("links", [])),
        } for b in bodies
    ]
if debug_log_fp is None:
    pass  # skip debug log write
else:
  debug_log_fp.write(json.dumps({
    "type": "window",
    "t_wall": time.time(),
    "class": pred,
    "rule_index": rule_idx,
    "is_empty": is_empty,
    "max_subcar_std": round(max_subcar_std, 4),
    "breathing_score": round(breathing_score, 4),
    "doppler_ready": doppler_ready,
    "doppler_pred_node": dop_pred_node,
    "doppler_pred_class": dop_pred_class,
    "doppler_smoothed_node": smoothed_node,
    "doppler_smoothed_class": smoothed_class,
    "doppler_smoothed_confidence": round(
        smoothed_confidence, 3
    ),
```

```
            "doppler_peak_bpm": (round(dop_peak_hz * 60.0, 1)
                                 if dop_peak_hz is not None
                                 else None),
            "n_bodies": len(bodies) if bodies else 0,
            "doppler_peak_hz": (round(dop_peak_hz, 3)
                                if dop_peak_hz is not None
                                else None),
            "doppler_active_fraction": (
                round(dop_active_frac, 4)
                if dop_active_frac is not None else None
            ),
            "bodies": bodies_summary,
            "n_records_buf": len(buf),
            "n_records_doppler_buf": len(doppler_buf),
            "per_link_record_counts": per_link_counts,
            "per_link_subcar_std": {f"{a}-{b}": round(per_link_subcar_stds.get((a, b), 0.0),
4)
                                    for (a, b) in ALL_LINKS},
            "per_link_mean_delta": {f"{a}-{b}": round(float(deltas[(a, b)].mean()), 4)
                                    if (a, b) in deltas else None
                                    for (a, b) in ALL_LINKS},
            "per_link_active_fraction": per_link_active_fraction,
            "per_link_peak_snr": per_link_peak_snr,
            "per_link_starve_streak": dict(link_starve_streak),
            "n_recent_resets": len(reset_intervals),
            "estimator_mode": ("continuous"
                               if continuous_estimator
```

```
                        else "binary_tally"),
    }) + "\n")
      debug_log_fp.flush()
# doppler_explanation came from the worker (above).
# 2D wall-pair localization. Operates on whatever data
# is available -- delta-std alone is enough for motion;
# Doppler SNR adds discrimination for stationary bodies.
position = None
per_body_positions = None
if localize:
    position = localizer_2d.estimate_position(
        deltas,
        robust_doppler=robust_features,
    )
    if position is not None and pos_smoother is not None:
        px, py, pinfo = position
        sx, sy = pos_smoother.update(px, py)
        position = (sx, sy,
                    {**pinfo, "raw_x": px, "raw_y": py,
                     "ema_n": pos_smoother.n})
    if bodies is not None and robust_features is not None:
        per_body_positions = (
            localizer_2d.estimate_position_per_body(
                bodies, robust_features, deltas=deltas,
            )
        )
yield {"class": pred, "rule_index": rule_idx,
```

```
            "deltas": deltas, "t_wall": time.time(),
            "n_records": len(buf), "is_empty": is_empty,
            "max_subcar_std": max_subcar_std,
            "doppler_explain": doppler_explanation,
            "robust_features": robust_features,
            "bodies": bodies,
            "position": position,
            "per_body_positions": per_body_positions,
            "window_records": list(buf),
            "doppler_smoothed_class": smoothed_class,
            "doppler_smoothed_confidence": smoothed_confidence,
            "doppler_pred_class": dop_pred_class,
            "doppler_peak_bpm": (
                round(dop_peak_hz * 60.0, 1)
                if dop_peak_hz is not None else None
            ),
            "doppler_active_fraction": dop_active_frac,
            "doppler_ready": doppler_ready,
            "n_bodies": len(bodies) if bodies else 0,
            "per_link_record_counts": per_link_counts,
            "per_node_record_counts": dict(per_node_counts),
            "window_seconds": window_s,
            "per_link_starve_streak": dict(link_starve_streak),
            "last_reset_info": (dict(last_reset_info)
                                if last_reset_info else None),
            "per_node_crash_count": dict(per_node_crash_count),
            "n_total_resets": len(reset_intervals),
```

```
                "recent_resets": list(recent_resets)}
        next_emit_t = t + stride_s
    finally:
        if worker is not None:
            worker.stop()
        rx.close()
        if not quiet:
            print("", file=sys.stderr, flush=True)



# ---------------------------------------------------------------------------
# CLI
# ---------------------------------------------------------------------------
def main(argv=None):
    parser = argparse.ArgumentParser(
        description="Live WiFi-CSI quadrant detector. "
                    "Continuous-attribution estimator + HMM smoother over the "
                    "4-state quadrant space.")
    sub = parser.add_subparsers(dest="cmd", required=True)

    p_udp = sub.add_parser("udp",
                           help="Live UDP from coord, stream predictions.")
    p_udp.add_argument("--baseline", type=Path, default=None,
                       help="Path to empty-room baseline JSON. Required for "
                            "v2 rule output, variance gate, and amp_delta-"
                            "based template matching. Omit for calibrationless "
                            "Doppler+SVD mode (no baseline needed -- the SVD "
```

```
                "denoise + breathing-band FFT works directly on "
                "raw amplitudes).")
p_udp.add_argument("--bind-host", default="0.0.0.0")
p_udp.add_argument("--bind-port", type=int, default=4211)
p_udp.add_argument("--window-seconds", type=float, default=WINDOW_SECONDS)
p_udp.add_argument("--stride-seconds", type=float, default=ADVANCE_SECONDS)
p_udp.add_argument("--smooth", type=int, default=5,
            help="Majority-vote over last N predictions (default 5).")
p_udp.add_argument("--quiet", action="store_true",
            help="Suppress the [live] packet counter on stderr.")
p_udp.add_argument("--empty-threshold", type=float, default=0.0,
            help="ACROSS-SUBCARRIER variance threshold: in "
                "each window, compute std across the 192-element "
                "per-(link, subcarrier) delta vector for each "
                "link, take max across links. If max < this, "
                "output EMPTY. Empty rooms have uniform deltas "
                "(low std). Body creates frequency-selective "
                "fading (high std). Drift is uniform = doesn't "
                "affect std. Try 1.5-2.5 to start.")
p_udp.add_argument("--baseline-alpha", type=float, default=0.0,
            help="Rolling-baseline EMA factor (0=fixed baseline). "
                "When --empty-threshold > 0 and the current window "
                "is detected as empty, the baseline updates by "
                "alpha * current + (1-alpha) * baseline, tracking "
                "mesh/room drift over time. Try 0.02 for a 3-hour "
                "session (~50-window time constant ~50s settling).")
p_udp.add_argument("--debug-log", type=Path, default=None,
```

```
                help="Write per-window diagnostics + reset events as "
                    "JSONL to this path. Each line is one event with "
                    "n_records, per-link counts, per-link subcarrier "
                    "std, mean-delta, and prediction. Reset events "
                    "include node_id and reason. Useful for offline "
                    "investigation of packet drops and mesh stability.")
p_udp.add_argument("--breathing-presence", type=float, default=0.0,
                help="Doppler-based presence detection: fraction of "
                    "subcarriers showing breathing-band power above "
                    "noise floor that constitutes 'occupied'. Try "
                    "0.05 (=5%% of subcarriers). 0=disable.")
p_udp.add_argument("--breathing-ratio-threshold", type=float,
                default=BREATHING_BAND_RATIO_THRESHOLD,
                help=f"Per-(link,sc) ratio threshold for considering "
                    f"a subcarrier 'breathing-active' (fraction of "
                    f"spectral power in 0.15-0.40 Hz band). Default "
                    f"{BREATHING_BAND_RATIO_THRESHOLD}.")
p_udp.add_argument("--breathing-window-sec", type=float,
                default=BREATHING_WINDOW_SECONDS,
                help=f"Window for the Doppler/breathing FFT in "
                    f"seconds. Default {BREATHING_WINDOW_SECONDS} "
                    f"(FFT resolution 1/window, so 90 s -> 0.011 "
                    f"Hz bins, ~24 bins in the breathing band, "
                    f"sharp peak-vs-noise discrimination). The "
                    f"variance gate / classifier still uses the "
                    f"short --window-seconds. Doppler scoring is "
                    f"skipped until at least "
```

```
                f"{BREATHING_MIN_DURATION_S:.0f}s of data is "
                f"buffered (warm-up).")
p_udp.add_argument("--explain-doppler", action="store_true",
            help="Print self-explanatory Doppler analysis after "
                "each prediction: peak breathing frequency, BPM, "
                "active subcarriers, top breathing-active links. "
                "Surfaces the physics, not just the verdict.")
p_udp.add_argument("--robust-doppler", action="store_true",
            help="Use Welch + Butterworth high-pass + 6-window "
                "coherent integration Doppler features and emit "
                "per-body breathing detections (frequency-"
                "clustered links). Multi-body capable.")
p_udp.add_argument("--localize", action="store_true",
            help="Compute continuous (x, y) position via wall-"
                "pair perturbation ratios. Emits a 2D estimate "
                "alongside the v2 quadrant verdict.")
p_udp.add_argument("--ascii-map", action="store_true",
            help="Render a small ASCII room map after each "
                "prediction. Requires --localize.")
p_udp.add_argument("--position-smooth-alpha", type=float, default=0.15,
            help="EMA factor for the (x, y) position estimate. "
                "Stationary bodies should give stable position; "
                "per-window noise needs smoothing. Try 0.1-0.2 "
                "(corresponds to ~5-10 window time constant). "
                "0=disable.")
p_udp.add_argument("--svd-drift-modes", type=int, default=0,
            help="Apply CARM-style SVD drift removal in the live "
```

```
                "Doppler pipeline. K=1 drops dominant temporal "
                "mode (drift). Calibrationless: no baseline or "
                "template needed -- pure physics (SVD-cleaned "
                "amplitudes -> Doppler -> dominant breathing-"
                "active node = quadrant).")
p_udp.add_argument("--single-body", action="store_true",
            help="Suppress all but the highest-SNR breathing "
                "body in the doppler-robust output. For "
                "single-occupant demos where a single body's "
                "breathing peak can fragment across links due "
                "to FFT jitter or harmonics, looking like 2+ "
                "bodies. Always combined with shared-node "
                "merge.")
p_udp.add_argument("--continuous-estimator", action="store_true",
            default=True,
            help="(default ON) Continuous-attribution "
                "estimator (soft endpoint attribution + "
                "sqrt(N) reliability weight) plus the HMM "
                "forward-filter smoother. This is the only "
                "supported estimator post the rule-classifier "
                "cleanup; the flag is kept for backwards "
                "compatibility.")
p_udp.add_argument("--no-continuous-estimator",
            dest="continuous_estimator",
            action="store_false",
            help="Disable the continuous estimator. Without "
                "it the headline will stay 'warming up' "
```

```
                "indefinitely unless a body is detected by "
                "the legacy body-finder pipeline.")
p_udp.add_argument("--hmm-p-stay", type=float, default=0.95,
            help="HMM transition self-probability. Higher = "
                "smoother, more confident, slower to switch. "
                "0.95 -> ~20-window smoothing horizon. Try "
                "0.90 for quicker response, 0.98 for very "
                "stable predictions.")
p_udp.add_argument("--hmm-min-confidence", type=float, default=0.50,
            help="HMM posterior confidence threshold below "
                "which the smoother emits None (ANALYZING). "
                "0.50 = top quadrant has at least 50%% of "
                "the posterior mass.")
p_udp.add_argument("--hmm-adjacency-weight", type=float, default=4.0,
            help="Kinematic constraint on HMM transitions. "
                "1.0 = uniform (any non-self state equally "
                "likely). >1.0 = wall-adjacent jumps "
                "preferred over diagonal-opposite jumps. "
                "4.0 (default) makes Q1->Q3 (adjacent) 4x "
                "more likely per window than Q1->Q4 "
                "(diagonal). Suppresses single-window "
                "noise-driven leaps across the room.")
p_udp.add_argument("--position-max-step-m", type=float, default=1.5,
            help="Kinematic clamp on (x,y) position updates. "
                "Raw localizer outputs farther than this "
                "from the previous smoothed position get "
                "pulled to the boundary along the line of "
```

```
                "travel. 1.5 m matches a brisk walk at "
                "1.5 m/s with 1s windows. 0 = disable.")
p_udp.add_argument("--starvation-warn-threshold", type=int, default=30,
            help="Number of consecutive windows a link can "
                "be excluded as 'starved' before the system "
                "recommends restarting the coord. With "
                "stride=2s, 30 windows = ~60s sustained "
                "starvation.")
p_udp.add_argument("--starvation-warning-interval-s", type=float,
            default=20.0,
            help="Minimum seconds between repeated coord-"
                "restart recommendations. Default 20s -- "
                "the warning persists, just doesn't spam.")
p_udp.add_argument("--pygame", action="store_true",
            help="Open the pygame demo display. Renders the "
                "4-quadrant room map, body asterisk + "
                "confidence circle, per-link breathing "
                "bars, and operator banners. R = reset HMM, "
                "Q/ESC = quit.")
p_udp.add_argument("--pygame-fullscreen", action="store_true",
            help="Open the pygame display in fullscreen.")
p_udp.add_argument("--hide-body-marker", action="store_true",
            help="Hide the (x, y) body asterisk + confidence "
                "glow circle on the pygame display. The "
                "quadrant headline and pulse remain. Useful "
                "when the (x, y) layer is noisy and you "
                "only want to demo the quadrant prediction.")
```

```
p_udp.add_argument("--per-link-normalize", action="store_true",
                help="Normalize each link's signal_score by its "
                    "rolling EMA baseline before soft endpoint "
                    "attribution. Each link contributes its "
                    "deviation from its own baseline rather than "
                    "its absolute strength. Use this when one "
                    "diagonal link is structurally stronger than "
                    "the other (e.g. SoftAP TXQ asymmetry biasing "
                    "predictions to one pair of quadrants).")
p_udp.add_argument("--per-link-normalize-alpha", type=float, default=0.02,
                help="EMA alpha for the per-link baseline. Smaller "
                    "= longer memory (~1/alpha windows). Default "
                    "0.02 -> ~50-window soft horizon (~50 s at "
                    "1 s stride). Only used when "
                    "--per-link-normalize is set.")
p_udp.add_argument("--use-blockage-signal", action="store_true",
                help="Use differential per-link starvation as a "
                    "positive position signal. A body in a link's "
                    "Fresnel zone drops that link's packet rate "
                    "while peers stay healthy; endpoints of "
                    "starved links get bonus node_energy. "
                    "Useful when breathing-band Doppler is weak "
                    "but the body is clearly blocking one link.")
p_udp.add_argument("--blockage-weight", type=float, default=0.30,
                help="Weight of the blockage signal relative to "
                    "the breathing-band energy. 0 disables "
                    "blockage; 0.3 (default) is moderate; 1.0 "
```

```
                    "makes blockage as strong as a fully-active "
                    "breathing link. Only used when "
                    "--use-blockage-signal is set.")
p_udp.add_argument("--min-link-packets", type=int, default=0,
                help="Drop links with fewer than this many "
                    "packets in the long (90 s) breathing "
                    "window from soft attribution entirely. "
                    "0 (default) = no filter. Use ~150 to "
                    "suppress chronically weak links whose "
                    "small-N denominator saturates "
                    "active_fraction with noise.")

args = parser.parse_args(argv)

if args.cmd == "udp":
    baseline = {}
    print(f"udp source: {args.bind_host}:{args.bind_port}", flush=True)
    print(f"waiting for first {args.window_seconds:.0f} s of packets "
          f"before first emission...", flush=True)
    if args.breathing_presence > 0:
        print(f"doppler/breathing gate enabled: window="
              f"{args.breathing_window_sec:.0f}s, "
              f"warm-up={BREATHING_MIN_DURATION_S:.0f}s "
              f"(score=0 until warm-up complete)", flush=True)
    stop = threading.Event()
    smooth_buf = deque(maxlen=max(1, args.smooth))
    debug_log_fp = None
```

```
if args.debug_log is not None:
    debug_log_fp = open(args.debug_log, "w", encoding="utf-8")
    print(f"writing debug log to {args.debug_log}", flush=True)

# Optional pygame demo display
pygame_disp = None
if args.pygame:
    try:
        from live.pygame_display import PygameDisplay
        pygame_disp = PygameDisplay(
            fullscreen=args.pygame_fullscreen,
            show_body_marker=not args.hide_body_marker,
        )
        pygame_disp.start()
        print(f"[live] pygame display started "
              f"(fullscreen={args.pygame_fullscreen}). "
              f"Press R in the pygame window to reset HMM, "
              f"Q/ESC to quit.", flush=True)
    except Exception as e:
        print(f"[live] WARNING: pygame display failed to start: "
              f"{e}. Continuing in ASCII-only mode.",
              file=sys.stderr, flush=True)
        pygame_disp = None
try:
    for emission in live_iter(
        baseline,
        bind_host=args.bind_host,
```

```
        bind_port=args.bind_port,
        window_s=args.window_seconds,
        stride_s=args.stride_seconds,
        breathing_window_s=args.breathing_window_sec,
        stop_event=stop,
        quiet=args.quiet,
        empty_threshold=args.empty_threshold,
        baseline_alpha=args.baseline_alpha,
        debug_log_fp=debug_log_fp,
        breathing_presence=args.breathing_presence,
        breathing_ratio_threshold=args.breathing_ratio_threshold,
        explain_doppler=args.explain_doppler,
        robust_doppler=args.robust_doppler,
        localize=args.localize or args.ascii_map,
        position_smooth_alpha=args.position_smooth_alpha,
        svd_drift_modes=args.svd_drift_modes,
        single_body=args.single_body,
        continuous_estimator=args.continuous_estimator,
        hmm_p_stay=args.hmm_p_stay,
        hmm_min_confidence=args.hmm_min_confidence,
        hmm_adjacency_weight=args.hmm_adjacency_weight,
        position_max_step_m=args.position_max_step_m,
        starvation_warn_threshold=args.starvation_warn_threshold,
        starvation_warning_interval_s=(
            args.starvation_warning_interval_s
        ),
        per_link_normalize=args.per_link_normalize,
```

```
        per_link_normalize_alpha=args.per_link_normalize_alpha,
        use_blockage_signal=args.use_blockage_signal,
        blockage_weight=args.blockage_weight,
        min_link_packets=args.min_link_packets,
        pygame_display=pygame_disp,
    ):
        smooth_buf.append(emission["class"])
        counts = {}
        for c in smooth_buf:
            counts[c] = counts.get(c, 0) + 1
        smoothed = max(counts, key=counts.get)
        print(format_prediction(smoothed,
                                emission["deltas"],
                                ts_wall=emission["t_wall"],
                                n_records=emission["n_records"],
                                rule_index=emission.get("rule_index")),
              flush=True)
        # If --explain-doppler is enabled, print the physics
        # summary after each prediction.
        # === HEADLINE: smoothed quadrant prediction ===
        # The displayed verdict consumed by the live console
        # output and the pygame visualizer.
        sm_class = emission.get("doppler_smoothed_class")
        sm_conf = emission.get("doppler_smoothed_confidence", 0.0)
        ts_str = time.strftime("%H:%M:%S")
        if sm_class:
            headline = (f"=== [{ts_str}] PREDICTION: {sm_class} "
```

```
                f"({sm_conf*100:.0f}% confidence) ===")
        elif sm_conf > 0:
            headline = (f"=== [{ts_str}] PREDICTION: ANALYZING "
                f"({sm_conf*100:.0f}% conf, below 50% "
                f"threshold) ===")
        else:
            headline = f"=== [{ts_str}] PREDICTION: warming up ==="
        print(headline, flush=True)
        # Push everything we just computed to the pygame display
        if pygame_disp is not None:
            _emis_pos = emission.get("position")
            _pos_xy = (_emis_pos[0], _emis_pos[1]) \
                if _emis_pos else None
            _exp = emission.get("doppler_explain") or {}
            _per_link_af = None
            if _exp.get("per_link"):
                _per_link_af = {
                    f"{a}-{b}": info.get("active_fraction", 0.0)
                    for (a, b), info in _exp["per_link"].items()
                }
            _starv = emission.get("per_link_starve_streak")
            _pkts = emission.get("per_link_record_counts")
            _bpm = emission.get("doppler_peak_bpm")
            _af_overall = emission.get("doppler_active_fraction")
            _starv_warn = False
            if _starv:
                _starv_warn = max(_starv.values()) >= 30
```

```
            pygame_disp.update(
                smoothed_class=sm_class,
                confidence=sm_conf,
                raw_class=emission.get("doppler_pred_class"),
                bpm=_bpm,
                active_fraction=_af_overall,
                per_link_active_fraction=_per_link_af,
                per_link_record_counts=_pkts,
                per_link_starve_streak=_starv,
                position_xy=_pos_xy,
                n_bodies=emission.get("n_bodies", 0),
                doppler_ready=emission.get("doppler_ready", False),
                starvation_warning_active=_starv_warn,
                n_records_short=emission.get("n_records"),
                per_node_record_counts=emission.get(
                    "per_node_record_counts"
                ),
                window_seconds=emission.get("window_seconds"),
                last_reset_info=emission.get("last_reset_info"),
                per_node_crash_count=emission.get(
                    "per_node_crash_count"
                ),
                n_total_resets=emission.get("n_total_resets"),
                recent_resets=emission.get("recent_resets"),
            )
            if pygame_disp.stop_event.is_set():
                stop.set()
```

```
            break
        exp = emission.get("doppler_explain")
        if exp:
            print(format_doppler_explanation(
                exp,
                use_continuous=args.continuous_estimator,
            ), flush=True)
        bodies = emission.get("bodies")
        robust_features = emission.get("robust_features")
        if bodies is not None and robust_features is not None:
            print(format_robust_doppler(robust_features, bodies),
                  flush=True)
        position = emission.get("position")
        if position is not None:
            px, py, pinfo = position
            print(localizer_2d.format_position(px, py, pinfo),
                  flush=True)
            if args.ascii_map:
                print(localizer_2d.render_ascii_room(px, py),
                      flush=True)
        per_body = emission.get("per_body_positions")
        if per_body:
            print(localizer_2d.format_per_body_positions(per_body),
                  flush=True)
            if args.ascii_map:
                print(
                    localizer_2d.render_ascii_room_multi(per_body),
```

```
                    flush=True,
                )
        except KeyboardInterrupt:
            print("\nshutdown requested", flush=True)
            stop.set()
        finally:
            if debug_log_fp is not None:
                debug_log_fp.close()
            if pygame_disp is not None:
                pygame_disp.stop()
        return 0

    return 1



if __name__ == "__main__":
    sys.exit(main())
```

```
"""2D wall-pair localizer.

Maps per-link perturbation strengths into a continuous (x, y) position
estimate in room coordinates, by ratioing perturbation on opposing
wall-edge links.

This is NOT triangulation in the time-of-flight sense -- our 40 MHz
HT40 bandwidth gives only c/(2*BW) = 3.75 m range resolution, useless
in a 9 m room. Instead, it is constraint-based: the 6 links between
```

```
4 corner nodes form 3 natural axis pairs:

    West edge L12 (n1=SW <-> n2=NW)  vs East edge L34 (n3=NE <-> n4=SE)
    South edge L14 (n1=SW <-> n4=SE) vs North edge L23 (n2=NW <-> n3=NE)
    Diagonal L13 (SW<->NE)           vs Diagonal L24 (NW<->SE)

The body lies somewhere on/near each link's line segment. A body close
to the west wall perturbs L12 strongly and L34 weakly -> the W/E
perturbation ratio gives x-coordinate. Same for y via S/N pair.

Diagonals are redundant with the wall pairs but useful for cross-
validation (hallway-adjacent links L24 and L34 may show inflated
perturbation from outside traffic; compare to L13 which is interior).

Per-link perturbation = per-link delta-vector std (frequency-selective
fading magnitude) + Doppler-band peak SNR (breathing modulation).
Either signal alone localizes; combining gives robustness when one
modality is weak (stationary body -> mostly Doppler; rapid motion ->
mostly std).

Geometry:
    Room: 3.81 m wide (W-E)  x  9.14 m long (S-N)
    Node 1 = Q3 = SW = (0.00, 0.00)
    Node 2 = Q1 = NW = (0.00, 9.14)
    Node 3 = Q2 = NE = (3.81, 9.14)
    Node 4 = Q4 = SE = (3.81, 0.00)
"""
```

```
from __future__ import annotations

from typing import Dict, Optional, Tuple

import numpy as np

ROOM_W = 3.81
ROOM_L = 9.14

NODE_POS = {
    1: (0.00, 0.00),
    2: (0.00, ROOM_L),
    3: (ROOM_W, ROOM_L),
    4: (ROOM_W, 0.00),
}

ALL_LINKS = ((1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4))

WEST_LINK = (1, 2)
EAST_LINK = (3, 4)
SOUTH_LINK = (1, 4)
NORTH_LINK = (2, 3)
DIAG_SW_NE = (1, 3)
DIAG_NW_SE = (2, 4)

NODE_TO_CLASS = {1: "Q3", 2: "Q1", 3: "Q2", 4: "Q4"}
```

```
def _link_perturbation(deltas, robust_doppler, link, doppler_weight=0.5):
    """Combined std + Doppler-SNR perturbation for a single link.

    std-of-delta captures frequency-selective fading from any motion or
    blockage. Doppler peak-SNR captures breathing-band coherent power
    (works even when delta is zero because baseline is stale).

    The (snr - 1) clamp at zero acts as a noise-floor subtraction:
    peak_snr <= 1 means peak is at or below the median in-band power
    (no real peak), so contributes nothing.
    """
    s = 0.0
    if deltas is not None and link in deltas:
        s += float(deltas[link].std())
    if robust_doppler is not None and link in robust_doppler:
        snr = float(robust_doppler[link].get("peak_snr", 0.0))
        s += doppler_weight * max(0.0, snr - 1.0)
    return s



def compute_link_perturbations(deltas, robust_doppler=None,
                               doppler_weight=0.5):
    """Compute per-link perturbation strength dict."""
    return {link: _link_perturbation(deltas, robust_doppler, link,
                                     doppler_weight=doppler_weight)
            for link in ALL_LINKS}
```

```
def estimate_position(deltas, robust_doppler=None,
                      alpha=1.5,
                      doppler_weight=0.5,
                      min_total_perturbation=0.05):
    """Wall-pair 2D position estimate.

    For each axis (x: W-E, y: S-N) compute a sharpened ratio of
    perturbation on the two opposing wall links:

        x = ROOM_W * s_E^alpha / (s_W^alpha + s_E^alpha)
        y = ROOM_L * s_N^alpha / (s_S^alpha + s_N^alpha)

    `alpha` > 1 sharpens the response (closer to step function),
    alpha = 1 is pure linear ratio. alpha = 1.5 is a good compromise
    for the demo: smooth movement, but corners go to corners.

    `min_total_perturbation` is a noise gate: if both wall sums are
    near zero, return None (no body detected, position undefined).

    Returns (x, y, info) tuple or None.
    """
    perts = compute_link_perturbations(deltas, robust_doppler,
                                       doppler_weight=doppler_weight)

    s_w = perts[WEST_LINK]
```

```
s_e = perts[EAST_LINK]
s_s = perts[SOUTH_LINK]
s_n = perts[NORTH_LINK]

if (s_w + s_e) < min_total_perturbation and \
   (s_s + s_n) < min_total_perturbation:
    return None

if (s_w + s_e) > 0:
    wa = s_w ** alpha
    ea = s_e ** alpha
    x = ROOM_W * ea / (wa + ea)
else:
    x = ROOM_W / 2.0

if (s_s + s_n) > 0:
    sa = s_s ** alpha
    na = s_n ** alpha
    y = ROOM_L * na / (sa + na)
else:
    y = ROOM_L / 2.0

diag_sw_ne = perts[DIAG_SW_NE]
diag_nw_se = perts[DIAG_NW_SE]
info = {
    "wall_perturbations": {
        "west_L12": s_w, "east_L34": s_e,
```

```
        "south_L14": s_s, "north_L23": s_n,
    },
    "diagonal_perturbations": {
        "L13_SW_NE": diag_sw_ne, "L24_NW_SE": diag_nw_se,
    },
    "link_perturbations": perts,
    "alpha": alpha,
  }
  return (x, y, info)



class PositionSmoother:
  """Exponential moving average for (x, y) position estimates.

  For STATIONARY bodies the per-window estimate is noisy because
  per-link std-of-delta has window-to-window variance even though
  the underlying body location is fixed. EMA averages out that
  variance with a configurable response time.

  alpha=0.1 -> ~20-window time constant (~20s at stride=1s, ~3 min
  at stride=10s). Tune for the demo: stationary subject -> low alpha
  (smoother). Moving subject -> higher alpha (faster response).
  """

  def __init__(self, alpha=0.15, max_step_m=None, stride_s=1.0):
    """
    Args:
```

```
        alpha: EMA blending factor (0=no update, 1=ignore history).
        max_step_m: kinematic constraint. If set, the per-update
            position can only move at most `max_step_m` meters from
            the previous estimate. Raw localizer outputs farther
            than that get pulled to the boundary along the line of
            travel. A person walking briskly at ~1.5 m/s with a
            stride_s=1s window can't move more than ~1.5 m per
            update; suppresses single-window jumps caused by
            upstream signal flips (SoftAP queue rotation, link
            starvation oscillation).
        stride_s: not used for EMA itself; recorded so callers can
            interpret the velocity bound (max_step_m / stride_s).
    """
    self.alpha = alpha
    self.max_step_m = max_step_m
    self.stride_s = stride_s
    self.x = None
    self.y = None
    self.n = 0

def update(self, x, y):
    # Kinematic clamp BEFORE EMA: the raw input represents what the
    # localizer thinks the body is right now. If it's >max_step_m
    # from our last estimate, treat the input as if the body moved
    # exactly max_step_m in the requested direction. EMA still
    # applies after clamping so the estimate is dampened.
    if self.x is not None and self.max_step_m is not None:
```

```
            dx = x - self.x
            dy = y - self.y
            d = (dx * dx + dy * dy) ** 0.5
            if d > self.max_step_m and d > 0:
                scale = self.max_step_m / d
                x = self.x + dx * scale
                y = self.y + dy * scale
        if self.x is None:
            self.x = x
            self.y = y
        else:
            self.x = (1 - self.alpha) * self.x + self.alpha * x
            self.y = (1 - self.alpha) * self.y + self.alpha * y
        self.n += 1
        return (self.x, self.y)

    def reset(self):
        self.x = None
        self.y = None
        self.n = 0



def estimate_position_per_body(bodies, robust_doppler,
                               deltas=None,
                               doppler_weight=1.0,
                               alpha=1.5):
    """One (x, y) per detected body.
```

```
Each body has a frequency cluster and a list of links that show
breathing at that frequency. We localize EACH body by computing
wall-pair perturbation using ONLY the body's link list. Other
links (assigned to different bodies, or quiet) contribute zero.

This is the "differential body localization" idea: in multi-body
rooms, the wall-pair ratio over ALL links averages every body's
contribution. By restricting to one body's link cluster, we
isolate that body's perturbation.

Args:
    bodies: list of body dicts from detect_bodies_from_doppler.
    robust_doppler: dict[link] -> {peak_snr, peak_freq_hz, ...}.
    deltas: optional per-link delta vectors. If provided, std is
            added to the per-link perturbation, but only on
            body-cluster links.
    doppler_weight: weight on Doppler peak-snr vs delta-std.
                    Higher means trust Doppler more.

Returns list of dicts:
    [{
        'body_index': 0,
        'frequency_hz': 0.25,
        'bpm': 15.0,
        'x': 1.5, 'y': 6.2,
        'links': [(1,2), (2,3)],
```

```
        'mean_peak_snr': 5.2,
    }, ...]
"""
out = []
for i, body in enumerate(bodies):
    body_links = set(body["links"])
    # Build per-link perturbation, zero outside the body's links
    per_link_pert = {}
    for link in ALL_LINKS:
        if link not in body_links:
            per_link_pert[link] = 0.0
            continue
        s = 0.0
        if deltas is not None and link in deltas:
            s += float(deltas[link].std())
        if robust_doppler is not None and link in robust_doppler:
            snr = float(robust_doppler[link].get("peak_snr", 0.0))
            s += doppler_weight * max(0.0, snr - 1.0)
        per_link_pert[link] = s

    s_w = per_link_pert[WEST_LINK]
    s_e = per_link_pert[EAST_LINK]
    s_s = per_link_pert[SOUTH_LINK]
    s_n = per_link_pert[NORTH_LINK]
    diag_sw_ne = per_link_pert[DIAG_SW_NE]
    diag_nw_se = per_link_pert[DIAG_NW_SE]
```

```
# If a wall pair is empty (no body link on either side), fall
# back to using the diagonals to fill in. Diagonal contains
# both x and y info: L13 (SW-NE) bias means body is in either
# SW or NE; L24 (NW-SE) bias means body is in either NW or SE.
# Combined with whichever wall pair is non-zero, this gives
# a unique quadrant.
if (s_w + s_e) <= 0 and (diag_sw_ne + diag_nw_se) > 0:
    # Diagonal-only x: L13 high -> body on diagonal SW-NE
    # (could be x=0 or x=ROOM_W). L24 high -> NW-SE diagonal.
    # We use which y is suggested (below) to disambiguate.
    s_w_eff = diag_sw_ne if s_n > s_s else 0.0  # SW or NE
    s_e_eff = diag_nw_se if s_n > s_s else 0.0  # NW or SE
    # If unclear, distribute evenly
    s_w = s_w_eff if (s_w_eff + s_e_eff) > 0 else diag_nw_se
    s_e = s_e_eff if (s_w_eff + s_e_eff) > 0 else diag_sw_ne
if (s_s + s_n) <= 0 and (diag_sw_ne + diag_nw_se) > 0:
    s_s_eff = diag_nw_se if s_e > s_w else diag_sw_ne
    s_n_eff = diag_sw_ne if s_e > s_w else diag_nw_se
    s_s = s_s_eff
    s_n = s_n_eff

if (s_w + s_e) > 0:
    wa = s_w ** alpha
    ea = s_e ** alpha
    x = ROOM_W * ea / (wa + ea)
else:
    x = ROOM_W / 2.0
```

```
        if (s_s + s_n) > 0:
            sa = s_s ** alpha
            na = s_n ** alpha
            y = ROOM_L * na / (sa + na)
        else:
            y = ROOM_L / 2.0

        out.append({
            "body_index": i,
            "frequency_hz": body["frequency_hz"],
            "bpm": body["bpm"],
            "x": x, "y": y,
            "links": body["links"],
            "mean_peak_snr": body["mean_peak_snr"],
            "dominant_node": body.get("dominant_node"),
        })
    return out



def format_per_body_positions(per_body):
    """Multi-line formatter for per-body localization output."""
    if not per_body:
        return "[per-body] no bodies detected"
    lines = []
    for body in per_body:
        link_str = ", ".join(f"L{a}{b}" for (a, b) in body["links"])
        q = quadrant_for_position(body["x"], body["y"])
```

```
        lines.append(
            f"[per-body] body {body['body_index']+1}: "
            f"{body['bpm']:.0f} BPM "
            f"(SNR {body['mean_peak_snr']:.1f}) "
            f"-> ({body['x']:.2f}m, {body['y']:.2f}m) {q}  "
            f"on {link_str}"
        )
    return "\n".join(lines)



def render_ascii_room_multi(per_body, width=21, height=15):
    """ASCII room with one digit per body (1, 2, ...) at its (x, y)."""
    grid = [[" " for _ in range(width)] for _ in range(height)]

    def to_grid(rx, ry):
        gx = int(round(rx / ROOM_W * (width - 1)))
        gy = int(round((1.0 - ry / ROOM_L) * (height - 1)))
        gx = max(0, min(width - 1, gx))
        gy = max(0, min(height - 1, gy))
        return gx, gy

    for n, (px, py) in NODE_POS.items():
        gx, gy = to_grid(px, py)
        grid[gy][gx] = str(n)

    for i, body in enumerate(per_body):
        gx, gy = to_grid(body["x"], body["y"])
```

```
        marker = chr(ord("a") + i) if i < 26 else "*"
        if grid[gy][gx] == " ":
            grid[gy][gx] = marker


    border = "+" + "-" * width + "+"
    rows = [border + "  N (top)"]
    for r in grid:
        rows.append("|" + "".join(r) + "|")
    rows.append(border)
    rows.append(f"  W <-- {ROOM_W:.2f}m x {ROOM_L:.2f}m --> E   "
                f"({len(per_body)} bodies)")
    return "\n".join(rows)



def quadrant_for_position(x, y):
    """Map (x, y) to its enclosing quadrant per the memory mapping."""
    if x < ROOM_W / 2:
        return "Q1" if y > ROOM_L / 2 else "Q3"
    return "Q2" if y > ROOM_L / 2 else "Q4"



def format_position(x, y, info=None):
    q = quadrant_for_position(x, y)
    out = f"[localizer] (x, y) = ({x:.2f}m, {y:.2f}m) -> {q}"
    if info is not None:
        wp = info["wall_perturbations"]
        out += (f"  W/E={wp['west_L12']:.2f}/{wp['east_L34']:.2f}  "
```

```
            f"S/N={wp['south_L14']:.2f}/{wp['north_L23']:.2f}")
    return out



def render_ascii_room(x, y, width=21, height=15, body_char="*"):
    """ASCII map: corners are node IDs 1..4, body is *.

    Coordinate convention: terminal row 0 = North (top of map);
    terminal column 0 = West (left of map). So we invert y for display.
    """
    grid = [[" " for _ in range(width)] for _ in range(height)]

    def to_grid(rx, ry):
        gx = int(round(rx / ROOM_W * (width - 1)))
        gy = int(round((1.0 - ry / ROOM_L) * (height - 1)))
        gx = max(0, min(width - 1, gx))
        gy = max(0, min(height - 1, gy))
        return gx, gy

    for n, (px, py) in NODE_POS.items():
        gx, gy = to_grid(px, py)
        grid[gy][gx] = str(n)

    if x is not None and y is not None:
        gx, gy = to_grid(x, y)
        if grid[gy][gx] in (" ", "."):
            grid[gy][gx] = body_char
```

```
        else:
            for dy in (-1, 0, 1):
                for dx in (-1, 0, 1):
                    if dx == 0 and dy == 0:
                        continue
                    nx, ny = gx + dx, gy + dy
                    if 0 <= nx < width and 0 <= ny < height \
                       and grid[ny][nx] == " ":
                        grid[ny][nx] = body_char
                        break
                else:
                    continue
                break

    border = "+" + "-" * width + "+"
    rows = [border + "  N (top)"]
    for r in grid:
        rows.append("|" + "".join(r) + "|")
    rows.append(border)
    rows.append(f"  W <-- {ROOM_W:.2f}m x {ROOM_L:.2f}m --> E")
    return "\n".join
```

```
"""Pygame display for the deterministic-detector live UDP path.

Pure view layer: receives per-emission state via `update()`, renders a
4-quadrant room map with the current prediction lit up, a body asterisk
+ confidence circle, big headline text, per-link health bars, and the
```

```
starvation/reset operator banners.


Run as a thread alongside live_iter; the main event/render loop polls
the latest state. 'r' keypress sets a reset flag the detector watches.
'q' keypress exits.
"""
from __future__ import annotations


import math
import threading
import time
from typing import Optional


try:
    import pygame
except ImportError:  # graceful fallback when pygame isn't installed
    pygame = None



# Colors
_BG = (12, 12, 18)
_DARK = (40, 40, 48)
_LIT = (40, 200, 100)
_LIT_DIM = (30, 140, 70)
_BORDER = (90, 90, 110)
_TEXT = (220, 220, 220)
_TEXT_MUTED = (160, 160, 170)
```

```
_BAR_BG = (60, 60, 70)
_BAR_FG = (90, 170, 220)
_BAR_STARVED = (210, 80, 80)
_BANNER_WARN = (210, 50, 50)
_BANNER_RESET = (90, 170, 220)
_CIRCLE = (255, 220, 100)
_ASTERISK = (255, 255, 255)

# Quadrant -> compass cell. Q1=NW (node 2), Q2=NE (node 3), Q3=SW
# (node 1), Q4=SE (node 4). Cell coords are (col, row) where col=0
# is left and row=0 is top.
_QUAD_CELL = {
    "Q1": (0, 0),  # NW
    "Q2": (1, 0),  # NE
    "Q3": (0, 1),  # SW
    "Q4": (1, 1),  # SE
}

_LINKS_ORDERED = ["1-2", "1-3", "1-4", "2-3", "2-4", "3-4"]



class PygameDisplay:
    """Thread-safe pygame demo display.

    Lifecycle:
        d = PygameDisplay(width=1280, height=720)
        d.start()        # spawn render thread
```

```
    # ... in detector loop:
    d.update(emission)
    # ... at end:
    d.stop()

Operator controls (rendered in the main pygame window):
    'r' or 'R' -> sets reset_event (read by the detector loop to
                  reset HMM + position smoother)
    'q' or ESC -> sets stop_event (detector should exit)
"""

def __init__(self, width: int = 1280, height: int = 720,
             fullscreen: bool = False,
             room_width_m: float = 3.81,
             room_length_m: float = 9.14,
             show_body_marker: bool = True):
    if pygame is None:
        raise RuntimeError(
            "pygame is not installed. Run: pip install pygame-ce"
        )
    self.width = width
    self.height = height
    self.fullscreen = fullscreen
    self.room_width_m = room_width_m
    self.room_length_m = room_length_m
    self.show_body_marker = show_body_marker
```

```
        self._lock = threading.Lock()
        self._latest = {
            "smoothed_class": None,
            "confidence": 0.0,
            "raw_class": None,
            "bpm": None,
            "active_fraction": None,
            "per_link_active_fraction": None,
            "per_link_record_counts": None,
            "per_link_starve_streak": None,
            "position_xy": None,  # (x, y) in meters, room coords
            "n_bodies": 0,
            "doppler_ready": False,
            "starvation_warning_active": False,
            "last_reset_t": 0.0,
            # RX stats panel: total IQ records and per-node breakdown
            # over the past `window_seconds` of the live stream.
            "n_records_short": None,         # int total, e.g. 522
            "per_node_record_counts": None,  # {1: .., 2: .., 3: .., 4: ..}
            "window_seconds": None,          # float, e.g. 6.0
            # Last-reset panel
            "last_reset_info": None,         # {node_id, reason_label, crash_count, t_wall}
            "per_node_crash_count": None,    # {1: .., 2: .., 3: .., 4: ..}
            "n_total_resets": 0,             # int, since detector start
            "recent_resets": None,           # list of recent reset dicts
        }
        self._thread: Optional[threading.Thread] = None
```

```
    self._stop = threading.Event()
    self.reset_event = threading.Event()
    self.stop_event = threading.Event()

def update(self, **kwargs):
    """Push the latest emission state. Thread-safe."""
    with self._lock:
        for k, v in kwargs.items():
            if k in self._latest:
                self._latest[k] = v

def trigger_reset_visual(self):
    with self._lock:
        self._latest["last_reset_t"] = time.time()

def start(self):
    if self._thread is not None:
        return
    self._thread = threading.Thread(target=self._run, daemon=True)
    self._thread.start()

def stop(self):
    self._stop.set()
    if self._thread is not None:
        self._thread.join(timeout=2.0)

def _run(self):
```

```
    pygame.init()
    if self.fullscreen:
        # On Windows, pygame.display.set_mode((0, 0), FULLSCREEN |
        # SCALED) raises "Cannot set 0 sized SCALED display mode".
        # Query the desktop size and pass it explicitly. SDL's
        # display info is reliable on Win32 + Pi OS.
        try:
            info = pygame.display.Info()
            desktop_w = info.current_w or 1920
            desktop_h = info.current_h or 1080
        except Exception:
            desktop_w, desktop_h = 1920, 1080
        flags = pygame.FULLSCREEN | pygame.SCALED
        screen = pygame.display.set_mode(
            (desktop_w, desktop_h), flags
        )
    else:
        flags = pygame.RESIZABLE
        screen = pygame.display.set_mode(
            (self.width, self.height), flags
        )
    pygame.display.set_caption("Glasshouse v2 — Demo")
    clock = pygame.time.Clock()

    # Fonts scale to screen height
    sw, sh = screen.get_size()
    font_headline = pygame.font.SysFont(None, max(int(sh * 0.10), 36))
```

```
font_quad = pygame.font.SysFont(None, max(int(sh * 0.06), 28))
font_meta = pygame.font.SysFont(None, max(int(sh * 0.035), 18))
font_small = pygame.font.SysFont(None, max(int(sh * 0.025), 14))

# Layout: top 70% = room view + headline, bottom 30% = link bars + status
while not self._stop.is_set():
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            self.stop_event.set()
            self._stop.set()
        elif event.type == pygame.KEYDOWN:
            if event.key in (pygame.K_q, pygame.K_ESCAPE):
                self.stop_event.set()
                self._stop.set()
            elif event.key == pygame.K_r:
                self.reset_event.set()
                self.trigger_reset_visual()
        elif event.type == pygame.VIDEORESIZE:
            sw, sh = event.size
            screen = pygame.display.set_mode(
                (sw, sh), pygame.RESIZABLE
            )
            font_headline = pygame.font.SysFont(
                None, max(int(sh * 0.10), 36)
            )
            font_quad = pygame.font.SysFont(
                None, max(int(sh * 0.06), 28)
```

```
        )
        font_meta = pygame.font.SysFont(
            None, max(int(sh * 0.035), 18)
        )
        font_small = pygame.font.SysFont(
            None, max(int(sh * 0.025), 14)
        )

    with self._lock:
        state = dict(self._latest)

    screen.fill(_BG)
    sw, sh = screen.get_size()

    # Top 70%: room map + headline
    map_h = int(sh * 0.70)
    self._draw_room(screen, state, 0, 0, sw, map_h,
                    font_quad, font_headline, font_meta,
                    font_small)

    # Side panels mirror each other across the room map: the
    # room is portrait (3.81 x 9.14 m), so when it's centered
    # there are wide empty margins on both sides of the screen.
    # We use them: RX RATE on the right, LAST RESET on the
    # left. Each panel gets the full vertical budget between
    # the headline reserve and the link-bars row, so neither
    # has to clip its rows.
```

```
        panel_w = min(300, sw // 4)
        col_top = int(sh * 0.22)        # below headline + sub
        col_bottom = map_h - 12         # above link-bars row
        col_h = max(0, col_bottom - col_top)
        right_panel_x = sw - panel_w - 20
        left_panel_x = 20
        self._draw_rx_stats(screen, state,
                            right_panel_x, col_top,
                            panel_w, col_h,
                            font_meta, font_small)
        self._draw_last_reset(screen, state,
                              left_panel_x, col_top,
                              panel_w, col_h,
                              font_meta, font_small)

        # Bottom 30%: link bars + status
        self._draw_link_bars(screen, state, 0, map_h, sw,
                             sh - map_h, font_meta, font_small)

        # Reset flash (within 1.5s of last reset)
        if state["last_reset_t"]:
            age = time.time() - state["last_reset_t"]
            if age < 1.5:
                self._draw_reset_flash(screen, sw, sh,
                                       age / 1.5,
                                       font_meta)
```

```
        # Starvation warning banner
        if state["starvation_warning_active"]:
            self._draw_warning_banner(screen, sw, sh, font_meta)

        # Reset / quit hint in corner
        hint = font_small.render(
            "R = reset HMM   Q/ESC = quit",
            True, _TEXT_MUTED,
        )
        screen.blit(hint, (sw - hint.get_width() - 10,
                           sh - hint.get_height() - 8))

        pygame.display.flip()
        clock.tick(20)

    try:
        pygame.quit()
    except Exception:
        pass

# ---- drawing helpers ----

def _draw_room(self, screen, state, x0, y0, w, h,
               font_quad, font_headline, font_meta, font_small):
    # Room rectangle in pixels: take the available area, fit a
    # 3.81 x 9.14 m portrait rectangle inside it. North is up.
    # Reserve more vertical space at the top for the headline +
```

```
# BPM line so they don't overlap with the room top edge.
margin = 40
avail_w = w - 2 * margin
head_reserve = int(h * 0.30)  # bigger reserve for headline+bpm
avail_h = h - 2 * margin - head_reserve

# Room aspect = width / length (3.81 / 9.14 = 0.417). Fit
# rectangle to available area.
room_aspect = self.room_width_m / self.room_length_m
if avail_w / max(1, avail_h) > room_aspect:
    # available area wider than room ratio -> bound by height
    room_h = avail_h
    room_w = int(room_h * room_aspect)
else:
    room_w = avail_w
    room_h = int(room_w / room_aspect)
room_x = x0 + (w - room_w) // 2
room_y = y0 + head_reserve + (avail_h - room_h) // 2

# Headline at top of map area
smoothed = state["smoothed_class"]
conf = state["confidence"] or 0.0
if smoothed:
    head_text = f"{smoothed}  ({int(conf * 100)}% confidence)"
    head_color = _LIT
else:
    head_text = f"ANALYZING  ({int(conf * 100)}%)"
```

```
    head_color = _TEXT_MUTED
surf = font_headline.render(head_text, True, head_color)
screen.blit(surf, (x0 + (w - surf.get_width()) // 2,
                   y0 + int(h * 0.05)))

# Sub-line: BPM
bpm = state["bpm"]
bpm_text = (f"breathing peak: {bpm:.0f} BPM"
            if bpm else "no breathing peak detected")
s2 = font_meta.render(bpm_text, True, _TEXT_MUTED)
screen.blit(s2, (x0 + (w - s2.get_width()) // 2,
                 y0 + int(h * 0.16)))

# Draw 4-quadrant room
cell_w = room_w // 2
cell_h = room_h // 2
for q, (col, row) in _QUAD_CELL.items():
    cx = room_x + col * cell_w
    cy = room_y + row * cell_h
    color = _DARK
    if smoothed == q:
        # Pulse the lit quadrant for visual presence
        pulse = (math.sin(time.time() * 3.0) + 1.0) * 0.5
        lit = tuple(int(_LIT_DIM[i]
                        + pulse * (_LIT[i] - _LIT_DIM[i]))
                    for i in range(3))
        color = lit
```

```
        pygame.draw.rect(screen, color, (cx, cy, cell_w, cell_h))
        pygame.draw.rect(screen, _BORDER, (cx, cy, cell_w, cell_h),
                         3)
        label = font_quad.render(q, True, _TEXT)
        screen.blit(label, (cx + 12, cy + 12))

    # Body asterisk + confidence circle
    pos = state["position_xy"]
    if pos and smoothed and self.show_body_marker:
        px_m = pos[0]
        py_m = pos[1]
        # Map (x in [0, room_width], y in [0, room_length]) to
        # pixel coords. y=0 is south (bottom), y=room_length is
        # north (top). x=0 is west (left), x=room_width is east
        # (right).
        px = room_x + int(px_m / self.room_width_m * room_w)
        py = room_y + int(
            (1.0 - py_m / self.room_length_m) * room_h
        )
        # Confidence-radius circle (large = uncertain, small = sure)
        base_r = int(min(room_w, room_h) * 0.18)
        r = max(8, int(base_r * (1.0 - conf)))
        # Draw soft glow circle
        for i, alpha in [(r + 12, 30), (r + 6, 60), (r, 120)]:
            surf = pygame.Surface((i * 2, i * 2),
                                  pygame.SRCALPHA)
            pygame.draw.circle(
```

```
            surf, (*_CIRCLE, alpha), (i, i), i
        )
        screen.blit(surf, (px - i, py - i))
    # Asterisk dot
    pygame.draw.circle(screen, _ASTERISK, (px, py), 6)

def _draw_rx_stats(self, screen, state, x0, y0, w, h,
                   font_meta, font_small):
    """Floating RX-rate panel.

    Big total IQ-record count over the last short window, plus
    per-node mini-bars. Each node's bar color matches its
    quadrant so the operator can read both rate health and
    spatial coverage in one glance.
    """
    n_total = state.get("n_records_short")
    per_node = state.get("per_node_record_counts") or {}
    window_s = state.get("window_seconds")
    if n_total is None and not per_node:
        return  # warming up: no data yet

    # Card background (semi-transparent rounded panel)
    card = pygame.Surface((w, h), pygame.SRCALPHA)
    card.fill((22, 22, 30, 210))
    pygame.draw.rect(card, _BORDER, (0, 0, w, h), 2,
                     border_radius=12)
    screen.blit(card, (x0, y0))
```

```
pad = 16
yy = y0 + pad

# Header line
hdr = font_meta.render("RX RATE", True, _TEXT)
screen.blit(hdr, (x0 + pad, yy))
yy += hdr.get_height() + 1

if window_s is not None:
    sub = font_small.render(f"last {window_s:.1f}s",
                            True, _TEXT_MUTED)
    screen.blit(sub, (x0 + pad, yy))
    yy += sub.get_height() + 6

# Big total number
if n_total is not None:
    big_size = max(int(h * 0.20), 44)
    big_font = pygame.font.SysFont(None, big_size, bold=True)
    big_color = _LIT if n_total > 0 else _TEXT_MUTED
    total_surf = big_font.render(f"{n_total:,}",
                                 True, big_color)
    screen.blit(total_surf, (x0 + pad, yy))
    yy += total_surf.get_height() - 6
    rec_lbl = font_small.render("IQ records", True,
                                _TEXT_MUTED)
    screen.blit(rec_lbl, (x0 + pad, yy))
```

```
    yy += rec_lbl.get_height() + 10

# Divider
pygame.draw.line(screen, _BORDER,
                 (x0 + pad, yy),
                 (x0 + w - pad, yy), 1)
yy += 10

# Per-node mini-bars. Color each bar by the node's quadrant so
# the operator gets a spatial read at the same time as a rate
# read. Node 1=Q3, 2=Q1, 3=Q2, 4=Q4 — palette below.
per_node_colors = {
    1: (170, 110, 220),   # Q3: violet
    2: (90, 170, 220),    # Q1: blue
    3: (240, 180, 90),    # Q2: amber
    4: (220, 110, 110),   # Q4: red-pink
}
per_node_label = {
    1: "N1 / Q3",
    2: "N2 / Q1",
    3: "N3 / Q2",
    4: "N4 / Q4",
}

nmax = max(per_node.values()) if per_node else 0
# Reserve space: label (left) ~ 70px, count (right) ~ 40px,
# bar in between.
```

```
    label_w = 72
    count_w = 44
    bar_x = x0 + pad + label_w
    bar_max_w = w - pad * 2 - label_w - count_w

    for nid in (1, 2, 3, 4):
        if yy + 22 > y0 + h - pad:
            break  # don't overflow card
        cnt = per_node.get(nid, 0)
        # Label
        lbl = font_small.render(per_node_label[nid], True, _TEXT)
        screen.blit(lbl, (x0 + pad, yy + 2))
        # Bar background
        b_h = 10
        b_y = yy + 5
        pygame.draw.rect(screen, _BAR_BG,
                         (bar_x, b_y, bar_max_w, b_h),
                         border_radius=3)
        # Bar fill (proportional to max across nodes — relative,
        # not absolute, because absolute capacity changes with
        # window length and per-node packet rate)
        if nmax > 0 and cnt > 0:
            fw = max(2, int(bar_max_w * cnt / nmax))
            pygame.draw.rect(screen, per_node_colors[nid],
                             (bar_x, b_y, fw, b_h),
                             border_radius=3)
        # Count (right-aligned within the row)
```

```
        cnt_surf = font_small.render(f"{cnt}", True, _TEXT)
        screen.blit(cnt_surf,
                    (x0 + w - pad - cnt_surf.get_width(),
                     yy + 2))
        yy += lbl.get_height() + 8

def _draw_last_reset(self, screen, state, x0, y0, w, h,
                     font_meta, font_small):
    """Most-recent-reboot panel.

    Pulls last_reset_info (populated when the detector sees a
    telemetry frame whose uptime regressed) and renders a compact
    forensic card: which node rebooted, ESP-IDF reason label,
    wall-clock age, total reset count this run, and per-node
    crash_count from NVS.
    """
    info = state.get("last_reset_info")
    per_node_crash = state.get("per_node_crash_count") or {}
    n_total = state.get("n_total_resets") or 0

    # Card background
    card = pygame.Surface((w, h), pygame.SRCALPHA)
    # Card tint: amber when there's a recent reset, dark when none
    recent = (info is not None
              and (time.time() - info.get("t_wall", 0.0)) < 60.0)
    if recent:
        card.fill((60, 30, 20, 220))
```

```
        border = (220, 130, 60)
    else:
        card.fill((22, 22, 30, 210))
        border = _BORDER
    pygame.draw.rect(card, border, (0, 0, w, h), 2,
                     border_radius=12)
    screen.blit(card, (x0, y0))

    pad = 16
    yy = y0 + pad

    # Header
    hdr_color = (255, 180, 80) if recent else _TEXT
    hdr = font_meta.render("LAST RESET", True, hdr_color)
    screen.blit(hdr, (x0 + pad, yy))
    # Total reset count, right-aligned in header row
    if n_total > 0:
        tot = font_small.render(f"{n_total} this run",
                                True, _TEXT_MUTED)
        screen.blit(tot,
                    (x0 + w - pad - tot.get_width(),
                     yy + 3))
    yy += hdr.get_height() + 6

    if info is None:
        none_lbl = font_small.render("No resets seen",
                                     True, _TEXT_MUTED)
```

```
    screen.blit(none_lbl, (x0 + pad, yy))
    return

nid = info.get("node_id")
reason = info.get("reason_label") or "Unknown"
ccount = info.get("crash_count")
t_wall = info.get("t_wall", 0.0)
age_s = max(0.0, time.time() - t_wall)

# Big line: "N4 · Brownout"
big_size = max(int(h * 0.22), 26)
big_font = pygame.font.SysFont(None, big_size, bold=True)
big_color = (255, 200, 90) if recent else _TEXT
big_text = f"N{nid}  {reason}" if nid else reason
big_surf = big_font.render(big_text, True, big_color)
# Truncate if too wide
if big_surf.get_width() > w - pad * 2:
    # Shrink to fit
    shrink_size = big_size
    while (big_surf.get_width() > w - pad * 2
           and shrink_size > 14):
        shrink_size -= 2
        big_font = pygame.font.SysFont(None, shrink_size,
                                       bold=True)
        big_surf = big_font.render(big_text, True, big_color)
screen.blit(big_surf, (x0 + pad, yy))
yy += big_surf.get_height() + 4
```

```
# Sub: "12s ago · crash_count=3"
if age_s < 60:
    age_str = f"{int(age_s)}s ago"
elif age_s < 3600:
    age_str = f"{int(age_s / 60)}m ago"
else:
    age_str = f"{int(age_s / 3600)}h ago"
ccount_str = (f" · crash_count={ccount}"
              if ccount is not None else "")
sub = font_small.render(f"{age_str}{ccount_str}",
                        True, _TEXT_MUTED)
screen.blit(sub, (x0 + pad, yy))
yy += sub.get_height() + 6

# Per-node crash_count one-line summary, only if any > 0.
crashes = [(n, c) for n, c in sorted(per_node_crash.items())
           if c and c > 0]
if crashes and (yy + 14) <= y0 + h - pad:
    crashes_str = "  ".join(f"N{n}:{c}" for n, c in crashes)
    cc = font_small.render(f"NVS crashes  {crashes_str}",
                           True, _TEXT_MUTED)
    screen.blit(cc, (x0 + pad, yy))
    yy += cc.get_height() + 12

# Recent reboot history. Renders as a list of one-line entries
# under a small divider, oldest at top -> newest at bottom.
```

```
    # Skips the most-recent entry (already shown above as the
    # headline line) so the user doesn't read the same event twice.
    recent = state.get("recent_resets") or []
    if len(recent) > 1 and (yy + 30) <= y0 + h - pad:
        pygame.draw.line(screen, _BORDER,
                         (x0 + pad, yy),
                         (x0 + w - pad, yy), 1)
        yy += 8
        hdr2 = font_small.render("Recent reboots",
                                 True, _TEXT_MUTED)
        screen.blit(hdr2, (x0 + pad, yy))
        yy += hdr2.get_height() + 6
        now_t = time.time()
        # Show all but the most-recent (which is the headline),
        # newest-first (drop the last/headline entry, then reverse).
        history = list(recent[:-1])
        history.reverse()
        for item in history:
            if yy + 18 > y0 + h - pad:
                break  # don't overflow
            _nid = item.get("node_id")
            _reason = item.get("reason_label") or "Unknown"
            _t = item.get("t_wall", 0.0)
            _age = max(0.0, now_t - _t)
            if _age < 60:
                _age_str = f"{int(_age)}s"
            elif _age < 3600:
```

```
            _age_str = f"{int(_age / 60)}m"
        else:
            _age_str = f"{int(_age / 3600)}h"
        # Left side: Nx · Reason   Right side: 12s
        left_str = f"N{_nid}  {_reason}"
        left_surf = font_small.render(left_str, True, _TEXT)
        right_surf = font_small.render(_age_str,
                                       True, _TEXT_MUTED)
        # Truncate left if it would overlap right
        max_left_w = w - pad * 2 - right_surf.get_width() - 12
        if left_surf.get_width() > max_left_w:
            # crude truncation: shrink reason label to fit
            while (left_surf.get_width() > max_left_w
                   and len(left_str) > 4):
                left_str = left_str[:-1]
                left_surf = font_small.render(left_str + "…",
                                              True, _TEXT)
        screen.blit(left_surf, (x0 + pad, yy))
        screen.blit(right_surf,
                    (x0 + w - pad - right_surf.get_width(),
                     yy))
        yy += left_surf.get_height() + 4

def _draw_link_bars(self, screen, state, x0, y0, w, h,
                    font_meta, font_small):
    # Title
    title = font_meta.render("Per-link breathing activity",
```

```
                        True, _TEXT)
    screen.blit(title, (x0 + 20, y0 + 8))

    af = state["per_link_active_fraction"] or {}
    pkts = state["per_link_record_counts"] or {}
    streak = state["per_link_starve_streak"] or {}

    bar_area_x = x0 + 20
    bar_area_y = y0 + int(h * 0.30)
    bar_area_w = w - 40
    bar_area_h = int(h * 0.55)

    n = len(_LINKS_ORDERED)
    bar_w = (bar_area_w - (n - 1) * 16) // n
    for i, link in enumerate(_LINKS_ORDERED):
        bx = bar_area_x + i * (bar_w + 16)
        by = bar_area_y
        af_v = (af.get(link) or 0.0)
        streak_v = (streak.get(link) or 0)
        n_pkts = pkts.get(link, 0)
        # Background bar
        pygame.draw.rect(screen, _BAR_BG,
                         (bx, by, bar_w, bar_area_h))
        # Foreground (active_fraction filled bar)
        fill_h = int(bar_area_h * min(1.0, af_v))
        color = _BAR_STARVED if streak_v > 30 else _BAR_FG
        if fill_h > 0:
```

```
        pygame.draw.rect(
            screen, color,
            (bx, by + (bar_area_h - fill_h), bar_w, fill_h)
        )
    # Link label
    lbl = font_small.render(f"L{link.replace('-', '')}",
                            True, _TEXT)
    screen.blit(lbl, (bx + (bar_w - lbl.get_width()) // 2,
                      by + bar_area_h + 4))
    # Active fraction %
    af_pct = font_small.render(f"{af_v*100:.0f}%",
                               True, _TEXT_MUTED)
    screen.blit(af_pct,
                (bx + (bar_w - af_pct.get_width()) // 2,
                 by - af_pct.get_height() - 2))
    # Starvation streak indicator
    if streak_v > 0:
        streak_color = (_BANNER_WARN
                        if streak_v > 30 else _TEXT_MUTED)
        s_lbl = font_small.render(
            f"sv {streak_v}", True, streak_color
        )
        screen.blit(s_lbl,
                    (bx + (bar_w - s_lbl.get_width()) // 2,
                     by + bar_area_h + 4
                     + lbl.get_height() + 2))
```

```
def _draw_warning_banner(self, screen, sw, sh, font_meta):
    msg = ">>> RESTART COORD recommended (sustained link starvation)"
    s = font_meta.render(msg, True, (255, 255, 255))
    bw = s.get_width() + 30
    bh = s.get_height() + 10
    bx = (sw - bw) // 2
    by = 10
    # Pulse
    pulse = (math.sin(time.time() * 4.0) + 1.0) * 0.5
    color = tuple(int(_BANNER_WARN[i] * (0.5 + 0.5 * pulse))
                  for i in range(3))
    pygame.draw.rect(screen, color, (bx, by, bw, bh))
    screen.blit(s, (bx + 15, by + 5))

def _draw_reset_flash(self, screen, sw, sh, t, font_meta):
    msg = "RESET — HMM cleared"
    s = font_meta.render(msg, True, (255, 255, 255))
    bw = s.get_width() + 30
    bh = s.get_height() + 12
    bx = (sw - bw) // 2
    by = sh - bh - 60
    alpha = int(255 * (1.0 - t))
    surf = pygame.Surface((bw, bh), pygame.SRCALPHA)
    surf.fill((*_BANNER_RESET, alpha))
    screen.blit(surf, (bx, by))
    screen.blit(s, (bx + 15, by + 6))
```

```
__all__ = ["PygameDisplay"]
```

Appendix B. Circuit Schematics, Wiring Diagrams, and PCB Layouts

This appendix documents the electrical design of the perimeter node, the coordinator node, and the central device. Each subsection has a wiring diagram and a net-list summary that a third party could use to reproduce the circuit. The wiring diagram for the perimeter-node PCB layout are mirrored in the public repository identified in Section 12.4 (https://github.com/jackjones1320/Project-Glass-House) under the hardware/ directory.

B.1 Perimeter Node, Wiring Diagram and Net List

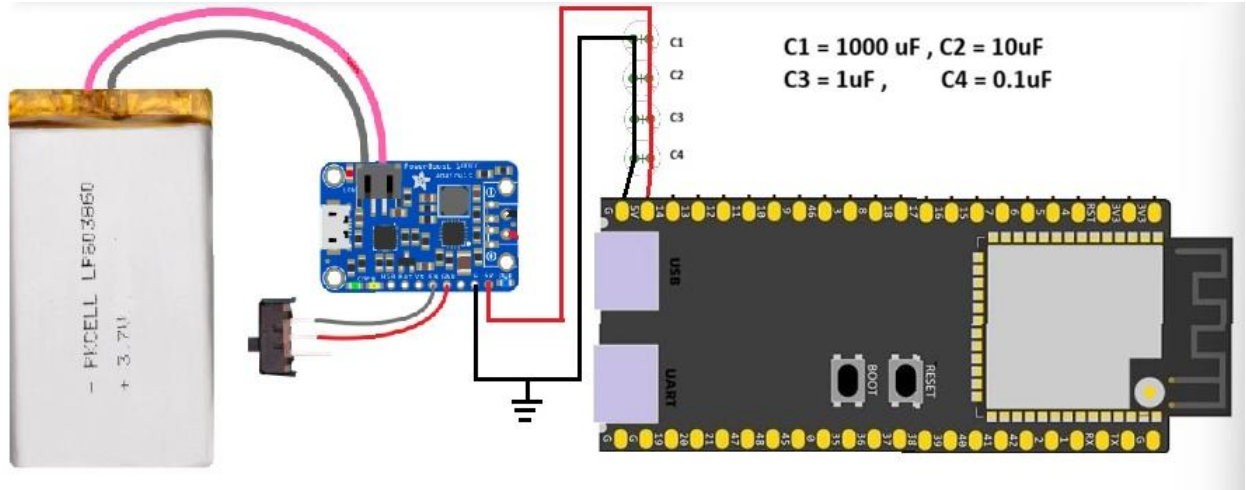


Figure B-1. Perimeter node wiring diagram, showing the LiPo battery, Adafruit PowerBoost 1000C charge and boost module, ESP32-S3 microcontroller (S3-N16R8), 2.4 GHz 7 dBi SMA antenna, and DPDT power switch.

Net	Source pin	Destination pin	Notes

BATT+	LiPo JST-PH +	PowerBoost 1000C BAT	3.7 V nominal. Current-limited by the integrated TPS61090 boost converter (no external polyfuse / PTC fitted).
BATT-	LiPo JST-PH -	PowerBoost 1000C GND	
SW	PowerBoost 1000C EN	SPDT switch common	High enables boost
SW_HIGH	DPDT switch normally-open	PowerBoost 1000C 5V	Pulls EN to 5 V to enable
5V	PowerBoost 1000C 5V	ESP32-S3 USB header +5V	5.0 V regulated, 1.0 A peak
5V_BULK	5V rail	GND	1000 uF aluminium electrolytic bulk capacitor for transient transmit bursts (see paragraph below)
5V_DECAP	5V / 3V3 rail	GND	10 uF + 1 uF + 0.1 uF MLCC ceramic decoupling stack at the ESP32-S3 supply pins
GND	PowerBoost 1000C GND	ESP32-S3 GND	Common ground
ANT	ESP32-S3 module RF pad	SMA pigtail center	2.4 GHz, 50 ohm, 50 mm or shorter trace
ANT_GND	ESP32-S3 module GND ring	SMA pigtail shield	RF return
USB_DBG	ESP32-S3 native USB-C	(open during deployment)	Used for flashing and serial debug only

Power budget at peak transmit. PowerBoost output of 5.0 V at about 280 mA is roughly 1.4 W. PowerBoost efficiency near 90 percent implies about 1.55 W draw from the LiPo. A 2,000 mAh cell at 3.7 V holds a nominal 7.4 Wh, which gives at least 4.5 hours of continuous transmit per node.

Protection features (Adafruit PowerBoost 1000C, integrated). The PowerBoost 1000C combines a TPS61090 synchronous boost converter and an MCP73871 single-cell LiPo charge controller on a single carrier board. The protection features used by this design are all integrated on that

module: battery undervoltage lockout at approximately 3.0 V to protect the LiPo cell from over-discharge, soft-start on the boost output to limit inrush current at power-on, an integrated current limit on the boost output (approximately 1 A continuous on the TPS61090) so no external polyfuse or PTC is required and none is fitted, reverse-battery protection on the LiPo input, thermal shutdown on the boost converter, output short-circuit protection, a low-battery LED indicator at approximately 3.2 V, and load-sharing between USB power and the LiPo cell so the battery is not discharged while the node is being charged from USB. Because all of these protections are integrated into the PowerBoost module itself, no external linear regulator (LDO) and no resettable polyfuse (PTC) appear in the perimeter node power tree.

Bulk capacitor on the 5 V rail (1000 uF electrolytic). The ESP32-S3 draws transient supply-current bursts of up to roughly 250 to 300 mA whenever the 802.11 power amplifier is active, which is every NDP transmission at the 50 Hz CSI emission rate. A 1000 uF aluminium electrolytic capacitor across the 5 V rail acts as a local energy reservoir that supplies these transient bursts directly, so the boost converter does not have to slew its output current instantaneously. Without this bulk decoupling, supply rail droop during transmit bursts would briefly pull the rail below the ESP32-S3 brown-out detector threshold and cause the node to reset mid-CSI capture. This is the same symptom that ended the bare-LiPo bring-up build described in Section 13.10.

Wideband ceramic decoupling stack (10 uF, 1 uF, 0.1 uF MLCC). A standard three-cap MLCC stack is placed close to the ESP32-S3 supply pins to keep the supply rail low-impedance across a wide frequency band. Each value targets a different impedance band: the 10 uF cap covers low-frequency decoupling in the kHz range and supplements the bulk electrolytic; the 1 uF cap covers mid-frequency decoupling in the hundreds-of-kHz to low-MHz range; and the 0.1 uF cap covers high-frequency decoupling in the multi-MHz range, including the approximately 1.5 MHz switching ripple from the TPS61090 boost converter and digital switching noise from the ESP32-S3 itself. Together they keep the supply rail clean during fast digital edges and minimize conducted and radiated EMI back into the 2.4 GHz CSI receiver, which is the same RF chain the system relies on to detect millivolt-scale breathing-band perturbations.

Appendix C. User Manual

C.1 Overview

Project Glasshouse is a distributed radio-frequency sensing system designed to detect and localize human presence within a monitored area without requiring line-of-sight access. The system consists of four battery-powered perimeter sensing nodes, one coordinator node, a Raspberry Pi 4 central processing hub, and a 7-inch LCD touchscreen display. When deployed correctly, the system displays which of four spatial quadrants within the monitored area is most likely occupied, providing first responders with an actionable localization estimate without entering an unstable structure.

This manual covers all steps required to prepare, deploy, operate, interpret, power down, and maintain the system. Read all sections before first use.

C.2 System Components

The complete Project Glasshouse kit contains the following items. Verify all components are present before deployment.

- Four (4) perimeter sensing nodes – gray 3D-printed PLA enclosures, each containing an ESP32-S3 microcontroller, Adafruit PowerBoost 1000C charge module, and LiPo battery. Each node has a slide power switch on the side wall and a 7 dBi SMA antenna attached to the top.
- One (1) central device enclosure – housing the Raspberry Pi 4, coordinator ESP32-S3, USB power bank, and 7-inch IPS LCD touchscreen. The enclosure has a slide power switch, a push-button labeled "PUSH THIS FIRST WHEN TURNING OFF", and a USB-C charging port.
- One (1) USB-C charging cable for the central device power bank.
- Four (4) USB-A to USB-C cables for charging the perimeter node LiPo batteries.
- One (1) carry bag or case for transport.

C.3 Before Deployment – Battery Charging

All batteries must be fully charged before each deployment. A partially charged node may shut down mid-operation, which reduces system accuracy.

To charge the perimeter nodes:

1. Ensure each node's slide power switch is in the OFF position.
2. Connect a USB-A to USB-C cable from any USB wall charger or power bank to the USB-C port on the Adafruit PowerBoost module, accessible through the side opening of the enclosure.
3. The PowerBoost LED will glow red while charging and turn blue when the battery is fully charged.
4. Charging time from empty is approximately two to three hours per node.

5. Disconnect the charging cable before deployment.

To charge the central device power bank:

1. Connect the USB-C charging cable to the power bank's input port, accessible on the exterior of the central device enclosure.
2. Charge until the power bank indicator shows a full charge.
3. Disconnect the charging cable before deployment.

C.4 Perimeter Node Deployment

The four perimeter nodes define the sensing enclosure. Their placement determines the quadrant layout that appears on the display. Correct placement is critical to accurate localization.

Node placement guidelines:

1. Place the four nodes at the corners of the area to be monitored, forming a rectangle or square. The monitored area should be sized to the region where a victim is suspected – a typical room-sized deployment of approximately 4 m × 4 m to 6 m × 6 m produces the best results.
2. Orient each node so that its antenna points upward and is not obstructed by the enclosure walls or surrounding debris.
3. Nodes are labeled 1 through 4 on their lids. Place Node 1 at the front-left corner, Node 2 at the front-right corner, Node 3 at the rear-right corner, and Node 4 at the rear-left corner, as viewed from the operator position. This layout corresponds to the quadrant map shown on the display.
4. Keep the nodes at approximately the same height where possible. Placing all nodes on the ground or all nodes at waist height produces more consistent results than mixing heights.
5. The central device enclosure does not need to be placed inside the monitored area and should be positioned outside, at the operator's station, within USB cable reach of the coordinator connection.

C.5 System Startup Procedure

Follow this startup sequence in order. Powering nodes before the central device is ready may result in missed Wi-Fi association and require a restart.

1. Power on the central device by sliding its power switch to the ON position. The Raspberry Pi will begin booting. Wait for the LCD display to show the four-quadrant room map before proceeding. Boot time is approximately 30 to 45 seconds.
2. Once the display is active and showing the quadrant map, power on each perimeter node by sliding its side-wall switch to the ON position. Power on all four nodes within 30 seconds of each other.

3. Each node's ESP32-S3 will automatically connect to the Wi-Fi SoftAP network hosted by the coordinator inside the central device enclosure. No manual network configuration is required.

4. Within approximately 10 to 20 seconds of powering the nodes, the display will begin updating as CSI data flows from the nodes to the coordinator and through to the Raspberry Pi pipeline.

5. Allow the system to run for at least 30 seconds before interpreting the display output. The 90-second long analysis window requires time to accumulate sufficient CSI data for reliable breathing-band detection.

6. The system is ready for use when the display shows stable quadrant highlights that respond consistently to the presence of a person within the monitored area.

C.6 Display Interpretation

The LCD display shows a top-down schematic map of the monitored area divided into four quadrants corresponding to the four node positions. The display updates in real time at approximately 20 Hz.

1. A quadrant highlighted in green indicates that the system has detected a breathing-band RF signature and has attributed it to that quadrant. This is the system's best estimate of where a survivor is located.

2. All quadrants shown in gray indicate that no occupancy has been detected in the current emission window. This may mean the area is unoccupied, the subject is stationary and not producing a detectable breathing signature, or insufficient CSI data has accumulated since startup.

3. A body marker position derived from the continuous (x, y) wall-pair perturbation estimate is displayed alongside the quadrant highlight and provides a finer-grained indication of probable subject location within the highlighted quadrant.

4. Per-link breathing-band activity bars are shown at the bottom of the display. These indicate the signal level detected on each of the six node-pair links. High activity on links adjacent to a particular quadrant is consistent with occupancy in that quadrant.

5. The system is sensitive to movement and respiration near but outside the node perimeter. If the display highlights a quadrant persistently but no subject is believed to be inside the enclosure, check whether personnel or equipment are positioned close to the outside of the node boundary and move them away before re-evaluating.

C.7 Operating Notes and Best Practices

1. Establish a quiet perimeter. During sensing, all non-essential personnel should stand at least two meters outside the node boundary. Movement by operators near the enclosure can produce false quadrant attributions.

2. Minimize RF interference. Avoid operating the system in environments with dense active 2.4 GHz Wi-Fi traffic if possible. If interference is suspected, power cycle all nodes and the central device and allow the pipeline to re-establish its rolling baselines.

3. Do not block the antennas. Ensure that node antennas are not pressed against walls, debris, or other objects. Antenna obstruction reduces CSI quality and degrades localization accuracy.

4. Allow stabilization time. After a subject moves from one quadrant to another, allow 10 to 15 seconds for the display to stabilize before recording the new quadrant estimate. The rolling analysis window requires time to reflect the change.

5. Battery runtime. Each fully charged perimeter node provides approximately two to four hours of continuous operation. The central device power bank provides approximately four to six hours. Monitor the node LED indicators – a flickering or dimming LED on the PowerBoost module indicates a low battery. Power off that node, swap in a charged spare if available, and restart the node.

C.8 System Shutdown Procedure

Always follow the shutdown sequence below to prevent data corruption on the Raspberry Pi microSD card.

1. Press the push-button labeled “PUSH THIS FIRST WHEN TURNING OFF” on the front face of the central device enclosure. This initiates a safe software shutdown of the Raspberry Pi. The LCD display will go dark within 10 to 15 seconds.

2. After the display has gone dark and the Raspberry Pi status LED has stopped blinking, slide the central device power switch to the OFF position.

3. Power off each perimeter node by sliding its side-wall switch to the OFF position.

4. If the system needs to be shut down immediately without following the safe shutdown procedure (emergency or power failure), slide all power switches to OFF. Note that this may require the microSD card to be re-imaged if file system corruption occurs.

C.9 Troubleshooting

Display shows no quadrant highlight after 90 seconds:

Confirm that all four perimeter nodes are powered on and that their LEDs are active. If a node LED is off, check that its battery is charged and its switch is in the ON position. Confirm that no

dense 2.4 GHz RF interference is present in the environment. If the issue persists, power cycle all nodes and wait 90 seconds for the analysis window to fill.

Display is frozen or shows no output:

Follow the safe shutdown procedure in Section C.8 and restart the system from Section C.5. If the display fails to appear after boot, check that the HDMI cable between the Raspberry Pi and the LCD is securely connected.

Node does not appear to connect to the network:

Ensure the central device was powered on and fully booted before the perimeter nodes were powered on. Power off the affected node, wait ten seconds, and power it back on. The ESP32-S3 will reattempt Wi-Fi association automatically.

Display highlights change rapidly between quadrants:

This behavior typically indicates that personnel or equipment are moving near the outside of the node perimeter. Establish a quiet perimeter as described in Section C.7 and allow the display to stabilize.

C.10 Maintenance

1. After each deployment, charge all perimeter node batteries and the central device power bank to full before returning the system to storage.
2. Inspect antenna SMA connectors on each node after deployment. Tighten any connector that has become loose by hand-turning the SMA nut clockwise. Do not use tools, which may over-tighten and damage the connector.
3. Inspect the 3D-printed PLA enclosures for cracks or damage after each deployment, particularly at the lid snap-fit points. Damaged enclosures should be reprinted before the next deployment. SolidWorks source files are available in the project repository.
4. The Raspberry Pi microSD card contains the operating system and the signal processing pipeline. Do not remove the microSD card during operation. If the card becomes corrupted, re-image it using the latest release image from the project repository and the instructions provided therein.
5. LiPo batteries that no longer hold a charge or that show signs of swelling must be removed from service immediately and disposed of at a certified battery recycling facility. Do not puncture, incinerate, or discard LiPo cells in general waste.

Appendix D. IEEE Code of Ethics

To uphold the highest standards of integrity, responsible behavior, and ethical conduct in professional activities, IEEE members commit to:

1. Hold paramount the safety, health, and welfare of the public; strive to comply with ethical design and sustainable development practices; protect the privacy of others; and disclose promptly factors that might endanger the public or the environment.
2. Improve the understanding by individuals and society of the capabilities and societal implications of conventional and emerging technologies, including intelligent systems.
3. Avoid real or perceived conflicts of interest whenever possible, and disclose them to affected parties when they do exist.
4. Avoid unlawful conduct in professional activities, and reject bribery in all its forms.
5. Seek, accept, and offer honest criticism of technical work; acknowledge and correct errors; be honest and realistic in stating claims or estimates based on available data; and credit properly the contributions of others.
6. Maintain and improve technical competence and undertake technological tasks only if qualified by training or experience, or after full disclosure of pertinent limitations.
7. Treat all persons fairly and with respect; not engage in discrimination based on characteristics such as race, religion, gender, disability, age, national origin, sexual orientation, gender identity, or gender expression.
8. Not engage in harassment of any kind, including sexual harassment or bullying behavior.
9. Avoid injuring others, their property, reputation, or employment by false or malicious actions, rumors, or any verbal or physical abuses.
10. Support colleagues and co-workers in following this code of ethics; strive to ensure the code is upheld; and not retaliate against individuals reporting a violation.